# TÀI LIỆU KHÓA HỌC

## Java Spring REST APIs - Vibe Coding Edition
## Build với AI, Hiểu như Senior

Tác giả: Hỏi Dân IT @hoidanit

Version: 1.0

**Overview** **4**
Tổng Quan Về Khóa Học 4
Source Code Khóa Học (Sử Dụng AI) 6
Về Quyền Tác Giả 7
Về Tác Giả Hỏi Dân IT 7
**Module 0: Mindset Shift - Thích Nghi Trong Kỷ Nguyên AI** **8**
#0.1 AI Đã Thay Đổi Cách Học Code Như Thế Nào ? 8
#0.2 Vibe Coding Là Gì? Fundamental-driven Approach 11
#0.3 Workflow: Prompt → Generate → Understand → Verify 14
#0.4 Khi Nào Không Nên Dùng AI 18
#0.5 Kinh Nghiệm Sử Dụng Trợ Lý AI 23
#0.6 Prompting Fundamentals Cho Developers 26
**Module 1: Spring Boot Big Picture** **31**
#1.1 Setup Môi Trường Thực Hành & Tài Liệu Java Spring 31
#1.2 [AI] Generate Spring Project (Vibe Coding) 33
#1.3 [AI] Hello World với Spring Boot 34
#1.4 Cách Setup Dự Án Thực Hành (Extra) 35
#1.5 Phân Tích Cấu Trúc Dự Án Spring Boot 36
#1.6 Spring Boot là gì? Auto-configuration explained 37
#1.7 Annotations Cơ Bản - Đọc Hiểu Code AI Generate 39
#1.8 Chạy Ứng Dụng với Spring Boot DevTools 40
**Module 2: Dependency Injection** **41**
#2.1 Xây Dựng Cấu Trúc Project Chuẩn 41
#2.2 [AI] Tạo User CRUD với hardcode data 43
#2.3 Dependency Injection là gì? Vấn đề nó giải quyết 45
#2.4 IoC Container - Behind the scenes 48
#2.5 @Autowired, @Component, @Service trong Spring 51
#2.6 Constructor vs Field Injection - Best Practice 53
#2.7 [AI] Đọc Code AI Generate - Identify DI Patterns 55
**Module 3: Spring Data JPA** **56**
#3.1 Cài đặt MySQL và Công Cụ Quản Lý 56
#3.2 Cấu hình Spring Boot kết nối MySQL 58
#3.3 ORM là gì ? Tại sao cần? 59
#3.4 Database Design Thinking 61
#3.5 Thiết Kế Database Khóa Học 63
#3.6 JPA và Hibernate trong Spring 66
#3.7 [AI] Tạo Entity đầu tiên - User 67
#3.8 Repository Pattern với JpaRepository 70
#3.9 [AI] Refactor UserRepository - Từ hardcode sang JPA 72
**Module 4: RESTful APIs** **73**
#4.1 RESTful API là gì? Principles & Constraints 73
#4.2 HTTP Methods, Status Codes & Conventions 77
#4.3 API Design Best Practices 80

@hoidanit

#4.4 Controller Layer — Annotations, Path Variable, Request Body & Request Param 82
#4.5 [AI] Response Entity - Kiểm Soát HTTP Response 83
#4.6 [AI] Response Format Chuẩn với ApiResponse 85
#4.7 [AI] Exception Handling - Xử Lý Lỗi Tập Trung 88
#4.8 Validation với @Valid 91
**Module 5: Spring Security 94**
#5.1 Phân biệt Authentication vs Authorization 94
#5.2 So Sánh Session vs Opaque Token vs JWT (JSON Web Token) 96
#5.3 JWT Deep Dive: Flow & Security Concerns 100
#5.4 [AI Warning] Tại Sao Không Nên Vibe Coding Security 105
#5.5 Cấu hình Spring Security cơ bản 108
#5.6 [AI] Filter Chain & Disable Security 110
#5.7 [AI] Password Encoding với BCrypt 111
#5.8 [AI] UserDetailsService - Login từ Database 114
#5.9 Phân biệt OAuth2 Resource Server vs OAuth2 Authorization Server 117
#5.10 [AI] Cấu hình JWT với OAuth2 Resource Server 120
#5.11 [AI] Login Endpoint - Trả về Access Token 122
**Module 6: Automation Testing (JUnit & Mockito) 126**
#6.1 Testing Mindset - Tại sao cần viết test ? 126
#6.2 Unit Test vs Integration Test vs E2E 128
#6.3 Cấu Trúc Test Trong Spring Boot 130
#6.4 [AI] Viết Unit Test cho Service Layer 132
#6.5 [AI] Viết Integration Test cho API 135
#6.6 Review & Đánh Giá Chất Lượng Test 137
**Module 7: Cấu Trúc Dự Án Thực Hành với AI 139**
#7.1 Layer-based vs Feature-based Package Structure 139
#7.2 Sử Dụng AI Agent Coding Hiệu Quả 142
#7.3 Flow Coding với AI (Overview) 144
#7.4 Flow Coding với AI (Backend) 144
#7.5 Setup Dự Án Thực Hành với AI 145
#7.6 Quy Trình Vibe Coding Với AI 145
**Module 8: [Vibe Coding] Permission & Company CRUD 146**
#8.1 Tổng quan về module 8 146
#8.2 [AI] Permission CRUD 147
#8.3 [AI] Company CRUD 148
#8.4 Setup Swagger & Test API 149
#8.5 Review Permission & Company CRUD 150
**Module 9: [Vibe Coding] Role CRUD & ManyToMany Permission 151**
#9.1 Tổng quan về module 9 151
#9.2 [AI] Role CRUD 152
#9.3 Review Role CRUD 153
**Module 10: [Vibe Coding] User CRUD & File Upload 154**
#10.1 Tổng quan về User CRUD 154

©hoidanit

#10.2 [AI] User CRUD 155
#10.3 Review User CRUD 156
#10.4 Tổng quan về File Upload (Avatar & Logo) 157
#10.5 [AI] File Upload (Avatar & Logo) 158
#10.6 Review File Upload 159
**Module 11: [Vibe Coding] Authentication & Refresh Token 160**
#11.1 Tổng quan về module 11 160
#11.2 Tạo Seed Data 161
#11.3 [AI] Refresh Token & Logout 163
#11.4 Review Authentication & Refresh Token 164
**Module 12: [Vibe Coding] Authorization 165**
#12.1 Tổng quan về module 12 165
#12.2 Phân Biệt Role và Authority 166
#12.3 [AI] Role-Based Access Control (RBAC) 168
#12.4 [AI] Permission-Based Access Control 170
#12.5 Review RBAC 170
**Module 13: Hoàn Thiện Dự Án Thực Hành 171**
#13.1 [AI] Pagination & Sorting 171
#13.2 [AI] Fix Tests & Permissions 172
#13.3 [AI] Search & Filter 173
#13.4 [AI] Scheduled Job - Cleanup Expired Tokens 174
#13.5 Review Backend với Giao Diện Frontend (Fix CORS) 175
#13.6 [AI] Review Toàn Bộ Project 176
#13.7 What's next - Học gì tiếp theo ? 177
**Module 14: Fix Các Lỗi Thường Gặp 180**
#14.1 Lỗi Trùng Port (EADDRINUSE: address already in use) 180
#14.2 Lỗi Class Not Found 181
#14.3 Hướng Dẫn Debug Xem Code Chạy 181

@hoidanit

**Overview**

**Tổng Quan Về Khóa Học**

1.Khóa học này dành cho ai ?
Khóa học phù hợp với bạn nếu bạn đã có kiến thức lập trình cơ bản và muốn bước vào phát triển backend với Java Spring Boot.

Đặc biệt, khóa học này dành cho những bạn đang sử dụng AI để hỗ trợ coding nhưng chưa thực sự hiểu code mình đang làm — hoặc những bạn muốn xây dựng một dự án thực hành bài bản, chuẩn bị kiến thức nền tảng phục vụ cho quá trình đi thực tập và đi làm.

2.Khóa học này KHÔNG phải là gì ?
Đây **không phải khóa học "nhờ AI viết code hộ rồi chạy."**

Cũng không phải khóa học lý thuyết thuần túy, không có dự án thực hành.

Sự khác biệt của khóa học này nằm ở cách tiếp cận: **Fundamental-driven Vibe Coding** — tức là nắm vững kiến thức nền tảng trước, sau đó dùng AI để tăng tốc implementation, nhưng luôn hiểu rõ code mình đang làm gì, tại sao lại làm vậy, và debug được khi có vấn đề.

3. Bạn sẽ học được gì ?
Kết thúc khóa học, bạn sẽ tự tay xây dựng hoàn chỉnh một **Hệ Thống Quản Lý Nhân Sự (HR Management System)** — từ thiết kế database, viết REST API chuẩn, xử lý authentication/authorization, đến viết test và hoàn thiện project.
Cụ thể:

- Hiểu và áp dụng được các kiến thức nền tảng của Spring Boot: Dependency Injection, Spring Data JPA, RESTful API Design, Exception Handling, Testing.

- Biết cách sử dụng AI Agent (Cursor, Claude Code...) hiệu quả trong một dự án thực tế: thiết lập Project Rules, viết prompt đúng cách, review và verify output của AI.

@hoidanit

- Xây dựng được hệ thống backend với đầy đủ tính năng production-level: CRUD cho User/Company/Role/Permission, JWT Authentication, Refresh Token, Role-Based & Permission-Based Access Control, File Upload, Pagination, Search & Filter.

- Hiểu quy trình làm việc thực tế: thiết kế database → viết API spec → coding theo session → viết test → review trước khi commit.

## 4.Cách tiếp cận của khóa học

Khóa học được chia làm hai phần chính.

**Phần đầu (Module 0–6) xây dựng kiến thức nền tảng:** từ mindset làm việc với AI, các khái niệm core của Spring Boot, cho đến cách viết test chất lượng.

**Phần sau (Module 7–13) là thực hành Vibe Coding toàn bộ dự án** — bạn sẽ trực tiếp dùng AI để coding từng tính năng, nhưng luôn có bước review, hiểu code, và verify kết quả trước khi làm bước tiếp theo.

Triết lý xuyên suốt khóa học: **Bạn là kiến trúc sư, AI là thợ xây**. Thợ giỏi thì xây nhanh — nhưng nếu không có bản vẽ đúng, ngôi nhà sẽ có vấn đề dù xây nhanh đến đâu.

@hoidanit

**Source Code Khóa Học (Sử Dụng AI)**

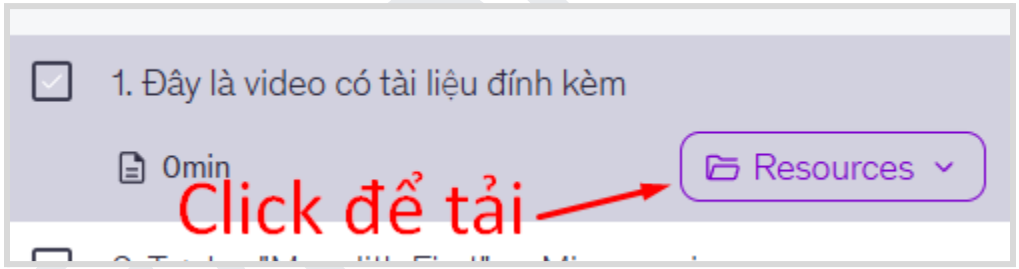
Trong khóa học sẽ có những video, chúng ta sử dụng công cụ AI (AI Agents) để hỗ trợ generate code.

Quy trình thực hiện & thực hành:
Khóa học cung cấp prompt → AI generate code → Khóa học hướng dẫn review (understand + verify) source code do AI tạo ra

Như vậy, quá trình (AI generate code) phụ thuộc vào từng model AI và version của AI đấy, dẫn tới vấn đề là **source code của bạn, và video hướng dẫn là KHÁC NHAU.**

**Cái giống nhau, chính là cách TƯ DUY để giải quyết vấn đề** (hiểu cách công cụ nó hoạt động)

Vì vậy, trường hợp bạn muốn tải source code đang có trong video thực hành, bạn download trực tiếp tài nguồn tài nguyên đính kèm.




@hoidanit

**Về Quyền Tác Giả**

Bản quyền bài giảng/khóa học (tác phẩm) thuộc về tác giả Hỏi Dân IT

Nghiêm cấm sao chép bài giảng dưới mọi hình thức khi chưa được sự cho phép của tác giả Hỏi Dân IT.

Mọi hành vi cố ý hoặc vô ý vi phạm, đều phải chịu trách nhiệm trước pháp luật.

Vì quyền lợi chính đáng của người học, Hỏi Dân IT (hoidanit.vn) rất mong nhận được sự hợp tác của tất cả các bạn.

Thông báo vi phạm, xin gửi về hòm thư: ads.hoidanit@gmail.com
hoặc inbox trực tiếp qua Telegram Hỏi Dân IT: https://t.me/hoidanit

**Về Tác Giả Hỏi Dân IT**

Mọi thông tin về Tác giả Hỏi Dân IT, các bạn có thể tìm kiếm tại đây:

- **Website** chính thức: https://hoidanit.vn
- **Youtube** "Hỏi Dân IT" : https://www.youtube.com/@hoidanit
- **Tiktok** "Hỏi Dân IT" : https://www.tiktok.com/@hoidanit
- **Facebook** "Hỏi Dân IT" : https://www.facebook.com/askITwithERIC
- **Telegram** "Hỏi Dân IT": https://t.me/hoidanit

**Module 0: Mindset Shift - Thích Nghi Trong Kỷ Nguyên AI**
*Định hướng tư duy đúng trước khi bắt đầu coding với sức mạnh của AI*

**#0.1 AI Đã Thay Đổi Cách Học Code Như Thế Nào ?**

1. Vấn đề gặp phải
Câu chuyện thực tế: Một developer 3 năm kinh nghiệm lo lắng khi fresher mới vào team dùng AI code nhanh gấp đôi. Tuy nhiên, khi gặp bug production, fresher đó loay hoay 2 ngày không fix được vì không hiểu code mình đã generate.

**Nghịch lý lớn nhất của thời đại AI:**
Tốc độ tăng, nhưng sự hiểu biết không tăng theo.

2. AI làm được gì, và không làm được gì ?
**AI làm được:**

- Generate boilerplate code trong vài giây
- Viết CRUD operations gần như hoàn chỉnh
- Suggest fix cho common errors
- Explain code khá tốt
- Viết unit tests cơ bản

**AI không/chưa làm được:**

- Không hiểu business context của dự án
- Không biết codebase hiện tại có những gì
- Không detect được architectural issues
- Không biết khi nào đang tạo ra security vulnerability
- Không debug được vấn đề phức tạp cross-module

**Kết luận:**
AI giỏi việc TẠO RA code, nhưng yếu việc ĐẢM BẢO code đúng trong context cụ thể.

@hoidanit

## 3. Sự thay đổi của vai trò Developer

**Trước đây (quá khứ):**

- Developer = Người viết code
- Giá trị = Viết code nhanh, ít bug
- Skill chính = Nhớ syntax, thuộc patterns

**Bây giờ: (tương lai):**

- Developer = Người kiểm soát code
- Giá trị = Đảm bảo code đúng và phù hợp
- Skill chính = Đọc hiểu, verify, debug, decision-making

**Góc nhìn mới:**
AI giống như một junior developer rất nhanh nhưng hay sai và không hiểu context.

Việc của developer không còn là gõ từng dòng code, mà là **review, guide, và verify output.**

## 4. Cách học code cần thay đổi

**Cách học cũ (không còn hiệu quả):**

- Học thuộc syntax
- Code từ đầu mọi thứ để "nhớ"
- Focus vào tốc độ gõ code
- Học một framework rồi gắn bó mãi mãi

@hoidanit

**Cách học mới (cần chuyển sang):**

| Nguyên tắc | Giải thích |
|---|---|
| Kiến thức nền tảng (fundamentals) quan trọng hơn bao giờ hết | Khi bug xảy ra, chỉ người hiểu kiến thức nền tảng mới fix được.<br>AI copy-paste giỏi, nhưng không debug được |
| Đọc code quan trọng hơn viết code | 70% thời gian sẽ là đọc code - code của AI, của teammate, code legacy. Skill đọc hiểu trở thành critical (quan trọng/then chốt) |
| Verify và Debug là kỹ năng sống còn | AI generate code, bạn verify (kiểm chứng/xác nhận) code. Debug skill từ "nice to have" thành "must have" |
| Học tư duy (concepts), không học cú pháp (syntax) | Syntax thay đổi và AI generate được. Concepts không đổi: authentication flow, database design, API patterns… |

## 5. Kết luận

**AI không thay thế developer. AI thay thế developer không chịu thay đổi.**

**Bạn sẽ gặp khó khăn nếu:**

- Chỉ biết gõ code mà không hiểu
- Không debug được khi có vấn đề
- Không verify được output

**Bạn sẽ thích nghi với thời đại, nếu:**

- Học cách kiểm soát AI
- Dùng AI như công cụ hỗ trợ để tăng năng suất lao động
- Nắm vững fundamentals (kiến thức nền tảng)

@hoidanit

### #0.2 Vibe Coding Là Gì? Fundamental-driven Approach

**Thuật ngữ "Vibe Coding"** được Andrej Karpathy - cựu Director AI của Tesla, co-founder OpenAI - đưa ra đầu năm 2025.

**Ý tưởng gốc:** Bạn không cần hiểu code. Chỉ cần "vibe" với AI - mô tả điều bạn muốn, AI generate code, chạy thử, sửa prompt nếu cần. Cứ lặp lại cho đến khi nó hoạt động.

**Câu hỏi đặt ra:** Nghe thì hay, nhưng approach này có thực sự hiệu quả cho developer chuyên nghiệp?

1. Pure Vibe Coding - Vấn đề là gì ?

**Cách hoạt động của Pure Vibe Coding:**

1. Mô tả yêu cầu cho AI
2. AI generate code
3. Chạy thử
4. Nếu lỗi → sửa prompt → lặp lại
5. Nếu chạy được → ship luôn

Vấn đề phát sinh:

| Vấn đề | Hậu quả |
|---|---|
| Không hiểu code đang làm gì | Không maintain được, không extend được |
| Không nhận ra bug ẩn | Code chạy được nhưng sai logic, sai business |
| Không detect security issues | Lỗ hổng bảo mật đi vào production |
| Không debug được | Khi lỗi xảy ra, loay hoay không biết fix ở đâu |
| Không trả lời được “tại sao” | Fail phỏng vấn, mất uy tín với team |

Thực tế:

**Code chạy được ≠ Code đúng**.

**Code đúng hôm nay ≠ Code maintain được ngày mai.**

@hoidanit

## 3. Fundamental - Driven Vibe Coding

**Định nghĩa:** Vẫn dùng AI để tăng tốc, nhưng đảm bảo bạn **HIỂU** mọi thứ AI generate. **AI** làm phần "**gõ code**", **bạn** làm phần "**kiểm soát chất lượng**".

**Nguyên tắc cốt lõi:**

| Nguyên tắc | Giải thích |
|---|---|
| AI generate, you understand | Không copy-paste mù quáng. Đọc hiểu trước khi dùng |
| Speed với confidence | Nhanh nhưng tự tin. Biết code đang làm gì, biết nó đúng hay sai |
| Fundamentals first | Nắm vững concept trước. AI chỉ là công cụ tăng tốc implementation |
| Verify everything | Không tin AI 100%. Luôn verify output trước khi ship production |

**Sự khác biệt:**

| Khía cạnh | Pure Vibe Coding | Fundamental-driven Vibe Coding |
|---|---|---|
| Tốc độ | Nhanh | Nhanh |
| Hiểu code | Không | Có |
| Debug được | Khó | Được |
| Maintain được | Khó | Được |
| Phỏng vấn | Dễ fail | Tỉ lệ pass cao hơn |
| Phù hợp cho | Side project, prototype | Production |

@hoidanit

## 4. Khi nào dùng Approach nào ?

**Pure Vibe Coding phù hợp khi:**

- Làm prototype nhanh để test ý tưởng
- Side project không quan trọng
- Học thử một công nghệ mới (giai đoạn khám phá)
- Automation script dùng một lần

**Fundamental-driven Vibe Coding cần thiết khi:**

- Code đi vào production
- Làm việc trong team
- Dự án cần maintain lâu dài
- Chuẩn bị phỏng vấn
- Muốn phát triển career nghiêm túc

**Quy tắc đơn giản:** Nếu có người khác sẽ đọc code của bạn (teammate, reviewer, tương lai của chính bạn) → dùng Fundamental-driven approach.

## 5. Mindset cần có

**Không phải:**

- "AI code hộ mình"
- "Chạy được là xong"
- "Không cần hiểu, cứ ship"

**Mà là:**

- "AI assist mình code nhanh hơn"
- "Chạy được + hiểu được + maintain được"
- "Hiểu rồi mới ship"

Bạn là kiến trúc sư, AI là thợ xây rất nhanh. Thợ xây giỏi xây, nhưng không biết thiết kế. Nếu bạn không hiểu bản vẽ, ngôi nhà sẽ có vấn đề dù xây nhanh đến mấy.

Vibe Coding là xu hướng không thể tránh. Câu hỏi không phải "có dùng AI không" mà là "dùng AI như thế nào".

@hoidanit

## #0.3 Workflow: Prompt → Generate → Understand → Verify

Workflow 4 bước:
PROMPT → GENERATE → UNDERSTAND → VERIFY
Mỗi bước đều quan trọng. Bỏ qua bước nào cũng sẽ gặp vấn đề.

### 1. Bước 1: Prompt - Yêu cầu đúng cách

**Vấn đề thường gặp:** Prompt mơ hồ → Output không như ý → Mất thời gian sửa đi sửa lại.

| Nguyên tắc | Ví dụ sai | Ví dụ đúng |
|---|---|---|
| Cụ thể | Tạo form đăng nhập | Tạo form đăng nhập với 2 field email/password, có validation và hiển thị error message |
| Nêu context | Fix bug này | Đây là function xử lý payment, đang bị lỗi amount = 0, giúp tôi fix |
| Chỉ rõ output | Viết code xử lý file | Viết function đọc file CSV, return array of objects, mỗi object là 1 row |
| Nêu constraints | Tạo API | Tạo API với response time < 100 ms, handle được 1000 concurrent requests |

**Template prompt cơ bản:**

Tôi cần [task cụ thể].
Context: [mô tả tình huống, tech stack đang dùng].
Input: [dữ liệu đầu vào].
Output mong muốn: [format, structure].
Constraints: [giới hạn, yêu cầu đặc biệt].

@hoidanit

## 2. Bước 2: Generate - Để AI làm việc

**Bước này đơn giản:** AI generate code dựa trên prompt của bạn.

**Lưu ý quan trọng:**

- Không copy-paste ngay lập tức
- Không chạy code ngay mà chưa đọc
- Đây chỉ là "bản nháp đầu tiên", không phải "sản phẩm cuối cùng"

**Sai lầm phổ biến:** Dừng lại ở bước này. Thấy code chạy được → ship luôn. Đây là lý do gây ra hầu hết các vấn đề.

## 3. Bước 3: Understand - Bước quan trọng nhất

**Đây là bước tạo ra sự khác biệt** giữa người dùng AI hiệu quả và người phụ thuộc AI.

Trước khi dùng code AI generate, trả lời được 4 câu hỏi:

| Câu hỏi | Mục đích |
|---|---|
| Code này làm gì ? | Hiểu high-level logic |
| Tại sao viết như vậy ? | Hiểu reasoning đằng sau |
| Có cách nào khác không ? | Biết alternatives, trade-offs |
| Đây có phải là best practice khôn ? | Đánh giá chất lượng |

**Cách thực hành:**

- Đọc code từ trên xuống, giải thích bằng lời của mình
- Nếu có đoạn không hiểu → hỏi AI giải thích đoạn đó
- Nếu vẫn không hiểu → đó là dấu hiệu cần học thêm concept đó

**Quy tắc:** Nếu không giải thích được code cho người khác, bạn chưa hiểu code đó.

@hoidanit

## 4. Bước 4: Verify - Kiểm tra trước khi dùng

**AI không phải lúc nào cũng đúng.** Verify là bước bảo vệ bạn khỏi bugs, security issues, và technical debt.

Checklist verify:

| Hạng mục | Câu hỏi cần trả lời |
|---|---|
| Functionality | Code có làm đúng yêu cầu không |
| Edge cases | Xử lý được các trường hợp đặc biệt chưa? (null, empty, overflow…) |
| Error handling | Có try-catch hợp lý không ? Error message như nào ? |
| Security | Có SQL injections, XSS…, hay lỗ hổng nào không ? |
| Performance | Có vòng lặp vô hạn, N + 1 query, memory leak không ? |
| Readability | Người khác đọc có hiểu không ? (tính maintain) |

**Cách verify nhanh:**

1. Đọc lại code với "con mắt reviewer"
2. Chạy thử với các test cases cơ bản
3. Chạy thử với edge cases
4. Nếu critical (security, payment) → nhờ người khác review

## 5. Ví dụ thực tế áp dụng workflow

**Tình huống**: Cần viết function validate email

**Bước 1 - Prompt:**
Viết function validates email address.
Input: string email
Output: boolean (true nếu valid, false nếu invalid)
Cần check: format đúng, có @ và domain, không có ký tự đặc biệt lạ

@hoidanit

**Bước 2 - Generate:**
AI trả về function với regex pattern

**Bước 3 - Understand**

- Đọc regex, hiểu từng phần đang match gì
- Hỏi AI giải thích nếu regex phức tạp
- Tự trả lời: "Function này check format email bằng regex, return true/false"

**Bước 4 - Verify**

- Test với email hợp lệ: "test@gmail.com" → true
- Test edge cases: "", null, "test@", "@gmail.com", "test@.com"
- Test ký tự đặc biệt: "test+tag@gmail.com" (có nên allow không?)

4 bước workflow:

1. **Prompt** - Yêu cầu rõ ràng, cụ thể
2. **Generate**- Để AI tạo code, nhưng đây chỉ là bản nháp
3. **Understand** - Đọc hiểu, giải thích được code
4. **Verify** - Kiểm tra functionality, edge cases, security

Bỏ bước nào cũng có hậu quả:

- Bỏ Prompt tốt → Output kém, mất thời gian
- Bỏ Understand → Không debug được, fail phỏng vấn
- Bỏ Verify → Bugs, security issues vào production

@hoidanit

### #0.4 Khi Nào Không Nên Dùng AI

AI rất mạnh, nhưng không phải công cụ vạn năng. Dùng sai thời điểm có thể gây hại nhiều hơn lợi.
Nguyên tắc chung: Dùng AI để tăng tốc những gì bạn đã hiểu, không phải để thay thế việc hiểu.

Trường hợp 1: Khi đang học concept mới lần đầu
Vấn đề: Nếu bạn chưa từng viết một vòng lặp, chưa hiểu recursion, chưa biết OOP là gì - đừng để AI viết hộ.
**Tại sao:**

- Học cần có "đau đầu" - struggle là một phần của quá trình
- Tự viết, tự sai, tự fix mới nhớ lâu
- AI viết hộ giống như nhờ người khác tập gym hộ - cơ bắp của bạn không phát triển

**Khi nào được dùng AI:**

- Sau khi đã hiểu concept, dùng AI để tăng tốc implementation
- Dùng AI để giải thích thêm sau khi đã tự thử

Quy tắc:
Concept mới → Tự làm trước ít nhất 1-2 lần → Sau đó mới dùng AI hỗ trợ.

Trường hợp 2: Security-sensitive code
Vấn đề: AI được train trên internet, bao gồm rất nhiều code không an toàn.
AI không biết context bảo mật của hệ thống bạn.

Các phần không nên để AI tự generate (cần verify):

| Ví dụ | Rủi ro nếu sai |
|---|---|
| Password hashing | Dùng algorithm yếu, salt không đúng → dữ liệu user bị lộ |
| Authentication flow | Logic sai → bypass được auth → truy cập trái phép |
| Input sanitization | Thiếu sót → SQL Injection, XSS attacks |
| Encryption/Decryption | Dùng sai mode, key management kém → data bị decrypt |
| Permission/Authorization | Logic sai → user thường có quyền admin |

@hoidanit

**Cách tiếp cận đúng:**

- Học và hiểu security concepts trước
- Nếu dùng AI, phải review từng dòng code
- Tốt nhất: dùng thư viện đã được audit thay vì tự viết

**Quy tắc:**
Security code → Hiểu 100% trước khi dùng. Không chắc → Không ship.

Trường hợp 3: Business logic phức tạp
Vấn đề: AI không biết business của bạn. Nó không biết:

- Khách hàng của bạn muốn gì
- Edge case nào quan trọng với doanh nghiệp
- Constraint nào bắt buộc phải tuân theo
- Quy định pháp lý liên quan

Ví dụ:

- Tính giá sản phẩm với nhiều loại discount chồng chéo
- Quy trình duyệt đơn hàng với nhiều cấp approval
- Logic tính lương, thuế, bảo hiểm
- Quy tắc nghiệp vụ ngân hàng, bảo hiểm, y tế

**Tại sao AI không phù hợp:**

- AI không có context về quy định công ty
- AI có thể suggest giải pháp "đúng về mặt kỹ thuật" nhưng "sai về mặt nghiệp vụ"
- Sai business logic có thể gây thiệt hại tài chính, pháp lý

**Cách tiếp cận đúng:**

- Tự viết hoặc pair với người hiểu business
- Dùng AI chỉ để hỗ trợ phần syntax, boilerplate
- Logic core phải do người hiểu nghiệp vụ quyết định

Quy tắc: Business logic = giá trị của bạn. Đừng outsource nó cho AI.

@hoidanit

Trường hợp 4: Khi chuẩn bị phỏng vấn

Vấn đề: Phỏng vấn technical không có AI hỗ trợ. Nếu quen để AI giải quyết mọi thứ, bạn sẽ đơ/failed khi phỏng vấn.

Các tình huống thường gặp:

- Live coding trên whiteboard hoặc shared screen
- Interviewer hỏi "tại sao dùng cách này?"
- Bị yêu cầu optimize solution
- Debug code có sẵn trong thời gian giới hạn

**Hậu quả nếu phụ thuộc AI:**

- Không viết được code cơ bản khi không có AI
- Không giải thích được logic của solution
- Không biết alternative approaches
- Mất tự tin, thể hiện kém

**Cách chuẩn bị:**

- Practice coding problems không dùng AI ít nhất 2-3 lần/tuần
- Tự giải thích solution bằng lời trước khi code
- Sau khi giải xong, mới dùng AI để xem có cách nào tốt hơn không

Quy tắc: Trước phỏng vấn 2-4 tuần → Giảm dùng AI, tăng tự code.

Trường hợp 5: Debugging vấn đề phức tạp

Vấn đề: Bug phức tạp thường liên quan đến context mà AI không có: state của hệ thống, dữ liệu thực tế, interaction giữa các modules.

**AI thường fail khi:**

- Bug chỉ xảy ra với data cụ thể
- Bug liên quan đến race condition, timing
- Bug do interaction giữa nhiều services
- Bug chỉ xảy ra trên production, không reproduce được local

**Cách tiếp cận đúng:**

Dùng AI như một "người bạn brainstorm", không phải "người giải quyết"

Cung cấp context đầy đủ nếu muốn AI hỗ trợ

Skill debug vẫn phải tự rèn luyện

Quy tắc: Bug phức tạp → Dùng AI để brainstorm hướng đi, không phải để fix hộ.

## Khi nào nên và không nên

**Không nên dùng AI:**

| Trường hợp | Lý do |
|---|---|
| Học concept mới lần đầu | Cần tự struggle để hiểu sâu |
| Security-sensitive code | Rủi ro quá cao, cần hiểu 100% |
| Business logic phức tạp | AI không có context nghiệp vụ |
| Chuẩn bị phỏng vấn | Phỏng vấn không có AI |
| Debug vấn đề phức tạp | AI thiếu context runtime |

**Nên dùng AI:**

| Trường hợp | Lý do |
|---|---|
| Boilerplate, CRUD | Tiết kiệm thời gian việc lặp lại |
| Concept đã hiểu, cần code nhanh | Tăng tốc implementation |
| Tìm hiểu về syntax mới | Reference nhanh |
| Giải thích code người khác viết | Hỗ trợ đọc hiểu |
| Viết tests cho code đã có | Sinh test cases nhanh |

@hoidanit

## #0.5 Kinh Nghiệm Sử Dụng Trợ Lý AI

Lý do không phụ thuộc vào tool cụ thể:

- Tools thay đổi liên tục - hôm nay tốt nhất, ngày mai có thể bị thay thế
- Mỗi tool có điểm mạnh riêng - nên biết dùng nhiều tool
- Principles quan trọng hơn specific tool - học cách dùng, không học tool

Mục tiêu của bài này: Giúp bạn hiểu landscape của AI tools và tự chọn được bộ công cụ phù hợp với mình.

### 1.Phân loại AI tools cho developer

**Loại 1: Chat-based AI**

- Cách dùng: hỏi đáp qua giao diện chat
- Phù hợp cho: giải thích code, generate snippets, brainstorm, debug
- Ví dụ: ChatGPT, Claude, Gemini, Perplexity
- Ưu điểm: linh hoạt, conversation dài, giải thích tốt
- Nhược điểm: phải copy-paste code qua lại

**Loại 2: IDE-integrated AI**

- Cách dùng: tích hợp trực tiếp trong code editor
- Phù hợp cho: Autocomplete, inline suggestions, code completion
- Ví dụ: GitHub Copilot, Cursor, Codeium, Tabnine
- Ưu điểm: nhanh, không cần switch context, hiểu codebase
- Nhược điểm: ít giải thích, suggestions ngắn

**Loại 3: Specialized AI tools**

- Cách dùng: chuyên cho một task cụ thể
- Phù hợp cho: generate UI, documentation, testing...
- Ví dụ: Mintlify (docs), Codium (tests), Figma...
- Ưu điểm: rất tốt cho task chuyên biệt
- Nhược điểm: chỉ dùng được cho một việc

@hoidanit

## 2.Cách setup môi trường hiệu quả

**Bước 1:** Chọn 1 chat-based AI

Cân nhắc:

- Free hay paid? (Paid thường output tốt hơn)
- Cần privacy không? (Một số tool có enterprise plan không lưu data)
- Dùng thử 2-3 tools rồi chọn cái phù hợp nhất

**Bước 2:** Chọn 1 IDE-integrated AI

Cân nhắc:

- IDE đang dùng support không? (VS Code, JetBrains, etc.)
- Free tier có đủ dùng không?
- Có conflict với extensions khác không?

**Bước 3:** Tạo workflow cố định

- Viết code mới (ngắn): IDE-integrated
- Viết code mới (dài, phức tạp): Chat-based
- Debug: Chat-based
- Explain code: Chat-based
- Refactor: IDE-integrated hoặc Chat-based
- Viết tests: Chat-based

## 3.Best practices khi dùng AI tools

**Practice 1: Tạo context rõ ràng**

Khi bắt đầu conversation mới, nên set context:

> Tôi đang làm project [loại project] với [tech stack].
> Codebase đang dùng [patterns, conventions].
> Tôi cần help với [task cụ thể].

**Practice 2: Một conversation = một topic**

Không nên:

- Hỏi nhiều vấn đề khác nhau trong 1 chat
- Chat dài vô tận. Chat quá dài → AI quên context

Nên:

- Mỗi vấn đề một conversation riêng
- Kết thúc conversation khi xong task

@hoidanit

**Practice 3: Save useful prompts**
Tạo một file/note lưu lại các prompts hay dùng:

- Prompt debug
- Prompt explain code
- Prompt generate tests
- Prompt review code

**Practice 4: Verify với nhiều nguồn**
Không chắc output đúng? Có thể:

- Hỏi tool khác xem có cho kết quả giống không
- Google/StackOverflow để cross-check
- Hỏi colleague

Những điều cần tránh

- Dùng quá nhiều tools: context switching giảm productivity
- Phụ thuộc vào 1 tool duy nhất: Tool có thể thay đổi, tăng giá, ngừng hoạt động
- Paste code công ty lên free tier: privacy concern, data có thể bị lưu
- Tin AI 100%: uôn verify output
- Dùng AI cho mọi thứ: biết khi nào không nên dùng

@hoidanit

### #0.6 Prompting Fundamentals Cho Developers

Prompting là kỹ năng giao tiếp với AI. Prompt tốt cho output tốt. Prompt tệ cho kết quả tệ - garbage in, garbage out.

Thực tế: Cùng một task, người viết prompt tốt có thể nhận được kết quả chính xác ngay lần đầu. Người viết prompt kém phải sửa đi sửa lại 5-10 lần.

**5 nguyên tắc viết prompt hiệu quả**

Nguyên tắc 1: Cụ thể (Be specific)

| Prompt kém | Prompt tốt |
|---|---|
| Tạo một API | Tạo REST API endpoint POST /users để tạo user mới, validate email unique, return 201 với user object nếu thành công hoặc 400 với error message nếu thất bại |
| Viết function sort | Viết function sort array of objects theo field 'createdAt' descending, xử lý trường hợp field không tồn tại |

**Tại sao:** AI không đọc được suy nghĩ của bạn. Càng cụ thể, output càng đúng ý.

Nguyên tắc 2: Cung cấp context

| Prompt kém | Prompt tốt |
|---|---|
| Fix bug này | Đây là function xử lý payment trong hệ thống e-commerce. Đang bị lỗi khi amount = 0. Expected: return error. Actual: tiếp tục xử lý. [paste code] |
| Code này chạy sai | Code này nằm trong module authentication. Input là JWT token. Expected output là user object. Actual: return null với token hợp lệ. [paste code] |

**Tại sao:** AI cần biết bối cảnh để cho lời khuyên phù hợp. Không có context, AI chỉ đoán mò.

### Nguyên tắc 3: Chỉ rõ format output

| Prompt kém | Prompt tốt |
|---|---|
| Giải thích code này | Giải thích code này theo format: 1) Tổng quan function làm gì, 2) Giải thích từng bước, 3) Potential issues nếu có |
| Tạo test cases | Tạo test cases cho function này, format mỗi test case gồm: tên test, input, expected output, lý do test case này quan trọng |

**Tại sao:** Định dạng output rõ ràng giúp AI trả về đúng thứ bạn cần, không thừa không thiếu.

### Nguyên tắc 4: Nêu constraints

| Prompt kém | Prompt tốt |
|---|---|
| Tạo form validation | Tạo form validation, không dùng thư viện external, chỉ dùng vanilla JS, phải support IE11 |
| Viết query lấy users | Viết query lấy users, phải chạy được trong O(n), không dùng subquery, database là PostgreSQL |

**Tại sao:** Constraints giúp AI giới hạn solution space, tránh đề xuất những thứ không phù hợp với dự án của bạn.

### Nguyên tắc 5: Yêu cầu giải thích reasoning

| Prompt kém | Prompt tốt |
|---|---|
| Code này có vấn đề gì? | Code này có vấn đề gì? Giải thích TẠI SAO đó là vấn đề và đề xuất fix kèm lý do tại sao cách fix đó tốt hơn |
| Cách nào tốt hơn? | So sánh 2 cách này. Với mỗi cách, nêu pros/cons và recommend cách nào cho use case của tôi kèm lý do |

**Tại sao:** Yêu cầu giải thích giúp bạn hiểu sâu hơn và verify được output có hợp lý không.

## 4 prompt patterns cho developers

**Pattern 1: Code generation**

Tôi cần viết [function/component/module] với yêu cầu:

- Functionality: [mô tả chi tiết function làm gì]
- Input: [mô tả input, data types]
- Output: [mô tả output, data types]
- Tech stack: [ngôn ngữ, framework đang dùng]
- Constraints: [giới hạn, yêu cầu đặc biệt]

Hãy viết code kèm comments giải thích các phần quan trọng.

**Khi nào dùng:** Cần viết code mới từ đầu.

**Pattern 2: Code review / Debug**

Tôi gặp vấn đề với đoạn code sau:

[paste code]

Context: [code này nằm ở đâu, làm gì]

Expected behavior: [mong muốn code chạy như thế nào]

Actual behavior: [thực tế code chạy như thế nào]

Error message (nếu có): [paste error]

Đã thử: [những cách đã thử nhưng chưa được]

Hãy giúp tôi:

1. Xác định nguyên nhân
2. Đề xuất cách fix kèm giải thích
3. Gợi ý cách tránh lỗi tương tự trong tương lai

**Khi nào dùng:** Debug lỗi, review code có vấn đề.

@hoidanit

**Pattern 3: Code explanation**

> Giải thích đoạn code sau:
>
> [paste code]
>
> Tôi đã biết: [những gì bạn đã hiểu]
>
> Chưa hiểu: [phần nào đang confused]
>
> Hãy giải thích theo format:
>
> 1. Tổng quan: code này làm gì (1-2 câu)
> 2. Chi tiết từng bước: giải thích logic từng phần
> 3. Tại sao viết như vậy: lý do chọn cách này
> 4. Potential issues: vấn đề có thể gặp nếu có

**Khi nào dùng:** Đọc code người khác viết, đọc code legacy, học codebase mới.

**Pattern 4: Learning / Research**

> Tôi muốn hiểu về [concept/technology].
>
> Background của tôi: [những gì đã biết liên quan]
>
> Mục đích học: [để làm gì, apply vào đâu]
>
> Hãy giải thích:
>
> 1. [Concept] là gì, giải quyết vấn đề gì
> 2. Khi nào nên dùng, khi nào không nên
> 3. Ví dụ cụ thể với [tech stack tôi đang dùng]
> 4. Common mistakes cần tránh
> 5. Resources để học thêm (nếu có)

**Khi nào dùng:** Học concept mới, research technology mới.

@hoidanit

## Prompting là quá trình lặp lại

Thực tế: Hiếm khi prompt đầu tiên cho kết quả perfect. Đó là bình thường.

Quy trình iterate:

| Bước | Hành động |
|---|---|
| 1 | Viết prompt đầu tiên |
| 2 | Đọc output, đánh giá |
| 3 | Output chưa đúng ý → refine prompt |
| 4 | Lặp lại đến khi output đạt yêu cầu |

**Cách refine prompt:**

- Output quá dài → thêm "Trả lời ngắn gọn trong X câu"
- Output thiếu chi tiết → thêm "Giải thích chi tiết hơn phần X"
- Output sai hướng → cung cấp thêm context hoặc ví dụ

Tip: Sau khi có prompt hoạt động tốt, save lại để dùng lần sau.

## Những sai lầm thường gặp

| Sai lầm | Hậu quả | Cách tránh |
|---|---|---|
| Prompt quá dài, quá nhiều yêu cầu | AI bị overwhelm, miss details | Chia nhỏ thành nhiều prompts |
| Prompt quá ngắn, quá chung chung | AI phải đoán, output generic | Thêm context, cụ thể hóa |
| Không cung cấp context | AI không biết môi trường của bạn | Luôn nêu tech stack, use case |
| Copy-paste error không kèm code | AI không thấy nguyên nhân | Paste cả code và error |
| Tin output 100% | Bugs, security issues | Luôn verify trước khi dùng |

@hoidanit

**Module 1: Spring Boot Big Picture**
*Viết chương trình Hello World với Java Spring Boot sử dụng AI (Vibe Coding)*

## #1.1 Setup Môi Trường Thực Hành & Tài Liệu Java Spring

Mục tiêu:

- Cài đặt môi trường để phát triển dự án với Spring Boot
- Biết các nguồn tài liệu chính thống để tự học
- Không phụ thuộc hoàn toàn vào AI khi gặp vấn đề

### 1.Cài đặt môi trường

**Cài đặt môi trường Java:**

Yêu cầu: JDK 25 (java version 25)
Download tại đây
Sau khi cài, mở terminal gõ: **java -version** để kiểm tra.

Nếu bạn gõ câu lệnh trên không ra gì, hoặc báo lỗi, tham khảo topic này để set tham số **JAVA_HOME**

**Cài đặt Git** để quản lý mã nguồn, và so sánh source code nếu cần thiết.
Nếu bạn chưa biết dùng Git, tham khảo tại đây

### 2. Cài đặt công cụ coding (IDE)

**Bạn có thể dùng bất cứ công cụ coding nào bạn muốn** (miễn là bạn thấy tiện dụng & thoải mái), không bắt buộc phải dùng công cụ như trong video, vì suy cho cùng, đều là gõ cú pháp của ngôn ngữ lập trình (giống nhau)

Tuy nhiên, **nếu được, mình khuyến khích dùng giống công cụ trong video của khóa học, nó sẽ đảm bảo thao tác thực hành giống 100%.**

Việc bạn dùng công cụ khác, có thể thao tác nó khác video hướng dẫn, nhưng bạn cũng đừng quá lo lắng, vì tư duy là giống hệt nhau.

@hoidanit

**Công cụ miễn phí:**

- IntelliJ IDEA (version community - không subscription)
- Visual Studio Code, cài thêm extension hỗ trợ Java Spring
- Eclipse với Spring Tool Suite

**Công cụ trả phí:**

- IntelliJ IDEA (version ultimate - subscription)

**Công cụ hỗ trợ coding với AI:**

- Visual Studio Code, cài thêm extension như Github Copilot (default), Claude…
- Cursor với extension hỗ trợ java
- Antigravity với extension hỗ trợ java

## 3. Tài Liệu của Java Spring

**Nguồn 1: Spring Official Documentation**
Đây là nguồn chính thống nhất. Bao gồm Getting Started guides, Reference documentation, và API docs.

Mẹo: Khi AI generate code, hãy search từ khóa trên docs này để verify.

**Nguồn 2**: Baeldung
Tutorials thực hành, dễ hiểu. Được cộng đồng Spring công nhận. Rất hữu ích khi cần ví dụ cụ thể

**Nguồn 3:** Stack Overflow
Tag: [spring-boot]
Nơi tìm giải pháp cho các lỗi cụ thể. Đọc cả câu hỏi lẫn câu trả lời để hiểu context.

@hoidanit

## #1.2 [AI] Generate Spring Project (Vibe Coding)

Lưu ý: bạn xem video này để hình dung vấn đề, không bắt buộc phải thực hành theo video.

Mục tiêu:

- Trải nghiệm thực tế việc **dùng AI khi chưa có kiến thức nền tảng**
- Thấy được các vấn đề thường gặp khi AI generate sai
- Hiểu tại sao cần học nền tảng trước khi dùng AI hiệu quả

Trong video này, chúng ta sẽ làm đúng như vậy - thử dùng AI tạo Spring Boot project với tư cách người mới hoàn toàn. Không biết gì về Spring. Không biết gì về cấu trúc project.

**Prompt:**
Tạo cho tôi một ứng dụng web bằng Java Spring đơn giản nhất.

Vấn đề không phải AI generate code sai. Vấn đề là bạn chưa có "ngôn ngữ chung" với AI:

- Khi bạn không biết Spring Boot project cần những file gì, bạn không biết AI thiếu file nào.
- Khi bạn không biết dependency là gì, bạn không biết tại sao code không chạy.

@hoidanit

## #1.3 [AI] Hello World với Spring Boot

### 1.Setup dự án thực hành

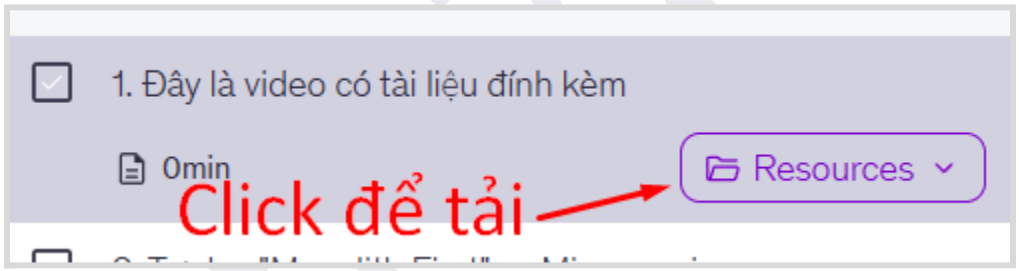
Bạn vui lòng **tải dự án thực hành được cung cấp sẵn**. Điều này đảm bảo, version phần mềm (môi trường thực hành dự án) là giống hệt nhau, hạn chế tối đa lỗi (bug) có thể xảy ra.

Version phần mềm vốn dĩ thay đổi theo thời gian, **cách học sẽ là chọn 1 version, học và hiểu nó, sau đó upgrade** (nâng cấp) lên version bạn mong muốn.

Chỉ không học 1 version nào đó, nếu như code không chạy được.

Download dự án thực hành của khóa học tại đây

Trường hợp công ty (doanh nghiệp) chặn domain của Google, bạn download trực tiếp tại phần **tài nguyên** (resources) đính kèm (trên Udemy - nếu có)




### 2. Viết Hello World

**Prompt sử dụng:**

Tạo file HelloController để viết code. Tôi muốn vào trang chủ, sẽ hiển thị dòng chữ Hello World.

Truy cập:
http://localhost:8080/

### #1.4 Cách Setup Dự Án Thực Hành (Extra)

**Lưu ý:** bạn xem video này cho biết, **không cần thực hành theo video này**.
Sử dụng trực tiếp source code tại video (Hello World với Spring Boot) để setup dự án thực hành.

**Mục đích video này ra đời, là giải đáp thắc mắc/tò mò** của nhiều bạn, không biết mình setup dự án như thế nào, và ngoài ra, hỗ trợ công cụ chính thống để cài đặt dự án thực hành mà **không phải phụ thuộc vào công cụ AI**.

Bạn chỉ nên làm theo video này khi bạn đã học xong dự án của khóa học, hoặc đang làm dự án của chính bạn. (tức là bạn đã có kiến thức nền tảng & có khả năng fix bugs)

Ưu điểm của cách làm này, là sử dụng tài liệu chính thống của Java Spring để tạo dự án với version mới nhất.

**Bước 1:** Truy cập spring-initializr
https://start.spring.io

**Bước 2:** Setup thông tin dự án
//todo

**Bước 3:** Download dự án thực hành
//todo

@hoidanit

## #1.5 Phân Tích Cấu Trúc Dự Án Spring Boot

Mục tiêu:

- Hiểu cấu trúc thư mục chuẩn của Spring Boot project
- Biết mỗi file, mỗi folder có vai trò gì
- Có thể nhìn vào project và không còn cảm giác "lạc lối"

**Thứ nhất: pom.xml**
File này nằm ở gốc project. Nó giống như "danh sách nguyên liệu" của project. Bạn muốn dùng thư viện gì, khai báo ở đây. Spring Boot version nào, Java version nào, cũng ở đây.
Khi AI generate code thiếu dependency, bạn sẽ cần thêm vào file này.

**Thứ hai: src/main/java**
Đây là nơi chứa code Java của bạn. Tất cả Controller, Service, Repository, Entity - đều nằm ở đây.

**Thứ ba: src/main/resources**
Đây là nơi chứa cấu hình. File quan trọng nhất là application.properties hoặc application.yml. Muốn đổi port ? Cấu hình database? Đều ở đây.

Còn folder src/test thì chứa code test. Chúng ta sẽ học ở các module tiếp theo

@hoidanit

**#1.6 Spring Boot là gì? Auto-configuration explained**

Sau video này, học viên sẽ:

- Hiểu **Spring Boot** giải quyết vấn đề gì
- Hiểu khái niệm **Auto-configuration** ở mức cơ bản
- Biết tại sao chỉ vài dòng code mà ứng dụng đã chạy được

1.Trước khi có Spring Boot (ví dụ tại đây)

Bạn muốn tạo một ứng dụng web Java đơn giản. Bạn cần làm những việc sau:

- Download và cài đặt Tomcat server
- Viết file web.xml để cấu hình servlet.
- Viết file applicationContext.xml để cấu hình Spring.
- Cấu hình datasource cho database.
- Cấu hình view resolver nếu dùng template.
- Đảm bảo các version thư viện tương thích với nhau.

Chưa viết dòng code nghiệp vụ nào, bạn đã mất vài giờ chỉ để setup.
Và **nếu cấu hình sai một chỗ → Lỗi.**
Đây là thực tế mà Java developer phải đối mặt trong nhiều năm.

2. Spring Boot ra đời (2014)

Spring Boot được giới thiệu với một triết lý đơn giản:
**Convention over Configuration**.
Nghĩa là: Thay vì bắt bạn cấu hình mọi thứ, Spring Boot sẽ tự cấu hình theo cách phổ biến nhất. Bạn chỉ cần cấu hình khi muốn làm khác đi.

Ví dụ:
Bạn thêm **dependency Spring Web** → Spring Boot tự hiểu bạn muốn làm web app → Tự động cấu hình Tomcat server chạy ở port 8080.

Bạn không cần nói gì. Spring Boot tự làm. Đó là Auto-configuration.

@hoidanit

## 3. Auto-configuration hoạt động thế nào?

Khi ứng dụng khởi động, Spring Boot sẽ:

**Bước 1:** Quét classpath

Spring Boot nhìn vào các thư viện (dependencies) bạn đã thêm trong pom.xml.

**Bước 2:** Đưa ra quyết định

Dựa trên những gì nó thấy, Spring Boot tự động cấu hình.

- Thấy spring-boot-starter-webmvc ? → Cấu hình Tomcat, cấu hình DispatcherServlet.
- Thấy spring-boot-starter-data-jpa + H2? → Cấu hình datasource, cấu hình EntityManager.
- Thấy spring-boot-starter-security? → Cấu hình security cơ bản, tạo trang login.

**Bước 3:** Cho phép bạn override

Nếu bạn muốn khác đi, bạn có thể tự cấu hình. Cấu hình của bạn sẽ đè lên cấu hình tự động.

Muốn đổi port ? Thêm **server.port=9090** vào application.properties → Xong.

**Hiểu Auto-configuration giúp bạn:**

- Debug tốt hơn. Khi có lỗi, bạn biết Spring Boot đang cố làm gì. Lỗi "Port already in use"? Bạn hiểu Spring Boot đang cố khởi động Tomcat.

- Prompt AI tốt hơn. Thay vì hỏi mơ hồ, bạn có thể hỏi cụ thể: "Làm sao để override auto-configuration của datasource?"

- Đọc code AI generate tốt hơn. Khi AI thêm dependency, bạn hiểu nó sẽ trigger những auto-configuration nào.

@hoidanit

### #1.7 Annotations Cơ Bản - Đọc Hiểu Code AI Generate

Khi bạn nhờ AI generate code Spring Boot, bạn sẽ thấy rất nhiều từ bắt đầu bằng dấu @. Đó là annotations.

**@SpringBootApplication** đánh dấu đây là class chính của ứng dụng.

- Nó bật tất cả tính năng auto-configuration của Spring Boot.
- Quy tắc: Mỗi project chỉ có đúng một class với annotation này. Thường đặt ở thư mục gốc của package.

**@RestController và @Controller**

- @RestController đánh dấu class này là nơi nhận request HTTP. Giống như "lễ tân" của ứng dụng - nơi đón khách.

Có một annotation tương tự là @Controller. Sự khác biệt:

- @Controller: Trả về tên view (HTML page)
- @RestController: Trả về dữ liệu trực tiếp (JSON)

Với REST API, chúng ta dùng @RestController.

**@GetMapping**("/") nghĩa là: Khi có request GET đến đường dẫn /, chạy method này.

@hoidanit

### #1.8 Chạy Ứng Dụng với Spring Boot DevTools

Hiện tại, mỗi lần sửa code, bạn phải dừng ứng dụng rồi chạy lại, khá mất thời gian.

**Spring Boot DevTools** giải quyết vấn đề này. Nó tự động restart ứng dụng khi bạn thay đổi code.

//pom.xml

```
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <scope>runtime</scope>
    <optional>true</optional>
  </dependency>
```

@hoidanit

**Module 2: Dependency Injection**
*Sử dụng Dependency Injection để coding Java Spring hiệu quả*

## #2.1 Xây Dựng Cấu Trúc Project Chuẩn

Tại sao cần cấu trúc ?
Hãy tưởng tượng một công ty:

- Lễ tân đón khách, không tự xử lý yêu cầu
- Nhân viên nghiệp vụ xử lý công việc, không tự lưu trữ hồ sơ
- Nhân viên kho lưu trữ và tìm kiếm hồ sơ

Mỗi người một việc. Rõ ràng, dễ quản lý. Spring Boot cũng vậy.
Chúng ta chia code thành các layer, mỗi layer một nhiệm vụ.

Kiến trúc 3 lớp, đây là kiến trúc phổ biến nhất trong Spring Boot:
Request → Controller → Service → Repository → Database

Layer 1: Controller
Nhiệm vụ: Nhận request từ client, trả response.

- Giống như: Lễ tân công ty.
- **Không làm:** Xử lý logic phức tạp, truy cập database trực tiếp.

Layer 2: Service
Nhiệm vụ: Xử lý business logic.

- Giống như: Nhân viên nghiệp vụ.
- **Không làm:** Nhận request trực tiếp, truy cập database trực tiếp.

Layer 3: Repository
Nhiệm vụ: Làm việc với database (lưu, đọc, xóa, sửa dữ liệu).

- Giống như: Nhân viên kho.
- Không làm: Xử lý logic nghiệp vụ.

Thành phần: Model (Entity)
Nhiệm vụ: Định nghĩa cấu trúc dữ liệu.
Giống như: Mẫu form, bản thiết kế hồ sơ.

@hoidanit

Tạo base cấu trúc dự án thực hành:

src/main/java/vn/hoidanit/springrestwithai/
├── SpringRestWithAiApplication.java
├── **controller**/
├── **service**/
├── **repository**/
└── **model**/

Tên folder tổ chức code, nên đặt số ít, hay số nhiều ?

├── **controllers**/
├── **services**/
├── **repositories**/
└── **models**/

@hoidanit

### #2.2 [AI] Tạo User CRUD với hardcode data

Video trước, chúng ta đã tạo cấu trúc folder rỗng: controller/, service/, repository/, model/. Bây giờ cần viết code thực sự.

Chúng ta sẽ làm **CRUD** cho User:

- Create: Tạo user mới
- Read: Lấy danh sách user, lấy user theo id
- Update: Cập nhật user
- Delete: Xóa user

Chưa có database, nên chúng ta sẽ hardcode data trong memory. Mục đích là hiểu flow giữa các layer trước.

Và thay vì gõ tay từng dòng, chúng ta sẽ dùng AI hỗ trợ. Nhưng lần này, bạn đã có kiến thức nền tảng để viết prompt tốt hơn.

//prompt: trang tiếp theo

//todo: giải thích code & test kết quả

@hoidanit

## Prompt gợi ý:

Tôi đang học Spring Boot, cần tạo CRUD đơn giản cho User với hardcode data (chưa dùng database).

Cấu trúc project hiện tại:
- Package gốc: vn.hoidanit.springrestwithai
- Đã có sẵn các folder rỗng: controller/, service/, repository/, model/
- Cần viết code cho: User (model), UserRepository, UserService, UserController

Yêu cầu:
1. User (model):
  - Các field: id (Long), name (String), email (String)
  - Constructor rỗng, constructor đầy đủ
  - Getters và Setters

2. UserRepository:
  - Dùng ArrayList để lưu data trong memory
  - Khởi tạo sẵn 3 user mẫu
  - Các method: findAll(), findById(), save(), update(), deleteById()

3. UserService:
  - Gọi UserRepository để thực hiện các thao tác
  - Các method tương ứng với repository

4. UserController:
  - REST API endpoints:
    + GET /users - lấy tất cả users
    + GET /users/{id} - lấy user theo id
    + POST /users - tạo user mới
    + PUT /users/{id} - cập nhật user
    + DELETE /users/{id} - xóa user

5. Chưa cần dùng @Autowired, tôi sẽ khởi tạo thủ công bằng new để hiểu vấn đề trước.

Hãy viết code cho từng file riêng biệt.

@hoidanit

## #2.3 Dependency Injection là gì? Vấn đề nó giải quyết

Trước khi nói về code, hãy tưởng tượng bạn mở một nhà hàng.

**Tình huống 1: Đầu bếp tự lo mọi thứ**
Bạn thuê 3 đầu bếp. Mỗi đầu bếp tự đi chợ mua nguyên liệu, tự mang về, tự nấu.
Vấn đề xảy ra:

- Đầu bếp A mua thịt bò ở chợ A, đầu bếp B mua ở chợ B → chất lượng khác nhau
- Mỗi người mua số lượng khác nhau → khó kiểm soát chi phí
- Muốn đổi nhà cung cấp → phải nói với từng đầu bếp

**Tình huống 2: Nhà hàng cung cấp nguyên liệu**
Bạn thuê một người quản lý kho. Người này chịu trách nhiệm mua nguyên liệu và phân phối cho tất cả đầu bếp.
Lợi ích:

- Tất cả đầu bếp dùng chung nguồn nguyên liệu → chất lượng đồng nhất
- Dễ kiểm soát chi phí
- Muốn đổi nhà cung cấp → chỉ nói với người quản lý kho

Trong lập trình, Dependency Injection hoạt động giống tình huống 2.

### 1. Dependency là gì

Dependency nghĩa là "**sự phụ thuộc**".
**Quay lại ví dụ nhà hàng:**
Đầu bếp cần nguyên liệu để nấu ăn → Nguyên liệu là dependency của đầu bếp
Nguyên liệu lấy từ kho → Kho là dependency của nguyên liệu

**Trong code** cũng vậy:
Controller cần Service để xử lý logic → Service là dependency của Controller
Service cần Repository để lấy dữ liệu → Repository là dependency của Service

**Một class cần class khác để hoạt động, class đó là dependency.**

@hoidanit

**Vấn đề khi tự tạo Dependency:**

```
// Pseudo code - không phải ngôn ngữ cụ thể
class Service {
    constructor() {
        this.repository = new Repository()  // Tự tạo
    }
}

class Controller {
    constructor() {
        this.service = new Service()  // Tự tạo
    }
}
```

**Vấn đề 1: Nhiều bản sao**
Mỗi Controller tạo Service riêng. Mỗi Service tạo Repository riêng. Nếu có 10 Controller, có 10 Service, có 10 Repository. Tốn bộ nhớ, dữ liệu không đồng bộ.

**Vấn đề 2: Khó thay đổi**
Muốn đổi từ Repository A sang Repository B? Phải sửa code ở tất cả các Service.

**Vấn đề 3: Khó test**
Muốn test Service mà không cần Repository thật? Không được, vì Service tự tạo Repository bên trong.

**Vấn đề 4: Vi phạm Single Responsibility**
Service vừa phải xử lý logic, vừa phải lo việc tạo Repository. Quá nhiều trách nhiệm.

@hoidanit

## 2. Dependency Injection - Giải pháp

Dependency Injection (DI) là kỹ thuật mà thay vì class tự tạo dependency, dependency được "tiêm" (inject) từ bên ngoài vào.

**Trước khi có DI:**

```
class Service {
  constructor() {
    this.repository = new Repository()  // Tự tạo bên trong
  }
}
```

**Sau khi áp dụng DI:**

```
class Service {
  constructor(repository) {
    this.repository = repository  // Được truyền từ bên ngoài
  }
}
```

## 3. Lợi ích của Dependency Injection

- **Single Instance** (Singleton)
  Chỉ có một Repository được tạo và chia sẻ cho tất cả Service. Tiết kiệm bộ nhớ, dữ liệu đồng bộ.

- Loose Coupling (Giảm phụ thuộc)
  Service không còn phụ thuộc vào cách tạo Repository. Muốn đổi Repository? Chỉ sửa một chỗ.

- Dễ test
  Muốn test Service? Truyền vào một MockRepository giả. Không cần database thật.

- Dễ bảo trì
  Code rõ ràng hơn. Mỗi class chỉ lo một việc. Thay đổi ở một chỗ không ảnh hưởng nhiều chỗ khác.

@hoidanit

**#2.4 IoC Container - Behind the scenes**

Dependency Injection là "tiêm dependency từ bên ngoài vào".
Nhưng có một câu hỏi chưa trả lời: Ai là "bên ngoài"? Ai chịu trách nhiệm tạo dependency và tiêm vào?

Quay lại ví dụ nhà hàng: Đầu bếp không tự đi chợ nữa. Vậy ai đi chợ? Ai quản lý kho? Ai phân phối nguyên liệu?

Câu trả lời: Người quản lý kho.
Trong lập trình, "người quản lý kho" đó chính là **IoC Container**.

1.IoC là gì ?

IoC là viết tắt của **Inversion of Control** - Đảo ngược quyền kiểm soát.
**Trước khi có IoC:**
Bạn (code của bạn) kiểm soát mọi thứ:

- Bạn quyết định khi nào tạo object
- Bạn quyết định tạo object nào
- Bạn quyết định object đó sống bao lâu

```
// Bạn kiểm soát
service = new Service()
repository = new Repository()
```

**Sau khi có IoC:**
Bạn nhường quyền kiểm soát cho framework:

- Framework quyết định khi nào tạo object
- Framework quyết định tạo bao nhiêu object
- Framework quyết định object sống bao lâu
- Bạn chỉ việc "xin" object khi cần

```
// Framework kiểm soát, bạn chỉ khai báo "tôi cần cái này"
@Inject service
```

**Đó là sự "đảo ngược"**. Trước đây bạn kiểm soát, giờ framework kiểm soát.
Giống như trước đây đầu bếp tự quyết định mua gì, giờ nhà hàng quyết định cung cấp gì.

@hoidanit

## 2. IoC Container là gì ?

IoC Container là "người quản lý" chịu trách nhiệm:

1. **Tạo object: k**hi ứng dụng khởi động, container đọc cấu hình, biết cần tạo những object nào.
2. **Quản lý lifecycle: c**ontainer quyết định object được tạo khi nào, sống bao lâu, bị hủy khi nào.
3. **Inject dependency:** khi một object cần object khác, container tự động "tiêm" vào.
4. **Lưu trữ object:** container giữ các object trong "kho". Khi ai đó cần, container lấy từ kho ra đưa.

Hãy tưởng tượng IoC Container như một nhà máy thông minh:

- Nhà máy biết cần sản xuất những sản phẩm gì (đọc bản thiết kế)
- Nhà máy sản xuất và lưu trong kho
- Khi ai đó đặt hàng, nhà máy lấy từ kho giao cho họ
- Nếu sản phẩm A cần linh kiện B, nhà máy tự động lắp B vào A trước khi giao

## 3. Cách Container hoạt động

Khi ứng dụng khởi động, container làm những việc sau:

**Bước 1:** Quét và đăng ký
Container quét toàn bộ code, tìm những class được đánh dấu "tôi muốn được quản lý".
Ví dụ: Class nào có annotation đặc biệt như @Service, @Repository, @Injectable...
Container lưu danh sách: "À, tôi cần quản lý UserService, UserRepository..."

**Bước 2:** Phân tích dependency
Container nhìn vào từng class: "UserService cần gì? À, cần UserRepository."
Container xây dựng "bản đồ phụ thuộc" trong đầu.

@hoidanit

**Bước 3:** Tạo object theo thứ tự
Container tạo object từ "gốc" lên:

Tạo UserRepository trước (vì nó không cần ai)
Tạo UserService sau (và inject UserRepository vào)
Tạo UserController sau cùng (và inject UserService vào)

**Bước 4:** Lưu vào kho
Các object được lưu trong container. Mặc định chỉ có 1 bản (singleton).

**Bước 5:** Cung cấp khi cần
Khi code của bạn "xin" UserService, container lấy từ kho và đưa cho bạn.

## 4. Singleton - Mặc định quan trọng

Mặc định, IoC Container tạo mỗi class chỉ một object duy nhất. Đây gọi là Singleton.

Ví dụ:
Có 10 Controller cần UserService. Container không tạo 10 UserService. Container tạo 1 UserService và đưa cho cả 10 Controller dùng chung.
Lợi ích:

- Tiết kiệm bộ nhớ
- Dữ liệu đồng bộ (tất cả dùng chung một nguồn)
- Hiệu suất tốt hơn

**Lưu ý:**
Vì dùng chung, cần cẩn thận với dữ liệu. Nếu Controller A thay đổi gì đó trong Service, Controller B cũng thấy thay đổi đó.

@hoidanit

### #2.5 @Autowired, @Component, @Service trong Spring

Mục tiêu:

- Hiểu **@Component**, **@Service**, **@Repository**, @Controller đánh dấu class cho Container
- Hiểu **@Autowired** yêu cầu Container inject dependency
- Refactor code video 2.2 để sử dụng DI đúng cách
- Thấy được sự khác biệt trước và sau khi dùng DI

Dependency Injection: Tiêm từ bên ngoài vào
IoC Container: "Người" chịu trách nhiệm tạo và tiêm

Làm sao để Spring Boot Container biết cần quản lý class nào và inject vào đâu?
→ Dùng annotations.

#### 1. @Component

@Component nói với Spring: "Hãy quản lý class này. Hãy tạo object và bỏ vào Container."

```
@Component
public class UserRepository {
    // ...
}
```

Khi Spring khởi động:

- Spring quét tất cả các class
- Thấy @Component → "À, class này cần được quản lý"
- Spring tạo object UserRepository
- Spring lưu vào Container

**Các "biến thể" của @Component:**
Spring cung cấp các annotation chuyên biệt hơn:
**@Service** → dùng cho Business logic
**@Repository** → dùng cho  Data access

Về mặt kỹ thuật, **chúng giống nhau**. Nhưng dùng đúng annotation **giúp code dễ đọc hơn.**

@hoidanit

## 2. @Autowired - Yêu cầu inject

@Autowired nói với Spring: "Tôi cần dependency này. Hãy inject vào cho tôi."

```
@Service
public class UserService {

    @Autowired
    private UserRepository userRepository; // Spring tự inject
}
```

Khi Spring tạo UserService:

- Spring thấy field userRepository có @Autowired
- Spring tìm trong Container: "Có UserRepository không?"
- Tìm thấy → Spring gán vào field này
- Không tìm thấy → Lỗi!

Chú ý: UserRepository phải có @Component (hoặc @Repository) thì Spring mới quản lý và mới inject được.

//todo: refactor code sử dụng DI

@hoidanit

### #2.6 Constructor vs Field Injection - Best Practice

Chúng ta đã dùng **@Autowired** để inject dependency:

```
@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;

    // Các method...
}
```

Code chạy tốt. Spring inject UserRepository thành công. Vậy có vấn đề gì không?
→ **Code hoạt động, nhưng không phải best practice.**

1.Vấn đề của Field Injection

Field Injection (dùng @Autowired trên field) có 4 vấn đề:

**Vấn đề 1: Không thể dùng final**

```
@Autowired
private UserRepository userRepository; // Không thể thêm final
```

Không có final nghĩa là dependency có thể bị thay đổi sau khi tạo object. Điều này không an toàn.

**Vấn đề 2: Dependency bị ẩn**

Nhìn từ bên ngoài class, không biết class cần dependency gì. Phải mở code ra mới thấy @Autowired.

**Vấn đề 3: Khó test**

Khi viết unit test, bạn muốn truyền mock repository vào. Với Field Injection, bạn phải dùng reflection hoặc Spring test context → Phức tạp.

**Vấn đề 4: Có thể tạo object không hợp lệ**

UserService service = new UserService(); // Tạo được, nhưng repository = null

Object được tạo mà không có dependency. Gọi method sẽ bị NullPointerException.

@hoidanit

## 2. Constructor Injection - Giải pháp

Constructor Injection giải quyết tất cả vấn đề trên:

```
@Service
public class UserService {
    private final UserRepository userRepository;
    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }
    // Các method...
}
```

Giải quyết vấn đề 1: Dùng được final → dependency không thể thay đổi.
Giải quyết vấn đề 2: Nhìn constructor biết ngay class cần gì.

Giải quyết vấn đề 3: Test dễ dàng:
// Trong unit test
UserRepository mockRepo = new MockUserRepository();
UserService service = new UserService(mockRepo); // Truyền mock vào

Giải quyết vấn đề 4: Không thể tạo object thiếu dependency:
UserService service = new UserService( ); // Lỗi compile! Phải truyền repository

//todo: refactor code sử dụng constructor injection.

@hoidanit

### #2.7 [AI] Đọc Code AI Generate - Identify DI Patterns

Khi bạn nhờ AI generate code Spring Boot, AI có thể dùng Field Injection hoặc Constructor Injection.

**Nếu thấy code như này:**

```
@Autowired
private SomeRepository repository;
```

→ **Field Injection**. Nên refactor sang Constructor Injection.

**Nếu thấy code như này:**

```
private final SomeRepository repository;

public SomeService(SomeRepository repository) {
   this.repository = repository;
}
```

→ **Constructor Injection**. Đây là best practice, giữ nguyên.

Khi nhờ AI generate code Spring Boot, thêm dòng này vào prompt:

```
Sử dụng Constructor Injection (không dùng @Autowired trên field).
Các dependency khai báo với private final.
```

@hoidanit

**Module 3: Spring Data JPA**
*Sử dụng database với Java Spring*

**#3.1 Cài đặt MySQL và Công Cụ Quản Lý**
Sau video này, học viên sẽ:

- Cài đặt được MySQL Server trên máy
- Cài đặt được công cụ quản lý database (MySQL Workbench hoặc DBeaver)
- Biết cách kiểm tra MySQL đang chạy hay không
- Tạo được database đầu tiên cho khóa học

1.Cài trực tiếp (Windows/Mac)
Video hướng dẫn chi tiết, tham khảo tại đây
**Bước 1:** Truy cập https://dev.mysql.com/downloads/mysql/

**Bước 2:** Chọn phiên bản phù hợp với hệ điều hành (Windows/macOS)

**Bước 3:** Download MySQL Installer

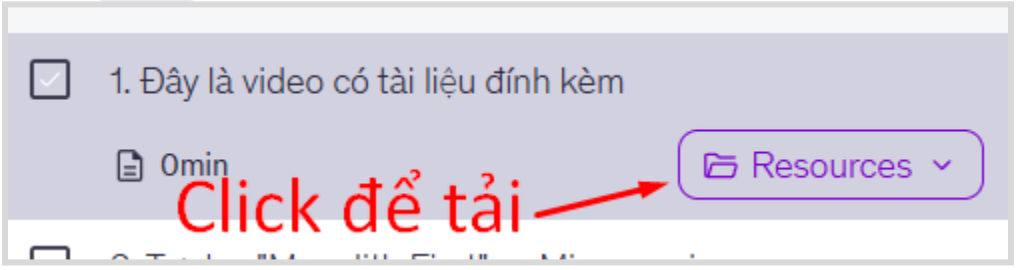
**Bước 4:** Trong quá trình cài đặt:

- Port: giữ mặc định 3306
- Root password: đặt password dễ nhớ (ví dụ: 123456 cho môi trường học tập)

**Bước 5:** Hoàn tất cài đặt

2. Dùng Docker (khuyến nghị cho developer/đi làm)
Download file docker compose tại đây

Trường hợp công ty (doanh nghiệp) chặn domain của Google, bạn download trực tiếp tại phần **tài nguyên** (resources) đính kèm (trên Udemy - nếu có)



## 3. Cài đặt công cụ quản lý

Có 2 lựa chọn phổ biến:

- Lựa chọn 1: **MySQL Workbench** (của MySQL)

- Lựa chọn 2: **DBeaver** (khuyến nghị)
  Download: https://dbeaver.io/download/

  Ưu điểm: Nhẹ, giao diện đẹp, hỗ trợ nhiều loại database
  Nhược điểm: Bản miễn phí đủ dùng, bản pro mất phí

## 4. Tạo database cho project

//todo

@hoidanit

## #3.2 Cấu hình Spring Boot kết nối MySQL

### 1.Thêm Dependencies

Mở file **pom.xml**, thêm 2 dependencies vào trong thẻ <dependencies>:

```
<!-- Spring Data JPA -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<!-- MySQL Driver -->
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
</dependency>
```

- spring-boot-starter-data-jpa: cung cấp JPA, Hibernate, các annotation như @Entity, @Repository
- mysql-connector-j: driver để Java kết nối với MySQL

### 2.Cấu hình application.properties

Mở file src/main/resources/application.properties, thêm cấu hình:

```
# ==============================
# DATABASE CONNECTION
# ==============================
spring.datasource.url=jdbc:mysql://localhost:3306/hr_management?allowPublicKeyRetrieval=true&useSSL=false
spring.datasource.username=root
spring.datasource.password=123456
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# ==============================
# JPA / HIBERNATE
# ==============================
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

## #3.3 ORM là gì ? Tại sao cần?

### 1.ORM là gì ?

**ORM = Object-Relational Mapping**
Dịch: Ánh xạ giữa Object và Relational (bảng trong database).

ORM là công cụ tự động chuyển đổi giữa:

- Object trong code ↔ Hàng trong database
- Class trong code ↔ Bảng trong database
- Thuộc tính trong class ↔ Cột trong bảng

Với ORM, thay vì viết SQL thủ công:

```
users = orm.findAll(User)

// ORM tự động:
// 1. Tạo SQL: SELECT * FROM users
// 2. Gửi đến database
// 3. Chuyển kết quả thành List<User>
```

### 2.ORM làm được gì ?

1. **Tự động tạo bảng từ Class**
Định nghĩa class User trong code → ORM tự tạo bảng users trong database.

2. **CRUD không cần viết SQL**

```
orm.save(user)     // INSERT INTO users...
orm.findById(1)    // SELECT * FROM users WHERE id = 1
orm.update(user)   // UPDATE users SET...
orm.delete(user)   // DELETE FROM users WHERE…
```

3. **Quản lý quan hệ**
User thuộc Company → ORM tự động JOIN khi cần.

4. **Bảo vệ SQL Injection**
ORM tự động escape input, tránh bị tấn công.

@hoidanit

## 3. Ưu nhược điểm của ORM

| Ưu điểm | Giải thích |
|---|---|
| Code ngắn gọn | Không viết SQL lặp lại |
| Ít lỗi | Không sai chính tả tên cột |
| Dễ bảo trì | Đổi tên cột, IDE báo lỗi ngay |
| Bảo mật | Tự động chống SQL Injection |
| Đổi database dễ | Chuyển MySQL sang Postgres, chỉ đổi config |

| Nhược điểm | Giải thích |
|---|---|
| Hiệu năng | Query phức tạp có thể chậm hơn SQL thuần |
| Learning curve | Cần học thêm công cụ mới |
| N + 1 problem | Dễ mắc lỗi nếu không hiểu cách ORM hoạt động |
| Khó debug | SQL được generate tự động, khó kiểm soát |

ORM phù hợp cho 90% trường hợp. Chỉ viết SQL thuần khi cần tối ưu hiệu năng cao.

@hoidanit

## #3.4 Database Design Thinking

### 1. Quy trình thiết kế database

Quy trình 4 bước:

**Bước 1:** Xác định Entity (những "thứ" cần quản lý)

**Bước 2:** Xác định thuộc tính của mỗi Entity

**Bước 3:** Xác định quan hệ giữa các Entity

**Bước 4:** Xác định khóa chính (Primary Key)

### 2. Xác định Entity

Entity là những "danh từ quan trọng" trong yêu cầu.
Cách làm: Đọc yêu cầu, gạch chân các danh từ quan trọng. (actor)

**Ví dụ yêu cầu:**
Hệ thống quản lý nhân sự. Nhân viên làm việc tại công ty. Mỗi nhân viên có vai trò khác nhau. Mỗi vai trò có các quyền truy cập khác nhau.

Gạch chân: Nhân viên, Công ty, Vai trò, Quyền truy cập
→ Entity: User, Company, Role, Permission

**Lưu ý: Không phải danh từ nào cũng là Entity**

- Entity: User, Company, Role (cần quản lý riêng, có nhiều thuộc tính)
- Thuộc tính: Email, Ngày sinh, Địa chỉ (thuộc về Entity, không đứng riêng)

### 3. Các loại quan hệ

Có 3 loại quan hệ cơ bản:

**One-to-One (1:1)**
Một A có đúng một B. Một B thuộc đúng một A.
Ví dụ: User - Profile (mỗi user có đúng 1 profile)

**One-to-Many (1:N)**
Một A có nhiều B. Một B thuộc một A.
Ví dụ: Company - User (một công ty có nhiều nhân viên, một nhân viên thuộc một công ty)

**Many-to-Many (M:N)**
Một A có nhiều B. Một B có nhiều A.
Ví dụ: User - Role (một user có nhiều role, một role gán cho nhiều user)
Cần bảng trung gian: user_roles

## 4. Cách xác định quan hệ - Đặt 2 câu hỏi

Với mỗi cặp Entity, đặt 2 câu hỏi:
Câu 1: Một A có bao nhiêu B?
Câu 2: Một B có bao nhiêu A?

**Ví dụ 1: User và Company**
Một User làm tại bao nhiêu Company? → 1
Một Company có bao nhiêu User? → Nhiều
→ Kết luận: One-to-Many (Company → User)

**Ví dụ 2: User và Role**
Một User có bao nhiêu Role? → Nhiều (vừa là ADMIN vừa là HR)
Một Role gán cho bao nhiêu User? → Nhiều
→ Kết luận: Many-to-Many

## 5. Sai lầm thường gặp

**Sai lầm 1:** Không phân tích, code ngay
Hậu quả: Thiếu bảng, thừa bảng, quan hệ sai → refactor tốn công sức/thời gian

**Sai lầm 2:** Nhầm thuộc tính thành Entity
Ví dụ: Tạo bảng riêng cho Email, Address
Thực tế: Chúng chỉ là cột trong bảng User

**Sai lầm 3:** Sai loại quan hệ
Thiết kế User-Role là 1:N (một user chỉ có 1 role)
Sau này yêu cầu thay đổi: user cần nhiều role → phải sửa lại schema

**Sai lầm 4:** Không nghĩ đến tương lai
Câu hỏi cần đặt: Tương lai có cần mở rộng không?
Ví dụ: Hiện tại 1 user thuộc 1 company. Tương lai có thể làm nhiều company không?

@hoidanit

**#3.5 Thiết Kế Database Khóa Học**

Bài trước chúng ta đã học lý thuyết Database Design Thinking. Bây giờ áp dụng trực tiếp vào dự án thực hành: Hệ thống Quản Lý Nhân Sự (HR Management).

## 1. Phân Tích Yêu Cầu

Yêu cầu nghiệp vụ:

Xây dựng hệ thống quản lý nhân sự. Hệ thống cho phép quản lý nhân viên, phân công nhân viên vào các công ty. Mỗi nhân viên có thể có nhiều vai trò (role) khác nhau trong hệ thống. Mỗi vai trò có các quyền hạn (permission) nhất định.

Gạch chân danh từ quan trọng: Nhân viên (User), Công ty (Company), Vai trò (Role), Quyền hạn (Permission)

→ Xác định được 4 Entity chính: **User, Company, Role, Permission**

## 2. Thiết Kế Các Entity

**Entity 1: User (Nhân viên)**

Đây là entity trung tâm của hệ thống.

Các field cần có:

- id: Long (khóa chính, tự tăng)
- name: String (họ tên nhân viên)
- email: String (email đăng nhập, unique)
- password: String (mật khẩu đã mã hóa)
- age: Integer (tuổi)
- address: String (địa chỉ)
- gender: Enum (MALE / FEMALE / OTHER)
- avatar: String (ảnh đại diện)
- createdAt: Instant (thời điểm tạo)
- updatedAt: Instant (thời điểm cập nhật cuối)

@hoidanit

**Entity 2: Company (Công ty)**
Quản lý thông tin các công ty trong hệ thống.

Các field cần có:
- id: Long (khóa chính, tự tăng)
- name: String (tên công ty)
- description: String (mô tả)
- address: String (địa chỉ công ty)
- logo: String (đường dẫn ảnh logo)
- createdAt: Instant
- updatedAt: Instant

**Entity 3: Role (Vai trò)**
Định nghĩa các vai trò trong hệ thống (ADMIN, HR, USER, MANAGER...).

Các field cần có:
- id: Long (khóa chính, tự tăng)
- name: String (tên role, ví dụ: ADMIN, HR)
- description: String (mô tả vai trò)
- createdAt: Instant
- updatedAt: Instant

**Entity 4: Permission (Quyền hạn)**
Định nghĩa các quyền cụ thể trong hệ thống (CREATE_USER, DELETE_USER, VIEW_REPORT...).

Các field cần có:
- id: Long (khóa chính, tự tăng)
- name: String (tên quyền, ví dụ: CREATE_USER)
- apiPath: String (đường dẫn API được phép, ví dụ: /api/v1/users)
- method: String (HTTP method: GET, POST, PUT, DELETE)
- module: String (module thuộc về, ví dụ: USER, COMPANY)
- createdAt: Instant
- updatedAt: Instant

@hoidanit

## 3. Xác Định Quan Hệ Giữa Các Entity

**Quan hệ 1: User - Company (Many-to-One)**
Câu hỏi:
- Một User làm tại bao nhiêu Company? → 1 (một nhân viên thuộc một công ty)
- Một Company có bao nhiêu User? → Nhiều (một công ty có nhiều nhân viên)
→ Kết luận: Many-to-One (nhiều User thuộc một Company)
→ Khóa ngoại: company_id nằm trong bảng users

**Quan hệ 2: User - Role (Many-to-Many)**
Câu hỏi:
- Một User có bao nhiêu Role? → Nhiều (vừa là ADMIN vừa là HR)
- Một Role gán cho bao nhiêu User? → Nhiều (nhiều người cùng là HR)
→ Kết luận: Many-to-Many
→ Cần bảng trung gian: user_roles (user_id, role_id)

**Quan hệ 3: Role - Permission (Many-to-Many)**
Câu hỏi:
- Một Role có bao nhiêu Permission? → Nhiều (ADMIN có thể làm nhiều thứ)
- Một Permission thuộc bao nhiêu Role? → Nhiều (quyền VIEW_USER có thể gán cho nhiều role)
→ Kết luận: Many-to-Many
→ Cần bảng trung gian: role_permissions (role_id, permission_id)

## 4. Tóm Tắt Các Bảng Sẽ Tạo

Tổng cộng 6 bảng:
- **users**: lưu thông tin nhân viên
- **companies**: lưu thông tin công ty
- **roles**: lưu các vai trò hệ thống
- **permissions**: lưu các quyền hạn cụ thể
- **user_role**: bảng trung gian User - Role (Many-to-Many)
- **permission_role**: bảng trung gian Role - Permission (Many-to-Many)

@hoidanit

**#3.6 JPA và Hibernate trong Spring**

Video trước, chúng ta học ORM là cầu nối giữa Object và Database.
Trong Java, có nhiều ORM khác nhau: Hibernate, EclipseLink, OpenJPA...
Vấn đề: Mỗi ORM có cách dùng khác nhau. Đổi ORM phải sửa code.

Giải pháp: Cần một "tiêu chuẩn chung" để các ORM tuân theo → Đó là JPA.

1. JPA là gì

**JPA = Jakarta Persistence API** (trước đây là Java Persistence API)
JPA là một specification (đặc tả, tiêu chuẩn). Nó định nghĩa:

- Các annotation như @Entity, @Table, @Column, @Id...
- Các interface như EntityManager, Query...
- Các quy tắc ORM phải tuân theo

Quan trọng: **JPA chỉ là "bản thiết kế", không phải code chạy được**.
Giống như:

- Bản vẽ kiến trúc nhà (JPA) vs Ngôi nhà thật (Hibernate)
- Luật giao thông (JPA) vs Người thực thi luật (Hibernate)

JPA nói: "Muốn đánh dấu một class là Entity, dùng annotation @Entity"
Nhưng JPA không tự làm được. Cần có ai đó implement.

2. Hibernate là gì

**Hibernate là một implementation (triển khai) của JPA**.
Hibernate đọc các annotation của JPA và thực hiện công việc thật:

- Thấy @Entity → Tạo bảng trong database
- Thấy @Column → Tạo cột trong bảng
- Gọi save() → Generate SQL INSERT và gửi đến database

@hoidanit

### #3.7 [AI] Tạo Entity đầu tiên - User

Mục tiêu:

- Viết Entity User đầu tiên với JPA annotations
- Hiểu @Entity, @Table, @Id, @GeneratedValue, @Column
- Thấy Spring Boot tự động tạo bảng trong database

#### 1. Entity là gì trong JPA?

Trong JPA, Entity là một class Java được "ánh xạ" (map) vào một bảng trong database.
Khi bạn khai báo class User là Entity, Hibernate sẽ tự động:

- Tạo bảng users trong database (nếu chưa có)
- Mỗi object User = một hàng trong bảng
- Mỗi field trong class = một cột trong bảng

#### 2. Viết Entity User

Các field cần có:
- id: Long (khóa chính, tự tăng)
- name: String (họ tên nhân viên)
- email: String (email đăng nhập, unique)
- password: String (mật khẩu đã mã hóa)
- age: Integer (tuổi)
- address: String (địa chỉ)
- gender: Enum (MALE / FEMALE / OTHER)
- avatar: String (ảnh đại diện)
- createdAt: Instant (thời điểm tạo)
- updatedAt: Instant (thời điểm cập nhật cuối)

@hoidanit

**PROMPT:**

Tôi đang làm dự án Java Spring Boot, cần tạo Entity cho đối tượng User.
Tên bảng trong database: users
Yêu cầu các field:

id: Long, khóa chính, tự tăng (AUTO_INCREMENT)
name: String
email: String, unique, không được null
password: String
age: Integer
address: String
gender: Enum gồm 3 giá trị MALE / FEMALE / OTHER, lưu dạng STRING vào database
avatar: String (ảnh đại diện)
createdAt: Instant
updatedAt: Instant

Yêu cầu kỹ thuật:

Dùng Jakarta Persistence (jakarta.persistence.*), không dùng javax.persistence
Khai báo Enum GenderEnum dưới dạng public enum trong cùng file
Có đầy đủ Getter/Setter cho tất cả các field. Viết hàm toString
Không dùng Lombok
Không cần constructor có tham số, chỉ cần constructor mặc định (no-args)
Thêm comment ngắn giải thích những annotation quan trọng

Chỉ trả về code Java, không giải thích thêm.

## 3. Giải Thích Từng Annotation

**@Entity**

Bắt buộc phải có. Đánh dấu class này là một JPA Entity.
Hibernate sẽ biết class này cần được map vào database.

**@Table(name = "users")**

Chỉ định tên bảng trong database là "users".
Nếu không khai báo, Hibernate mặc định dùng tên class (User → user).
Best practice: luôn khai báo tường minh để tránh nhầm lẫn.

**@Id**

Đánh dấu field này là khóa chính (Primary Key) của bảng.
Mỗi Entity bắt buộc phải có đúng một field @Id.

@hoidanit

**@GeneratedValue(strategy = GenerationType.IDENTITY)**
Chỉ định cách tạo giá trị cho khóa chính:

- IDENTITY: database tự tăng (AUTO_INCREMENT của MySQL)
- SEQUENCE: dùng sequence (phổ biến với PostgreSQL)
- AUTO: để Hibernate tự quyết định

→ Với MySQL, dùng IDENTITY là phù hợp nhất.

**@Column**(unique = true, nullable = false)
Tùy chỉnh cột trong database:
- unique = true: không được trùng giá trị (ví dụ: email phải unique)
- nullable = false: không được null (bắt buộc phải có giá trị)
- name = "ten_cot": đặt tên cột khác tên field (nếu cần)

**@Enumerated(EnumType.STRING)**
Chỉ định cách lưu Enum vào database:

- EnumType.STRING: lưu tên của enum ("MALE", "FEMALE", "OTHER") → dễ đọc, khuyến nghị
- EnumType.ORDINAL: lưu số thứ tự (0, 1, 2) → nguy hiểm nếu thêm/xóa giá trị enum

@hoidanit

**#3.8 Repository Pattern với JpaRepository**

Mục tiêu:

- Hiểu JpaRepository cung cấp những gì
- Tạo UserRepository sử dụng JpaRepository
- Biết cách tạo custom query methods

## 1. JpaRepository Là Gì?

**JpaRepository là một interface được Spring Data JPA cung cấp sẵn**.

Khi bạn tạo interface kế thừa JpaRepository, Spring Data JPA sẽ tự động generate code implementation.

Bạn không cần viết gì thêm. Chỉ cần khai báo interface, Spring lo hết.

## 2. Những Gì JpaRepository Cung Cấp Sẵn

Khi kế thừa JpaRepository, bạn đã có ngay các method sau mà không cần viết code:

**CRUD cơ bản:**

- save(entity): lưu hoặc cập nhật (nếu id đã tồn tại → UPDATE, chưa có → INSERT)
- findById(id): tìm theo id, trả về Optional<User>
- findAll(): lấy tất cả
- deleteById(id): xóa theo id
- existsById(id): kiểm tra id có tồn tại không
- count(): đếm tổng số bản ghi

**Phân trang và sắp xếp:**

- findAll(Pageable pageable): lấy có phân trang
- findAll(Sort sort): lấy có sắp xếp

@hoidanit

**Query Methods - Magic Của Spring Data JPA**
Tham khảo tài liệu tại đây

Spring Data JPA có khả năng tự động generate SQL dựa trên tên method.
Quy tắc: findBy + TênField + Điều_kiện

Ví dụ:
- findByEmail(String email) → SELECT * FROM users WHERE email = ?
- findByAge(Integer age) → SELECT * FROM users WHERE age = ?
- findByNameAndEmail(String name, String email) → WHERE name = ? AND email = ?
- findByAgeGreaterThan(Integer age) → WHERE age > ?
- findByNameContaining(String name) → WHERE name LIKE '%name%'
- findByOrderByNameAsc() → ORDER BY name ASC

Không cần viết SQL. Chỉ cần đặt tên method đúng quy tắc.

@hoidanit

## #3.9 [AI] Refactor UserRepository - Từ hardcode sang JPA

Mục tiêu:

- Xóa bỏ hardcode ArrayList, thay bằng JPA thật
- Cập nhật UserService và UserController cho phù hợp
- Test API với Postman, dữ liệu lưu vào MySQL thật

### 1.Cài đặt phần mềm Postman

Download Postman tại đây

### 2. Refactor - Dùng JpaRepository

//todo: prompt

> Tôi đang refactor dự án Java Spring Boot từ hardcode data (dùng ArrayList) sang JPA thật (MySQL database).
>
> Context hiện tại:
> - Đã có Entity User với JPA annotations hoàn chỉnh
> - Đã cấu hình kết nối MySQL trong application.properties
> - UserRepository hiện tại là class dùng ArrayList để lưu data trong memory
> - UserService đang gọi các method của UserRepository (ArrayList)
> - UserController đang hoạt động tốt, KHÔNG ĐƯỢC thay đổi UserController
>
> Yêu cầu refactor:
> 1. UserRepository chuyển thành interface, sử dụng JpaRepository
>
> 2. UserService:
> - Inject UserRepository thông qua constructor injection
> - Cập nhật các method để dùng JpaRepository, GIỮ NGUYÊN signature (tên method, tham số, kiểu trả về) để UserController không bị ảnh hưởng
>
> Yêu cầu kỹ thuật:
> - Dùng Constructor Injection, không dùng @Autowired trên field
> - Các dependency khai báo private final
> - Import đúng: jakarta.persistence.*, org.springframework.data.jpa.repository.*
> - Không dùng Lombok
> - KHÔNG dùng ResponseEntity (chưa học)
> - KHÔNG thay đổi UserController
>
> Output mong muốn:
> Trả về code đầy đủ cho 2 files:
> 1. UserRepository.java (interface)
> 2. UserService.java (class với method signatures giống code cũ)
>
> Không cần giải thích, chỉ cần code.

**Module 4: RESTful APIs**
*Sử dụng backend java spring để viết API theo chuẩn RESTful*

**#4.1 RESTful API là gì? Principles & Constraints**
1. API là gì ?
**API = Application Programming Interface**
Dịch đơn giản: Giao diện để các ứng dụng nói chuyện với nhau.

Client gửi request (yêu cầu) đến API
API xử lý, tương tác với database
API trả về response (kết quả)

| Không có API | Có API |
|---|---|
| Frontend phải truy cập database trực tiếp | Frontend chỉ cần gọi API |
| Mobile app phải biết cấu trúc database | Mobile app không biết database là gì |
| Muốn đổi database phải sửa cả frontend | Đổi database chỉ cần sửa backend |
| Không thể chia sẻ logic giữa web và mobile | Web và mobile dùng chung 1 API |

2. REST là gì ?
**REST = Representational State Transfer**
REST không phải là công nghệ, không phải là thư viện, không phải là framework.

REST là một architectural style (phong cách kiến trúc) — tập hợp các nguyên tắc và ràng buộc để thiết kế hệ thống phân tán (distributed systems).

REST được đề xuất bởi Roy Fielding trong luận án tiến sĩ năm 2000. Ông là một trong những tác giả chính của giao thức HTTP. Vì vậy, REST được thiết kế để tận dụng tối đa sức mạnh của HTTP.

**Ý tưởng cốt lõi:**
Mọi thứ trong hệ thống đều là Resource (tài nguyên). Và bạn tương tác với resource thông qua các thao tác chuẩn của HTTP.

## 3. Resource là gì ?

Resource là bất kỳ "thứ" nào bạn muốn quản lý, chia sẻ, hay truy cập qua mạng.
Ví dụ trong hệ thống HR Management:

- Danh sách nhân viên → Resource
- Thông tin một nhân viên cụ thể → Resource
- Danh sách công ty → Resource
- Một role cụ thể → Resource

## 4. Sáu Ràng Buộc (Constraints) của REST

Một hệ thống tuân thủ đủ 6 ràng buộc này mới được gọi là RESTful.

**Constraint 1: Client-Server**

Client và Server phải tách biệt nhau.

- Client lo giao diện, trải nghiệm người dùng
- Server lo lưu trữ dữ liệu, xử lý logic

**Lợi ích:**

- Frontend và backend có thể phát triển độc lập
- Có thể thay đổi giao diện mà không ảnh hưởng server
- Có thể scale server mà không ảnh hưởng client

**Constraint 2: Stateless**

Stateless = Server không lưu trạng thái của client giữa các request.
Mỗi request phải chứa đầy đủ thông tin để server xử lý. Server không nhớ request trước là gì.

**Lợi ích:**

- Dễ scale: thêm server mới không cần sync session
- Dễ debug: mỗi request độc lập, tái hiện lỗi dễ hơn

**Hệ quả thực tế:** Đây là lý do tại sao REST API thường dùng JWT Token (thay vì Session). Token chứa đầy đủ thông tin user, gửi kèm mỗi request.

**Constraint 3: Cacheable**

Response có thể được cache (lưu tạm) ở client hoặc ở tầng trung gian.
Server cần đánh dấu rõ: response này có được cache không, cache trong bao lâu.

@hoidanit

**Lợi ích:**

- Giảm tải cho server
- Tăng tốc độ phản hồi cho client
- Tiết kiệm băng thông

Ví dụ thực tế: Danh sách tỉnh/thành phố không thay đổi → Cache 24 giờ. Thông tin tài khoản ngân hàng → Không cache.

**Constraint 4: Uniform Interface**

Đây là ràng buộc đặc trưng nhất của REST.

- Resource Identification in Requests
  Mỗi resource phải có URI định danh rõ ràng.

- Resource Manipulation through Representations
  Client không tương tác trực tiếp với resource (row trong database). Client làm việc với representation (biểu diễn) của resource, thường là JSON.

- Self-descriptive Messages
  Mỗi message phải chứa đủ thông tin để server/client hiểu cần làm gì. HTTP Method + URI + Headers + Body phải đủ rõ.

- HATEOAS (Hypermedia as the Engine of Application State)
  Response nên chứa các link dẫn đến các hành động liên quan. Ràng buộc này ít được implement trong thực tế.

**Constraint 5: Layered System**

Client không cần biết mình đang nói chuyện trực tiếp với server hay qua một tầng trung gian.
Hệ thống có thể có nhiều lớp: Load Balancer → API Gateway → Server → Database. Client chỉ biết địa chỉ API, không biết bên trong có bao nhiêu lớp.

**Lợi ích:**

- Dễ scale: thêm server vào sau load balancer, client không biết
- Bảo mật tốt hơn: API Gateway có thể làm auth, rate limiting
- Linh hoạt: có thể thêm/bớt tầng mà không ảnh hưởng client

@hoidanit

**Constraint 6: Code on Demand (Optional)**
Server có thể gửi code để client thực thi (ví dụ: JavaScript). Ràng buộc này là optional — không bắt buộc để được gọi là RESTful.

## 5. REST vs RESTful — Khác nhau ở đâu ?

| Khái niệm | Ý nghĩa |
|---|---|
| REST | Tập hợp các nguyên tắc, ràng buộc kiến trúc do Roy Fielding định nghĩa |
| RESTful | Tính từ mô tả một hệ thống/API tuân thủ các nguyên tắc của REST |
| REST API | API được thiết kế theo phong cách REST |

Trong thực tế, nhiều API tự gọi là "REST API" nhưng không tuân thủ đầy đủ 6 ràng buộc — đặc biệt là HATEOAS và Stateless. Điều này chấp nhận được ở mức độ thực dụng, miễn là thiết kế nhất quán và dễ sử dụng.

## 6. Tại Sao REST Phổ Biến ?

Hiện nay có nhiều kiểu API architecture khác: SOAP, GraphQL, gRPC... Nhưng REST vẫn là lựa chọn phổ biến nhất vì:

| Lý do | Giải thích |
|---|---|
| Dựa trên HTTP | Không cần học thêm protocol mới |
| Đơn giản | Dễ đọc, dễ debug bằng browser hoặc Postman |
| Stateless | Dễ scale horizontal |
| Phổ biến | Tài liệu nhiều, community lớn, tooling tốt |
| Linh hoạt | Không ràng buộc ngôn ngữ lập trình hay platform |

@hoidanit

## #4.2 HTTP Methods, Status Codes & Conventions

### 1. HTTP Methods — Động từ của REST

URI là "đối tượng", HTTP Method là "việc bạn muốn làm với đối tượng đó".

**GET — Lấy dữ liệu**

Dùng khi: Muốn đọc dữ liệu, không thay đổi gì trên server.
Đặc điểm quan trọng:

- Safe: Không thay đổi dữ liệu trên server
- Idempotent: Gọi 1 lần hay 100 lần, kết quả như nhau
- Không có request body
- Có thể cache được

**POST — Tạo mới**

Dùng khi: Muốn tạo một resource mới trên server.
Đặc điểm quan trọng:

- Không Safe: Thay đổi dữ liệu (tạo mới)
- Không Idempotent: Gọi 2 lần → tạo 2 bản ghi khác nhau
- Có request body chứa dữ liệu cần tạo
- Server trả về resource vừa tạo kèm id

**PUT — Cập nhật toàn bộ**

Dùng khi: Muốn thay thế hoàn toàn một resource bằng dữ liệu mới.
Đặc điểm quan trọng:

- Idempotent: Gọi nhiều lần với cùng data → kết quả như nhau
- Phải gửi toàn bộ các field, kể cả field không thay đổi
- Nếu thiếu field → field đó bị xóa hoặc null

**PATCH — Cập nhật một phần**

Dùng khi: Muốn cập nhật một số field của resource, giữ nguyên phần còn lại.
Đặc điểm quan trọng:

- Chỉ gửi những field cần thay đổi
- Các field không gửi → giữ nguyên giá trị cũ
- Thực tế: nhiều team dùng PUT cho cả 2 trường hợp

@hoidanit

**DELETE — Xóa**
Dùng khi: Muốn xóa một resource.
Đặc điểm quan trọng:

- Idempotent: Xóa user 1 lần hay nhiều lần → user đó vẫn không còn
- Thường không có request body
- Có 2 kiểu xóa: xóa cứng (hard delete) và xóa mềm (soft delete)

## 2. HTTP Status Codes — Kết quả của mỗi request

Status code là con số 3 chữ số server trả về, cho client biết request xử lý thế nào.
Có 5 nhóm, phân biệt bằng chữ số đầu tiên:

- 1xx → Informational (Đang xử lý)
- 2xx → Success (Thành công)
- 3xx → Redirection (Chuyển hướng)
- 4xx → Client Error (Lỗi do client)
- 5xx → Server Error (Lỗi do server)

## 3. REST API Conventions — Quy ước thiết kế

Conventions không phải quy tắc bắt buộc, nhưng là thỏa thuận ngầm giữa các developer. Tuân thủ để API dễ đọc, dễ dùng, dễ bảo trì.

**Convention 1: Đặt tên URI**
Dùng danh từ số nhiều (plural nouns):
✅ /users (số nhiều)
❌ /user (số ít)
❌ /getUsers (có động từ)
❌ /fetchUserList (quá dài, có động từ)

Dùng chữ thường, phân cách bằng dấu gạch ngang:
✅ /job-positions
✅ /user-profiles
❌ /jobPositions (camelCase)
❌ /JobPositions (PascalCase)
❌ /job_positions (underscore)

@hoidanit

**Convention 2: Kết hợp Method + URI = Hành động rõ ràng**
//todo: Bảng thiết kế chuẩn cho User CRUD

**Convention 3: Versioning — Quản lý phiên bản API**
Khi API thay đổi, cần đảm bảo client cũ không bị ảnh hưởng.
Cách phổ biến nhất: đưa version vào URI.

- /api/v1/users → Version 1 (đang dùng)
- /api/v2/users → Version 2 (mới, có breaking changes)

**Convention 4: Query Parameters — Lọc, Tìm kiếm, Phân trang**
Dùng query parameters cho filtering, sorting, và pagination. Không tạo endpoint riêng.

```
# Phân trang
GET /users?page=1&size=10

# Sắp xếp
GET /users?sort=name&order=asc

# Lọc
GET /users?gender=MALE
```

**Convention 5: Response Format nhất quán**
Mọi API endpoint nên trả về cùng một cấu trúc JSON. Client dễ xử lý, dễ debug.

@hoidanit

## #4.3 API Design Best Practices

### 1. Tại Sao Cần Best Practices ?

Viết API chạy được là một chuyện. Viết API người khác dùng được dễ dàng là chuyện khác.

Hãy tưởng tượng bạn là frontend developer, gọi API của một team khác và nhận được json:

```
// API 1 trả về
{ "status": 1, "users": [...] }

// API 2 trả về
{ "code": 200, "data": { "list": [...] } }

// API 3 trả về
{ "success": true, "result": [...] }
```

Ba endpoint, ba format khác nhau. Frontend phải viết ba đoạn code xử lý khác nhau. Đây là hệ quả của việc không có best practices.

**API tốt = API mà developer khác có thể dùng mà không cần hỏi.**

### 2. Response Format Chuẩn

Nhất quán tuyệt đối — mọi endpoint cùng cấu trúc:

- statusCode + message + data
- Lỗi: statusCode + message + error
- Validation: trả tất cả lỗi cùng lúc

**Error Handling:**

- Dùng đúng HTTP Status Code
- Error message có ý nghĩa, đủ thông tin để fix
- Không expose stack trace trong production

@hoidanit

**Pagination:**

- Luôn phân trang, không trả toàn bộ dữ liệu
- meta: page + pageSize + pages + total
- Giới hạn pageSize tối đa

**Naming:**

- ISO 8601 cho timestamp
- Nhất quán xuyên suốt toàn hệ thống
- Version API nếu cần thiết
- Không expose dữ liệu nhạy cảm

@hoidanit

## #4.4 Controller Layer — Annotations, Path Variable, Request Body & Request Param

**Controller** là cửa ngõ đầu tiên của ứng dụng. Mọi request từ bên ngoài đều đi qua đây trước. Nhiệm vụ của Controller:

- Nhận request
- Gọi Service xử lý
- Trả response về cho client

Controller không tự xử lý logic. Nó chỉ điều phối.

**@RestController = @Controller + @ResponseBody**
→ Đánh dấu class là REST Controller
→ Tự động convert return value → JSON

**@RequestMapping**
→ Đặt prefix URL cho toàn bộ class
→Tránh lặp lại prefix ở từng method

**@GetMapping** → Xử lý HTTP GET (lấy dữ liệu)

**@PostMapping** → Xử lý HTTP POST (tạo mới dữ liệu)

**@PutMapping** → Xử lý HTTP PUT (cập nhật toàn bộ dữ liệu)

**@PatchMapping** → Xử lý HTTP PATCH (cập nhật một phần dữ liệu)

**@DeleteMapping** → Xử lý HTTP DELETE (xóa dữ liệu)

**@PathVariable** → Lấy giá trị từ URL path (/users/{id})

**@RequestBody** → Lấy data từ request body (JSON → Object)

**@RequestParam** → Lấy query parameters (?page=1&size=10)

@hoidanit

## #4.5 [AI] Response Entity - Kiểm Soát HTTP Response

Mục tiêu:

- Hiểu ResponseEntity là gì và tại sao cần dùng
- Biết cách trả về status code, headers, và body một cách linh hoạt
- Refactor UserController để sử dụng ResponseEntity đúng cách

### 1.Vấn đề với cách trả về hiện tại

| Vấn đề | Hậu quả |
|---|---|
| Không kiểm soát được status code | Mọi response đều trả về 200 OK, kể cả khi không tìm thấy user |
| Không thêm được custom headers | Không thể set Cache-Control, Location, hay headers khác |
| Khó xử lý trường hợp null | User không tồn tại vẫn trả về 200 với body null |
| Response format không nhất quán | Client khó xử lý khi có lúc trả object, có lúc trả null |

### 2. ResponseEntity là gì?

ResponseEntity là một class của Spring cho phép bạn kiểm soát hoàn toàn HTTP response:

- Status code: 200, 201, 404, 500...
- Headers: Content-Type, Location, Cache-Control...
- Body: Dữ liệu trả về (JSON)

### 3. Các cách tạo ResponseEntity

**Cách 1:** Dùng static methods

```
ResponseEntity.ok(user);                // 200 OK + body
ResponseEntity.created(uri).body(user);     // 201 Created + body
ResponseEntity.notFound().build();          // 404 Not Found
ResponseEntity.noContent().build();         // 204 No Content
ResponseEntity.badRequest().body(error);    // 400 Bad Request + body
```

**Cách 2:** Dùng constructor

```
new ResponseEntity<>(user, HttpStatus.OK);
new ResponseEntity<>(HttpStatus.NOT_FOUND);
```

**Cách 3:** Dùng status() khi cần custom status
ResponseEntity.status(HttpStatus.CONFLICT).body(data);

## 4. Prompt Refactor UserController

Tôi cần refactor UserController để sử dụng ResponseEntity.

Context hiện tại:
- UserController đang trả về trực tiếp object (User, List<User>)
- UserService đã có sẵn các method: findAll(), findById(), create(), update(), deleteById(), existsById()
- Package: vn.hoidanit.springrestwithai.controller

Yêu cầu:
1. Sử dụng ResponseEntity để kiểm soát HTTP response
2. Trả về đúng status code theo chuẩn REST:
  - GET /users: 200 OK
  - GET /users/{id}: 200 OK nếu tìm thấy, 404 Not Found nếu không
  - POST /users: 201 Created với header Location
  - PUT /users/{id}: 200 OK nếu tìm thấy, 404 Not Found nếu không
  - DELETE /users/{id}: 204 No Content nếu tìm thấy, 404 Not Found nếu không

3. Với POST, thêm header Location trỏ đến resource vừa tạo

Yêu cầu kỹ thuật:
- Dùng Constructor Injection, không dùng @Autowired trên field
- Import: org.springframework.http.ResponseEntity, org.springframework.http.HttpStatus
- Dùng static methods của ResponseEntity (ok(), created(), notFound(), noContent()...)
- Không tạo thêm class mới, chỉ refactor UserController

Chỉ trả về code Java, không giải thích thêm.

## #4.6 [AI] Response Format Chuẩn với ApiResponse

**Vấn đề với cách trả về hiện tại:**
Sau khi dùng ResponseEntity, API đã trả về đúng status code. Tuy nhiên, response body chưa nhất quán.

Format mong muốn:

```
// Success Response
{
  "statusCode": 200,
  "message": "Lấy user thành công",
  "data": {
    "id": 1,
    "name": "Nguyen Van A"
  }
}

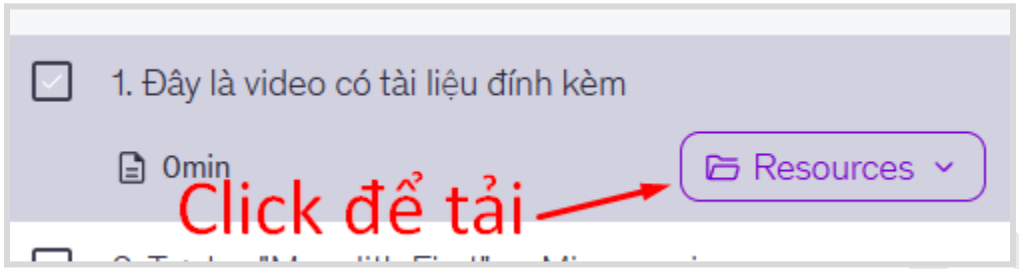
// Error Response
{
  "statusCode": 404,
  "message": "Không tìm thấy user",
  "error": "Not Found"
}

// Validation Error Response (nhiều lỗi)
{
  "statusCode": 400,
  "message": "Dữ liệu không hợp lệ",
  "error": "Bad Request",
  "details": [
    "Email không đúng định dạng",
    "Tên không được để trống"
  ]
}
```

@hoidanit

Download file ApiResponse tại đây

Trường hợp công ty (doanh nghiệp) chặn domain của Google, bạn download trực tiếp tại phần **tài nguyên** (resources) đính kèm (trên Udemy - nếu có)




@hoidanit

## Prompt Refactor UserController với ApiResponse:

Tôi cần refactor UserController để sử dụng ApiResponse wrapper class.

Context hiện tại:
- UserController đang dùng ResponseEntity<User>, ResponseEntity<List<User>>
- Đã có class ApiResponse<T> với các static methods:

SUCCESS methods:
+ ApiResponse.success(T data) - 200 OK
+ ApiResponse.success(String message, T data) - 200 OK
+ ApiResponse.created(T data) - 201 Created
+ ApiResponse.created(String message, T data) - 201 Created

ERROR methods:
+ ApiResponse.notFound(String message) - 404 Not Found
+ ApiResponse.badRequest(String message) - 400 Bad Request
+ ApiResponse.badRequest(String message, List<String> details) - 400 với chi tiết
+ ApiResponse.conflict(String message) - 409 Conflict
+ ApiResponse.unauthorized(String message) - 401 Unauthorized
+ ApiResponse.forbidden(String message) - 403 Forbidden
+ ApiResponse.error(String message) - 500 Internal Server Error

Yêu cầu:
1. Wrap tất cả response trong ApiResponse (file này đã có, sử dụng lại, không tạo mới)
2. Sử dụng đúng static method cho từng trường hợp:
- GET thành công: ResponseEntity.ok() + ApiResponse.success()
- GET không tìm thấy: ResponseEntity.status(NOT_FOUND) + ApiResponse.notFound()
- POST thành công: ResponseEntity.created() + ApiResponse.created()
- POST duplicate: ResponseEntity.status(CONFLICT) + ApiResponse.conflict()
- PUT thành công: ResponseEntity.ok() + ApiResponse.success()
- PUT không tìm thấy: ResponseEntity.status(NOT_FOUND) + ApiResponse.notFound()
- DELETE thành công: ResponseEntity.noContent().build() (không cần body)
- DELETE không tìm thấy: ResponseEntity.status(NOT_FOUND) + ApiResponse.notFound()

3. Message phải có ý nghĩa và tiếng Việt

Chỉ trả về code Java, không giải thích thêm.

## #4.7 [AI] Exception Handling - Xử Lý Lỗi Tập Trung

Mục tiêu:

- Hiểu vấn đề khi không xử lý exception đúng cách
- Tạo Global Exception Handler với @ControllerAdvice
- Trả về lỗi theo format ApiResponse nhất quán
- Bảo mật: không để lộ stack trace ra ngoài

### 1.Vấn đề khi không xử lý Exception

| Vấn đề | Hậu quả |
|---|---|
| Lộ stack trace | Hacker biết được cấu trúc code, dễ tấn công |
| Format không nhất quán | Khác với ApiResponse đã định nghĩa |
| Message khó hiểu | User/Frontend không biết lỗi gì |
| Không kiểm soát được | Mỗi loại lỗi trả về format khác nhau |

**Nguyên tắc bảo mật:** Không bao giờ để lộ stack trace, tên class, hay cấu trúc code ra production.

### 2. @ControllerAdvice - Xử lý Exception tập trung

**@ControllerAdvice** (hoặc **@RestControllerAdvice**) cho phép bạn xử lý exception ở một nơi duy nhất, áp dụng cho toàn bộ ứng dụng.

**Cách hoạt động:**

Request → Controller → Exception xảy ra
↓
GlobalExceptionHandler bắt
↓
Trả về ApiResponse chuẩn

Best Practices:

| **Practice** | **Lý do** |
|---|---|
| Dùng **@RestControllerAdvice** | Xử lý exception tập trung, một nơi duy nhất |
| Tạo Custom Exception | Rõ ràng nghiệp vụ, dễ đọc, dễ maintain |
| Throw exception từ Service | Controller gọn, logic tập trung ở Service |
| Không lộ stack trace | Bảo mật, không lộ cấu trúc code |
| Log exception | Debug được khi có lỗi production |
| Message tiếng Việt rõ ràng | User/Frontend dễ hiểu |

@hoidanit

**Prompt Tạo GlobalExceptionHandler**

Tôi cần tạo GlobalExceptionHandler cho dự án Spring Boot.

Context:
- Package: vn.hoidanit.springrestwithai.exception

Yêu cầu:
1. Tạo class GlobalExceptionHandler với @RestControllerAdvice
2. Xử lý các exception sau:
  - MethodArgumentTypeMismatchException → 400 Bad Request
  - HttpMessageNotReadableException → 400 Bad Request
  - HttpRequestMethodNotSupportedException → 405 Method Not Allowed
  - NoHandlerFoundException → 404 Not Found
  - Exception (catch-all) → 500 Internal Server Error

3. Tạo 2 custom exception:
  - ResourceNotFoundException (resourceName, fieldName, fieldValue)
  - DuplicateResourceException (resourceName, fieldName, fieldValue)

4. Thêm handler cho 2 custom exception:
  - ResourceNotFoundException → 404 Not Found
  - DuplicateResourceException → 409 Conflict

5. Message tiếng Việt, rõ ràng
6. KHÔNG để lộ stack trace trong response
7. Log exception ra console để debug

Chỉ trả về code Java, không giải thích.

## #4.8 Validation với @Valid

### 1.Vấn đề khi không có Validation

Hiện tại, API của chúng ta chấp nhận mọi dữ liệu client gửi lên
→ Dữ liệu sai vẫn được lưu vào database

| Vấn đề | Hậu quả |
|---|---|
| Không kiểm tra dữ liệu đầu vào | Dữ liệu rác trong database |
| Email sai format | Không gửi được mail, lỗi hệ thống |
| Tên rỗng | Hiển thị xấu, logic bị lỗi |
| Tuổi âm | Vô nghĩa, báo cáo sai |
| Password quá ngắn | Bảo mật kém |

**Nguyên tắc:** Không bao giờ tin dữ liệu từ client. Luôn validate trước khi xử lý.

### 2.Bean Validation là gì?

Bean Validation (JSR 380) là specification định nghĩa các annotations để validate dữ liệu trong Java.
Hibernate Validator là implementation phổ biến nhất, được Spring Boot tích hợp sẵn.

**Thêm dependency vào pom.xml:**
(Lưu ý cho Spring Boot 4.x: Nếu cần viết test cho validation, thêm dependency test)

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation-test</artifactId>
  <scope>test</scope>
</dependency>
```

## 3. Các Validation Annotations Phổ Biến

**Kiểm tra null/rỗng:**

| Annotation | Mô tả | Ví dụ |
|---|---|---|
| @NotNull | Không được null (nhưng có thể rỗng " ") | @NotNull Integer age |
| @NotEmpty | Không được null và không được rỗng | @NotEmpty List<String> items |
| @NotBlank | Không được null, rỗng, hoặc chỉ có khoảng trắng | @NotBlank String name |

Khi nào dùng cái nào?
**@NotBlank**: Dùng cho String bắt buộc (name, email, password...)
**@NotNull**: Dùng cho số, ngày, object bắt buộc (age, birthDate...)
**@NotEmpty**: Dùng cho Collection, Array bắt buộc có ít nhất 1 phần tử

**Kiểm tra chuỗi:**

| Annotation | Mô tả | Ví dụ |
|---|---|---|
| @Size(min, max) | Độ dài trong khoảng | @Size(min=2, max=50) String name |
| @Email | Đúng format email | @Email String email |
| @Pattern(regexp) | Khớp với regex | @Pattern(regexp="^[0-9]{10}$") String phone |

@hoidanit

**Bước 1:** Update entity

```
    @NotBlank(message = "Tên không được để trống")
    @Size(min = 2, max = 100, message = "Tên phải từ 2 đến 100 ký tự")
    private String name;

    @NotBlank(message = "Email không được để trống")
    @Email(message = "Email không đúng định dạng")
    private String email;

    @NotBlank(message = "Mật khẩu không được để trống")
    @Size(min = 6, message = "Mật khẩu phải có ít nhất 6 ký tự")
    private String password;

    @Min(value = 1, message = "Tuổi phải lớn hơn 0")
    @Max(value = 150, message = "Tuổi không hợp lệ")
    private Integer age;

    @Size(max = 255, message = "Địa chỉ không được quá 255 ký tự")
    private String address;
```

**Bước 2:** Update controller với **@Valid**

//todo

**Bước 3:** Xử lý global exception

```
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<ApiResponse<Object>> handleValidationException(
        MethodArgumentNotValidException ex) {

    List<String> errors = ex.getBindingResult()
            .getFieldErrors()
            .stream()
            .map(this::formatFieldError)
            .collect(Collectors.toList());

    return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(
            ApiResponse.badRequest("Dữ liệu không hợp lệ", errors)
    );
}

private String formatFieldError(FieldError fieldError) {
    return fieldError.getField() + ": " + fieldError.getDefaultMessage();
}
```

**Module 5: Spring Security**
*Bảo vệ ứng dụng với Spring Security*

## #5.1 Phân biệt Authentication vs Authorization

### 1. Authentication (Xác thực) là gì ?

**Authentication = "Bạn là ai?"** (Quá trình xác minh danh tính của người dùng)
Ví dụ thực tế:

- Khi vào sân bay, bạn phải xuất trình CMND/CCCD → Xác minh "bạn là Nguyễn Văn A"
- Khi đăng nhập Facebook, nhập email + password → Xác minh "bạn là chủ tài khoản này"
- Khi mở khóa iPhone bằng Face ID → Xác minh "bạn là chủ điện thoại"

Các phương thức Authentication phổ biến:

- **Username + Password** (truyền thống nhất)
- OTP qua SMS/Email
- Biometric (vân tay, khuôn mặt)
- Social Login (Login with Google, Facebook)
- Multi-Factor Authentication (MFA): kết hợp nhiều phương thức

**Kết quả của Authentication:**

✅ Thành công → Hệ thống biết "bạn là ai", cấp token/session
❌ Thất bại → Từ chối truy cập, yêu cầu đăng nhập lại

### 2. Authorization (Phân quyền) là gì ?

**Authorization = "Bạn được phép làm gì?" (**Quá trình xác định quyền hạn của người dùng đã được xác thực)

Ví dụ thực tế:

- Bạn đã vào được sân bay (authenticated), nhưng chỉ vào được phòng chờ của hạng vé bạn mua (authorized)

- Bạn đã đăng nhập Facebook (authenticated), nhưng chỉ xóa được bài viết của chính bạn (authorized), không xóa được của người khác

@hoidanit

**Các cơ chế Authorization phổ biến:**

- **Role-Based Access Control** (RBAC): Dựa trên vai trò (ADMIN, USER, MANAGER)
- Permission-Based: Dựa trên quyền cụ thể (CREATE_USER, DELETE_POST)
- Attribute-Based Access Control (ABAC)

**Kết quả của Authorization:**

✅ Có quyền → Cho phép thực hiện hành động
❌ Không có quyền → Từ chối với lỗi 403 Forbidden

## 3. So Sánh Authentication vs Authorization

| Tiêu chí | Authentication | Authorization |
|---|---|---|
| Câu hỏi | Bạn là ai ? | Bạn được làm gì ? |
| Thời điểm | Trước khi vào hệ thống | Sau khi đã vào hệ thống |
| Xác minh | Danh tính (Identity) | Quyền hạn (Permissions) |
| Phương thức | Password, OTP, Biometric | Roles, Permissions, Policies |
| HTTP Status nếu fail | 401 Unauthorized | 403 Forbidden |
| Có thể thay đổi? | Ít thay đổi (trừ đổi pass) | Thường xuyên (gán/bỏ quyền) |

## 4. Mối Quan Hệ Giữa Authentication và Authorization

Quy tắc vàng: Authentication phải xảy ra TRƯỚC Authorization. Không thể phân quyền (authorize) cho ai đó mà chưa biết người đó là ai (authenticate).

**Flow chuẩn:**

1. User gửi credentials (email + password)
2. Hệ thống xác thực → Authentication
3. Nếu đúng → Cấp token/session, lưu thông tin user (id, roles)
4. User gửi request kèm token
5. Hệ thống kiểm tra token → Biết user là ai
6. Hệ thống kiểm tra quyền → Authorization
7. Nếu có quyền → Xử lý request
8. Nếu không có quyền → 403 Forbidden

## #5.2 So Sánh Session vs Opaque Token vs JWT (JSON Web Token)

### 1. Session-Based Authentication

**Session = Phiên làm việc được lưu trữ trên server.**
Sau khi user đăng nhập thành công, server tạo một session, lưu thông tin user vào đó, và gửi Session ID về client (thường qua cookie).

**Flow hoạt động:**
1. User gửi username + password
2. Server xác thực thành công
3. Server tạo session, lưu vào memory/database/Redis
   Session: {
    sessionId: "abc123",
    userId: 5,
    roles: ["USER"]  }
4. Server gửi sessionId về client qua cookie
5. Client lưu cookie: sessionId=abc123
6. Mỗi request sau, browser tự động gửi cookie kèm theo
7. Server nhận sessionId, tìm session trong storage
8. Nếu tìm thấy → Biết user là ai → Xử lý request

**Ưu điểm:**
✅ Dễ revoke: Xóa session trên server → user bị logout ngay lập tức
✅ Kiểm soát tốt: Server có toàn quyền kiểm soát session
✅ Ít dữ liệu gửi đi gửi lại: Chỉ gửi sessionId (nhỏ gọn)
✅ Secure hơn: Thông tin nhạy cảm lưu trên server, không expose ra client

**Nhược điểm:**
❌ Khó scale horizontal: Cần shared storage (Redis) khi có nhiều server
❌ Tốn bộ nhớ server: Mỗi user = 1 session lưu trong RAM/Redis
❌ CSRF vulnerability: Dễ bị tấn công CSRF nếu không có protection
❌ Không phù hợp cho mobile app: Mobile không dùng cookie tốt

@hoidanit

## 2. Opaque Token-Based Authentication

**Opaque Token = Chuỗi ký tự ngẫu nhiên** (random string) được server generate và gửi về client. (token này chẳng có ý nghĩa gì, ngoại trừ là chuỗi ngẫu nhiên)

Client lưu token (localStorage, sessionStorage, memory) và gửi kèm mỗi request qua HTTP Header (thường là Authorization: Bearer <token>)

**Flow hoạt động:**

1. User gửi username + password
2. Server xác thực thành công
3. Server generates a random token: "a3f8b2c9d1e4f5..."
4. Server lưu token vào database/Redis:

```
Token: {
  token: "a3f8b2c9d1e4f5...",
  userId: 5,
  expiresAt: "2025-01-20T10:00:00"
}
```

5. Server gửi token về client
6. Client lưu token (localStorage hoặc memory)
7. Mỗi request, client gửi: Authorization: Bearer a3f8b2c9d1e4f5...
8. Server nhận token, tìm trong database/Redis
9. Nếu tìm thấy và chưa expire → Xử lý request

**Ưu điểm:**

✅ Không dùng cookie: Tránh CSRF, phù hợp mobile app

✅ Flexible: Dễ dàng gửi token qua nhiều cách (header, query param)

✅ Cross-domain: Không bị giới hạn same-origin như cookie

**Nhược điểm:**

❌ Vẫn cần storage: Server vẫn phải lưu token (giống session)

❌ Overhead: Mỗi request phải query database/Redis để verify token

❌ XSS vulnerability: Nếu lưu localStorage, dễ bị XSS steal token

@hoidanit

## 3. JWT (JSON Web Token)

**JWT = Token tự chứa thông tin (self-contained), không cần server lưu trữ.**
JWT gồm 3 phần: Header.Payload.Signature, được mã hóa bằng thuật toán (HS256, RS256).

**Flow hoạt động:**

1. User gửi username + password
2. Server xác thực thành công
3. Server tạo JWT, đưa thông tin user vào payload
4. Server ký JWT bằng secret key
5. Server gửi JWT về client
6. Client lưu JWT (localStorage, memory, cookie)
7. Mỗi request, client gửi: Authorization: Bearer <JWT>
8. Server nhận JWT
9. Server verify signature (kiểm tra JWT có bị sửa đổi không)
10. Server decode payload → Biết user là ai
11. KHÔNG CẦN query database → Xử lý request ngay

**Ưu điểm:**

✅ Stateless: Server không cần lưu trữ gì, dễ scale

✅ Hiệu năng cao: Không query database mỗi request

✅ Cross-service: Dễ dàng share JWT giữa nhiều service (microservices)

✅ Mobile-friendly: Không dùng cookie, phù hợp mobile app

✅ Decentralized: Service khác có thể verify JWT mà không cần gọi về auth server

**Nhược điểm:**

❌ Không thể revoke ngay: Không thể hủy JWT trước khi expire (trừ khi dùng blacklist)

❌ Token size lớn: JWT chứa nhiều thông tin → size lớn hơn session ID

❌ Expose thông tin: Payload có thể decode → không nên chứa thông tin nhạy cảm

❌ Secret key leak: Nếu secret key bị lộ → attacker tạo JWT giả được

@hoidanit

Nhiều hệ thống thực tế dùng kết hợp JWT + Refresh Token:

**Access Token (JWT):**

- Short-lived (5-15 phút)
- Stateless, không cần query DB
- Dùng để access resources

**Refresh Token (Random String):**

- Long-lived (7-30 ngày)
- Lưu trong database/Redis
- Dùng để lấy Access Token mới

**Flow hoạt động (Access Token + Refresh Token)**

1. Login → Server trả về:
   - accessToken (JWT, expire 15 phút)
   - refreshToken (random string, expire 30 ngày)

2. Client lưu cả 2 tokens

3. Mỗi request → Gửi accessToken (JWT)

4. Khi accessToken expire (15 phút):
   - Client gửi refreshToken lên server
   - Server verify refreshToken (query DB/Redis)
   - Server trả về accessToken mới

5. Khi logout hoặc security issue:
   - Xóa refreshToken khỏi database
   - User phải login lại

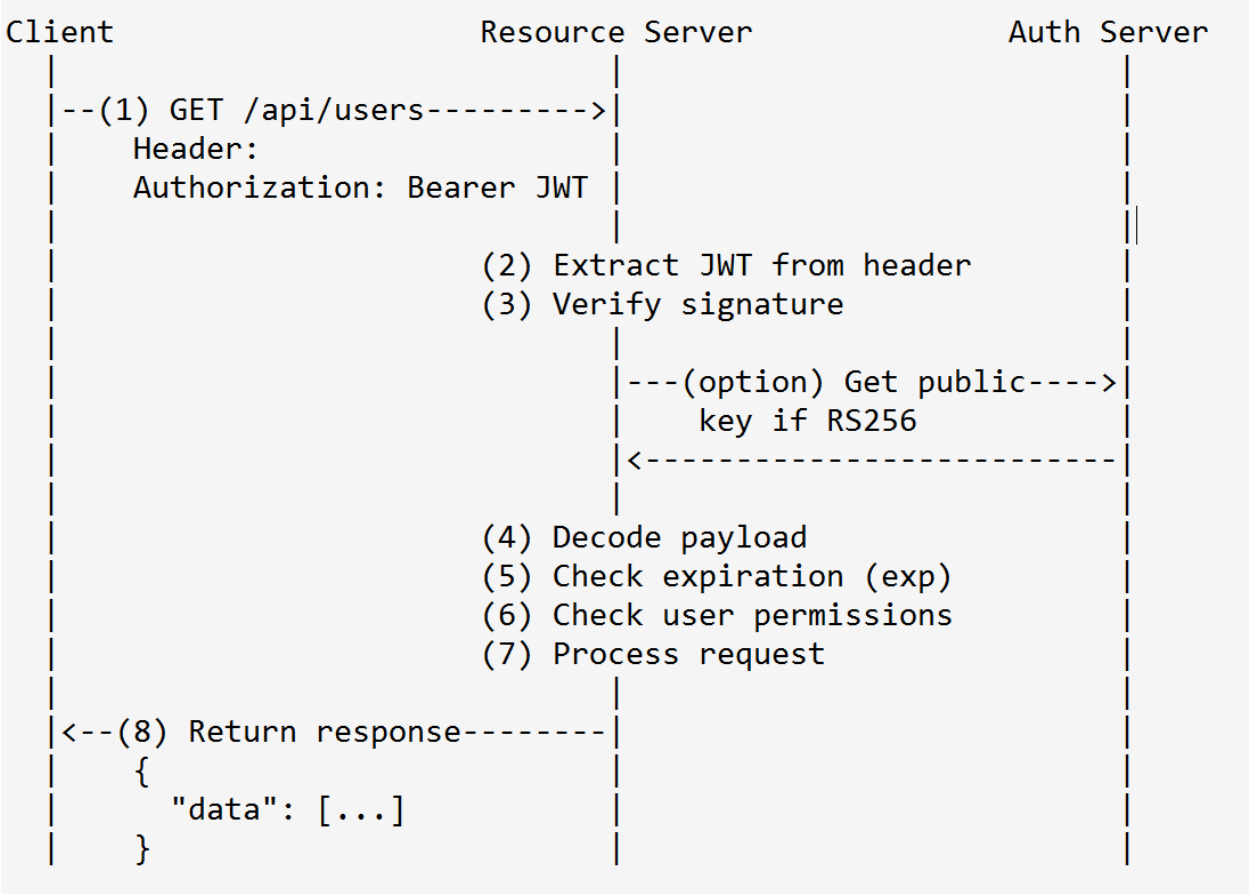
@hoidanit

### #5.3 JWT Deep Dive: Flow & Security Concerns

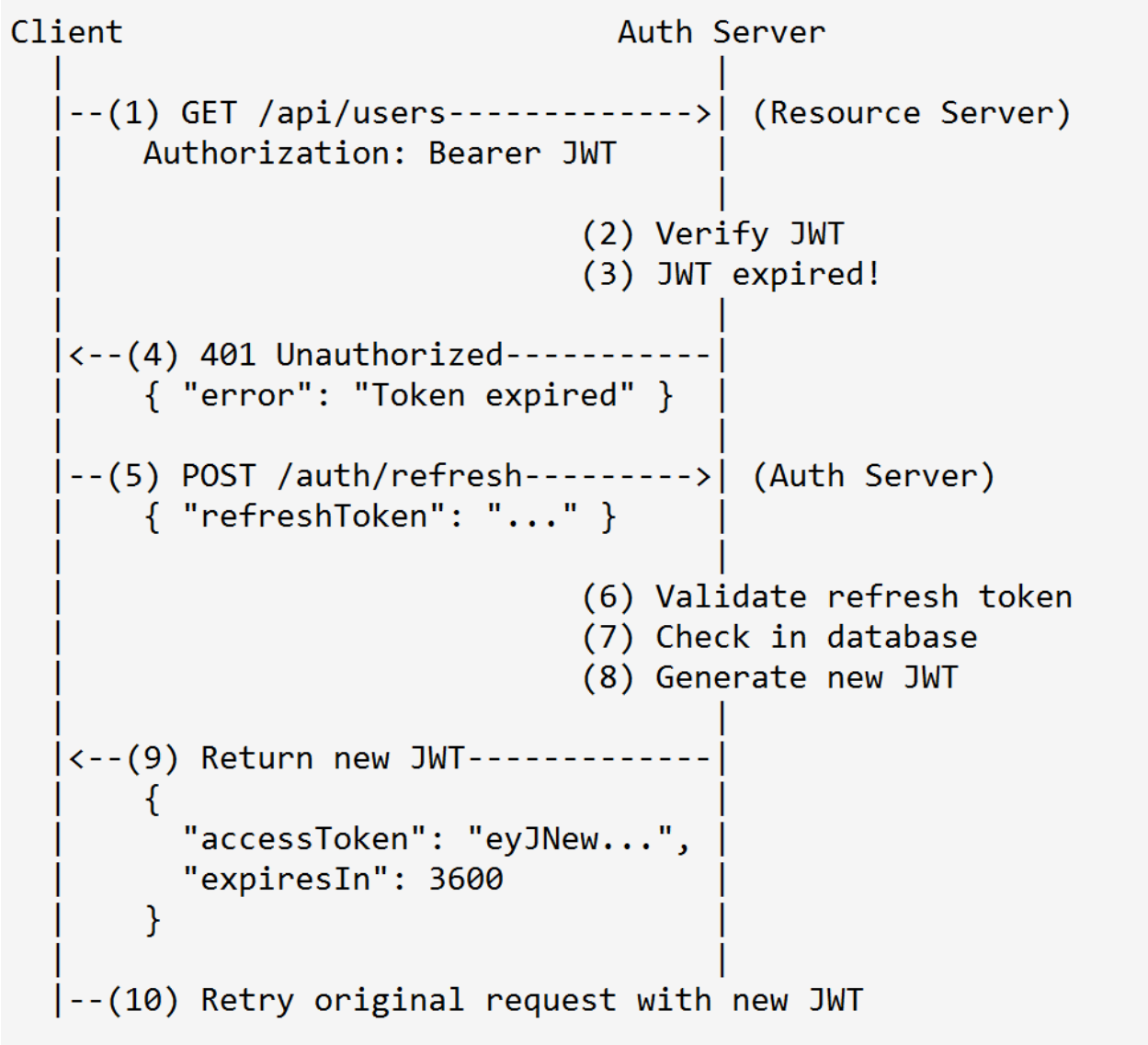
Flow 1: Login & Get JWT

```
Client                              Auth Server
  |                                      |
  |---(1) POST /auth/login---------->|
  |    { email, password }               |
  |                                      |
  |                              (2) Validate credentials
  |                              (3) Query user from DB
  |                              (4) Create JWT payload:
  |                                   {
  |                                     "userId": 5,
  |                                     "username": "hoidanit@example.com",
  |                                     "roles": ["USER"],
  |                                     "iat": 1706184000,
  |                                     "exp": 1706187600
  |                                   }
  |                              (5) Sign JWT with secret key
  |                                      |
  |<--(6) Return JWT-----------------|
  |    {                                 |
  |      "accessToken": "eyJhbG...",     |
  |      "tokenType": "Bearer",          |
  |      "expiresIn": 3600               |
  |    }                                 |
  |                                      |
  |--(7) Store JWT (memory/localStorage)
```

## Flow 2: Access Protected Resource



## Flow 3: Token Expired - Refresh Flow



## Concern 1: JWT Bị Đánh Cắp (Token Theft)

**Cách tấn công:**
XSS (Cross-Site Scripting): Inject malicious script → steal JWT từ localStorage
MITM (Man-in-the-Middle): Sniff HTTP traffic → steal JWT

- Sử dụng HTTPS (thay vì http)
- Short expiration time
  "exp": 1706184900  // 15 phút thay vì 24 giờ
- Lưu JWT ở đâu an toàn?
  ❌ localStorage`: Dễ bị XSS steal
  ❌ sessionStorage: Vẫn dễ bị XSS
  ✅ Memory (JavaScript variable): Mất khi refresh
  ✅ HttpOnly Cookie: Không thể access bằng JavaScript

- Content Security Policy (CSP): Content-Security-Policy: default-src 'self'; script-src 'self'
- Implement refresh token rotation
  Refresh token chỉ dùng 1 lần
  Mỗi lần refresh → tạo refresh token mới

## Concern 2: Algorithm Confusion Attack

**Cách tấn công:** Attacker thay đổi algorithm trong header từ RS256 → HS256, và dùng public key làm secret key để sign.

Nếu server verify mà không check algorithm → attacker có thể tạo JWT hợp lệ.

Quy tắc: LUÔN chỉ định rõ algorithm khi verify, KHÔNG lấy từ JWT header.

## Concern 3: Sensitive Data in Payload

JWT payload chỉ được encode (Base64), KHÔNG được encrypt. Bất kỳ ai cũng decode được.

❌ KHÔNG BAO GIỜ lưu trong JWT:

- Password (plain text hoặc hashed)
- Credit card number...

@hoidanit

## Concern 4: JWT Không Thể Revoke

JWT là stateless, server không lưu trữ. Một khi JWT được tạo và chưa expire, không thể hủy.

**Cách 1: JWT Blacklist**

// Khi user logout, lưu JWT vào blacklist

// Khi verify JWT, check blacklist

Nhược điểm: Mất tính stateless của JWT.

**Cách 2:** Short-lived JWT + Refresh Token

Access token (JWT): 15 phút

Refresh token: 30 ngày, lưu trong database

Khi logout/ban user → xóa refresh token

**Cách 3:** Version/Generation trong JWT

// Khi verify, check version trong database

@hoidanit

## #5.4 [AI Warning] Tại Sao Không Nên Vibe Coding Security

### 1. Security Code ≠ CRUD Code

Điểm khác biệt cốt lõi:

| Khía cạnh | CRUD Code | Security Code |
|---|---|---|
| Phát hiện lỗi | Ngay lập tức | Có thể không bao giờ biết cho đến khi bị hack |
| Hậu quả | UX kém | Mất dữ liệu, tiền bạc, kiện tụng |
| Test | Unit test đơn giản | Cần security testing chuyên sâu |
| Độ phức tạp | Logic rõ ràng | Nhiều edge cases ẩn |
| AI hiểu không | Có | Không đủ context |

### 2. Tại Sao AI Hay Sai Trong Security ?

**Lý do 1:** AI được train trên Internet
Internet chứa rất nhiều bad code examples

**Lý do 2:** AI không biết context của bạn

```
// AI generate:
@PreAuthorize("hasRole('ADMIN')")
@DeleteMapping("/users/{id}")
public void deleteUser(@PathVariable Long id) {
    userService.deleteById(id);
}

// Nhìn qua có vẻ OK, nhưng:
// ❌ Không check: ADMIN có thể xóa chính mình không?
// ❌ Không check: Có thể xóa SUPER_ADMIN không?
// ❌ Không check: Xóa user có cascade delete data quan trọng không?
// ❌ Không log: Ai xóa, xóa ai, khi nào?

// AI không biết business rules của công ty bạn
```

@hoidanit

**Lý do 3:** AI tối ưu cho "code chạy được", không phải "code an toàn"
//todo

**Lý do 4:** Security cần hiểu CÁCH TẤN CÔNG
//todo

## 3. Khi Nào CÓ THỂ Dùng AI (Với Điều Kiện)

**KHÔNG BAO GIỜ:**
❌ Vibe code security features
❌ Copy-paste AI security code mà không hiểu
❌ Trust AI với password hashing, JWT, authorization
❌ Skip security review vì "AI đã generate"
❌ Tin "code chạy được" = "code an toàn"

**LUÔN LUÔN:**
✅ Hiểu 100% mọi dòng security code
✅ Verify với official docs
✅ Review với security checklist
✅ Test attack scenarios
✅ Assume breach mindset

@hoidanit

## 4. Security Development Workflow

**1. Understand Requirements**

↓

What are we protecting?
Who can access what?
What are the threats?

**2. Design Security Controls**

↓

Authentication mechanism
Authorization rules
Data protection strategy

**3. Implement (Minimal AI)**

↓

Use proven libraries
Follow OWASP (Open Worldwide Application Security Project) guidelines
Simple, clear code

**4. Review**

↓

Self-review với security checklist
Peer review
Security team review (if available)

**5. Test**

↓

Unit tests
Integration tests
Security tests (attack scenarios)
Penetration testing (if critical)

**6. Monitor**

↓

Audit logging
Anomaly detection
Incident response plan

@hoidanit

## #5.5 Cấu hình Spring Security cơ bản

Mục tiêu:

- Hiểu kiến trúc của Spring Security (Filter Chain)
- Biết cách configure Spring Security từ đầu KHÔNG dùng AI

### 1. Spring Security Là Gì ?

**Spring Security là framework bảo mật mạnh mẽ và highly customizable** cho các ứng dụng Spring-based.

Nó cung cấp:

- **Authentication**: "Bạn là ai?"
- **Authorization**: "Bạn được làm gì?"
- **Protection**: Chống các attack phổ biến (CSRF, Session Fixation, Clickjacking...)

Spring Security hoạt động dựa trên Servlet **Filters**:

```
HTTP Request
     ↓
[Servlet Container]
     ↓
┌──────────────────────────────────────────────────┐
│    Spring Security Filter Chain                  │
├──────────────────────────────────────────────────┤
│ 1. SecurityContextPersistenceFilter  ← Load SecurityContext
│ 2. LogoutFilter                      ← Handle logout
│ 3. UsernamePasswordAuthFilter        ← Handle login
│ 4. BasicAuthenticationFilter         ← HTTP Basic Auth
│ 5. RequestCacheAwareFilter           ← Cache request
│ 6. SecurityContextHolderFilter       ← Store SecurityContext
│ 7. AnonymousAuthenticationFilter     ← Anonymous user
│ 8. SessionManagementFilter           ← Manage session
│ 9. ExceptionTranslationFilter        ← Handle exceptions
│ 10. FilterSecurityInterceptor        ← Authorization check
└──────────────────────────────────────────────────┘
     ↓
[Your Controllers]
     ↓
Response
```

**Cách hoạt động:**

- Request đi qua từng filter theo thứ tự
- Mỗi filter làm một nhiệm vụ cụ thể
- Nếu authentication/authorization fail → ném exception
- ExceptionTranslationFilter bắt exception → trả về 401/403
- Nếu pass hết filters → request đến controller

## 2. Setup Dependencies

Thêm Spring Security Starter
Mở **pom.xml,** thêm dependency:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security-test</artifactId>
  <scope>test</scope>
</dependency>
```

Sau khi thêm dependency, **restart ứng dụng**.
✅ Spring Security được enable tự động
✅ Tất cả endpoints đều bị protect
✅ Default username: **user**
✅ Default password: **In ra console** (random mỗi lần khởi động)

//todo: test logic session (default)

Cấu hình Username/Password, tham khảo tại đây
//update file : **application.properties**

```
spring.security.user.name=hoidanit
spring.security.user.password=123456
```

@hoidanit

## #5.6 [AI] Filter Chain & Disable Security

1. Request vào → đi qua filter
2. Mỗi filter làm 1 việc cụ thể
3. Filter (FilterSecurityInterceptor) check authorization:
   - User có quyền access endpoint này không?
   - Nếu KHÔNG → throw AccessDeniedException
4. ExceptionTranslationFilter catch exception:
   - Chưa login → 401 Unauthorized
   - Đã login nhưng không có quyền → 403 Forbidden
5. Nếu pass tất cả filters → Request đến Controller

Prompt:

Tôi đang làm dự án Spring Boot REST API, cần config Spring Security để allow tất cả requests (không cần authentication) phục vụ testing.

Yêu cầu:
1. Tạo SecurityConfig class trong package vn.hoidanit.springrestwithai.config

2. SecurityFilterChain bean:
   - Disable CSRF
   - Allow tất cả requests (.anyRequest().permitAll())

Annotations:
- @Configuration
- @EnableWebSecurity

Chỉ trả về code, không giải thích.

## #5.7 [AI] Password Encoding với BCrypt

Mục tiêu:

- Hiểu tại sao KHÔNG BAO GIỜ lưu password dạng plain text
- Biết BCrypt hoạt động như thế nào
- Implement PasswordEncoder bean
- Lưu User với hashed password khi đăng ký

### 1. Vấn Đề: Plain Text Password

**Scenario 1:** Database bị leak
Hacker access database → Thấy ngay tất cả passwords

**Scenario 2:** Internal threat
Database admin, DevOps engineer có thể thấy passwords

**Scenario 3:** Users reuse passwords
Password email = password bank = password work
Leak 1 site → Tất cả accounts bị compromise

Quy tắc vàng:
KHÔNG BAO GIỜ lưu password dạng plain text
LUÔN LUÔN hash password trước khi lưu database

### 2. BCrypt Là Gì ?

BCrypt = Password Hashing Algorithm

3 Đặc điểm quan trọng:
**Đặc điểm 1:** Auto-generated Salt
Salt = Random string thêm vào password trước khi hash.
**Không có salt:**
hash("123456") → "abc123def456"
hash("123456") → "abc123def456"  // Luôn giống nhau!

// Nếu 2 users cùng password "123456":
// Database:
// john  | abc123def456
// jane  | abc123def456  ← Hacker biết ngay 2 người cùng password

**Có salt:**
hash("123456" + "saltA") → "xyz789abc"
hash("123456" + "saltB") → "def456ghi"  // Khác nhau!

// Database:
// john  | xyz789abc
// jane  | def456ghi  ← Hacker KHÔNG biết 2 người cùng password

**BCrypt tự động:**
- Generate random salt
- Lưu salt trong hash (không cần lưu riêng)

$2a$10$N9qo8uLOickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWy
│  │  │ └─ Salt (22 chars)      └─ Hash (31 chars)
│  │  └─ Work factor
│  └─ Version
└─ BCrypt ID

**Đặc điểm 2:** Slow by Design
BCrypt cố tình chậm để chống brute force.
Work Factor = Số lần lặp thuật toán:

Work Factor 10 = 2^10 = 1,024 iterations
Work Factor 12 = 2^12 = 4,096 iterations

| Work Factor | Time/hash | Brute force 8-char password |
|---|---|---|
| 10 | ~0.1s | ~12 years |
| 12 | ~0.4s | ~50 years |

Khuyến nghị: Work factor 10-12 cho web apps.

**Đặc điểm 3:** One-way Function
Password → Hash ✓ (Có thể)
Hash → Password ✗ (KHÔNG thể reverse)

@hoidanit

## 4. Dùng AI Generate Code

Prompt:

> Tôi đang làm dự án Spring Boot, cần implement password encoding với BCrypt.
>
> Context:
> - Entity User đã có (fields: id, email, password, name, age, createdAt, updatedAt)
> - Package: vn.hoidanit.springrestwithai.**config**
>
> Yêu cầu:
> 1. Tạo PasswordEncoder bean trong SecurityConfig
>  - Dùng BCryptPasswordEncoder
>
> 2. Update UserService với method create(User user):
>  - Hash password bằng passwordEncoder.encode()
>
> 3. Inject PasswordEncoder vào UserService:
>  - Dùng Constructor Injection
>  - Không dùng @Autowired trên field
>
> Yêu cầu kỹ thuật:
> - Không dùng Lombok
> - Constructor Injection với private final
> - Không cần giải thích, chỉ code

@hoidanit

## #5.8 [AI] UserDetailsService - Login từ Database

Vấn đề hiện tại:
❌ Chưa có cơ chế login
❌ Spring Security không biết load user từ đâu
❌ Không verify được password

Mục tiêu video này:
✅ Load user từ database khi login
✅ Verify password tự động
✅ Test login thành công

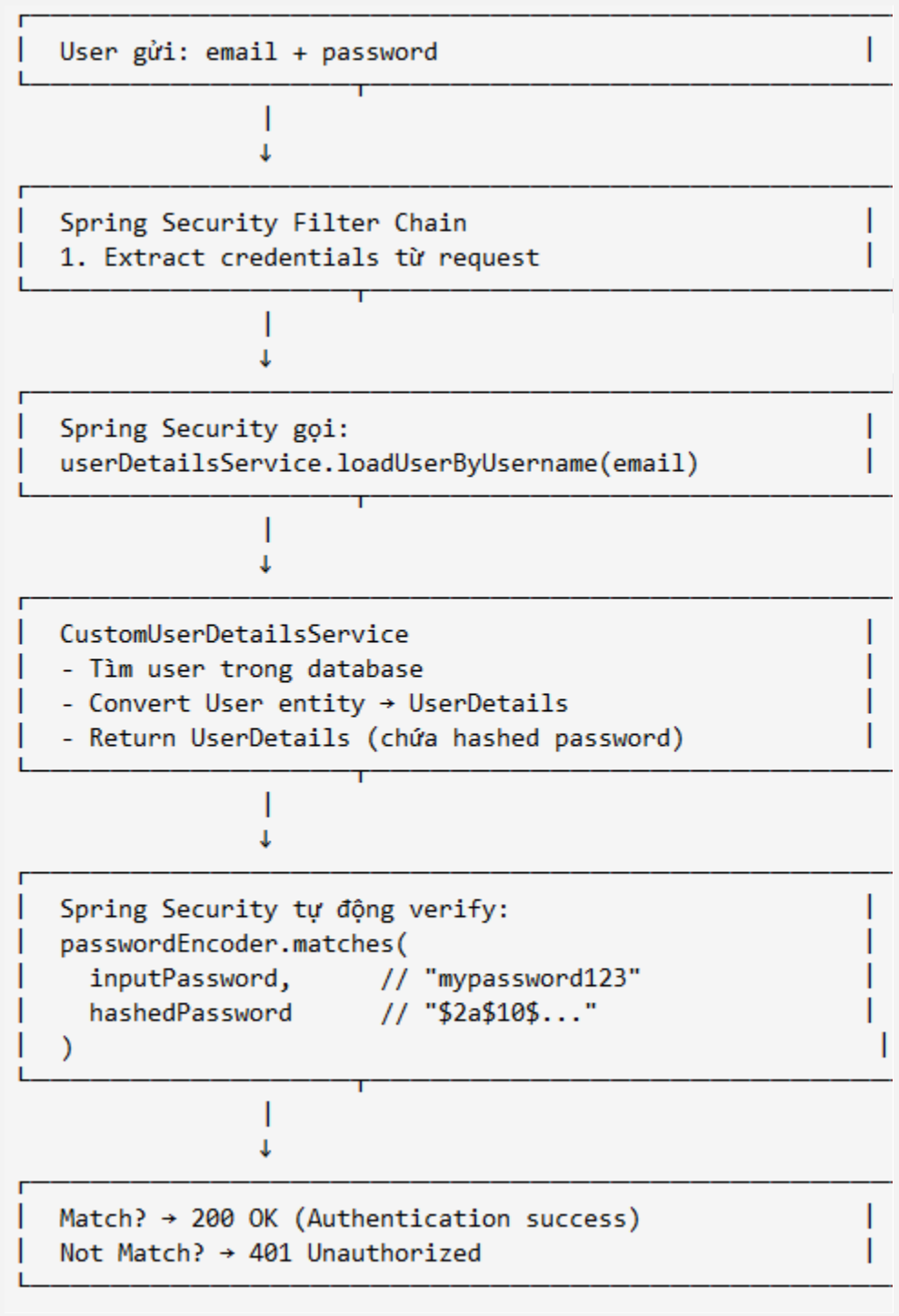
### 1. UserDetailsService Là Gì ?

UserDetailsService = Interface Spring Security dùng để load user information.

**Chỉ có 1 method:**
- **Input**: Username (trong app của chúng ta là email)
- **Output**: UserDetails object
- **Exception**: UsernameNotFoundException nếu không tìm thấy

@hoidanit

Vai trò trong login flow:



Prompt AI Generate Code:

Tôi đang làm Spring Boot với Spring Security, cần implement UserDetailsService để load user từ database.

Context:
- Entity User: id, email, password (đã hash), name, age
- Package: vn.hoidanit.springrestwithai.config

Yêu cầu:
Tạo CustomUserDetailsService implement UserDetailsService:

1. Inject UserRepository (constructor injection, private final)

2. Method loadUserByUsername(String email):
  - Tìm user: userRepository.findByEmail(email)
  - Không tìm thấy → throw UsernameNotFoundException("User not found with email: " + email)
  - Tìm thấy → return UserDetails:
    * username = user.getEmail()
    * password = user.getPassword()
    * roles = "USER"
    * Dùng org.springframework.security.core.userdetails.User.builder()

3. Annotation: @Service

Không dùng Lombok, không dùng @Autowired trên field.
Chỉ code, không giải thích.

### #5.9 Phân biệt OAuth2 Resource Server vs OAuth2 Authorization Server

1.So sánh Resource Server và Authorization Server

| Feature | Resource Server | Authorization Server |
|---|---|---|
| Dependency | oauth2-resource-server | oauth2-authorization-server |
| Main Job | Validate tokens | Issue tokens |
| JWT Support | ✅ Decode & validate | ✅ Generate & sign |
| OAuth2 Flows | ❌ Không | ✅ Authorization Code, Client Credentials... |
| Login UI | ❌ Không | ✅ Có thể có |
| Token Endpoint | ❌ Không | ✅ /oauth2/token |
| Protect APIs | ✅ Main purpose | ❌ Không (trừ khi add resource-server) |
| User Management | ❌ Không | ✅ Có |
| Client Management | ❌ Không | ✅ Có |
| Complexity | Đơn giản | Phức tạp |
| Lines of Code | ~50-100 | ~500-1000+ |

2. Khi Nào Dùng Gì ?

**Scenario 1: Simple REST API (90% use cases)**

Cần:

- API backend với JWT authentication
- Login endpoint đơn giản
- 1 application (không tách riêng)

**Tự implement**: (→ dùng oauth2-resource-server)

- Login endpoint tự viết (POST /auth/login)
- Generate JWT bằng JwtEncoder
- Validate JWT bằng JwtDecoder (từ resource-server)

**Ưu điểm**: đơn giản, dễ maintain, đủ cho hầu hết apps.

**Scenario 2: OAuth2 Provider (Enterprise)**

Cần:

- Authorization Server riêng cho nhiều applications
- Support OAuth2 standard flows
- Third-party integrations
- Multiple clients (web, mobile, desktop)

Sử dụng: **Authorization Server + Resource Server** (cho API endpoints)

Ưu điểm:
✅ OAuth2 compliant
✅ Support standard flows
✅ Multiple clients
✅ Suitable for microservices

Nhược điểm: phức tạp, nhiều code, khó debug

**Scenario 3: Use External Auth Provider (Recommended)**

Dùng:

- Keycloak (free, open-source)
- Auth0 (commercial)
- Okta (commercial)
- AWS Cognito
- Firebase Auth

**Cần**: oauth2-resource-server

**Config:**

```
spring.security.oauth2.resourceserver.jwt.issuer-uri=https://your-keycloak.com/realms/myrealm
```

**Ưu điểm:**
- ✅KHÔNG cần implement Auth Server
- ✅ Production-ready
- ✅ Features đầy đủ (MFA, Social login, etc.)
- ✅ Dễ scale

@hoidanit

## 3. Cách dùng trong khóa học

Chúng ta sẽ dùng: oauth2-resource-server, ứng với **Scenario 1**

**Lý do:**
- ✅ Đơn giản, dễ học
- ✅ Đủ cho 90% use cases
- ✅ Tự implement login endpoint (hiểu rõ flow)
- ✅ Không phụ thuộc external services

KHÔNG dùng: oauth2-authorization-server
Lý do:
- ❌ Quá phức tạp cho beginner
- ❌ Overkill cho small/medium apps
- ❌ Khó maintain

Ngoài ra, chúng ta không tự viết code phần filter để xử lý jwt, ví dụ:

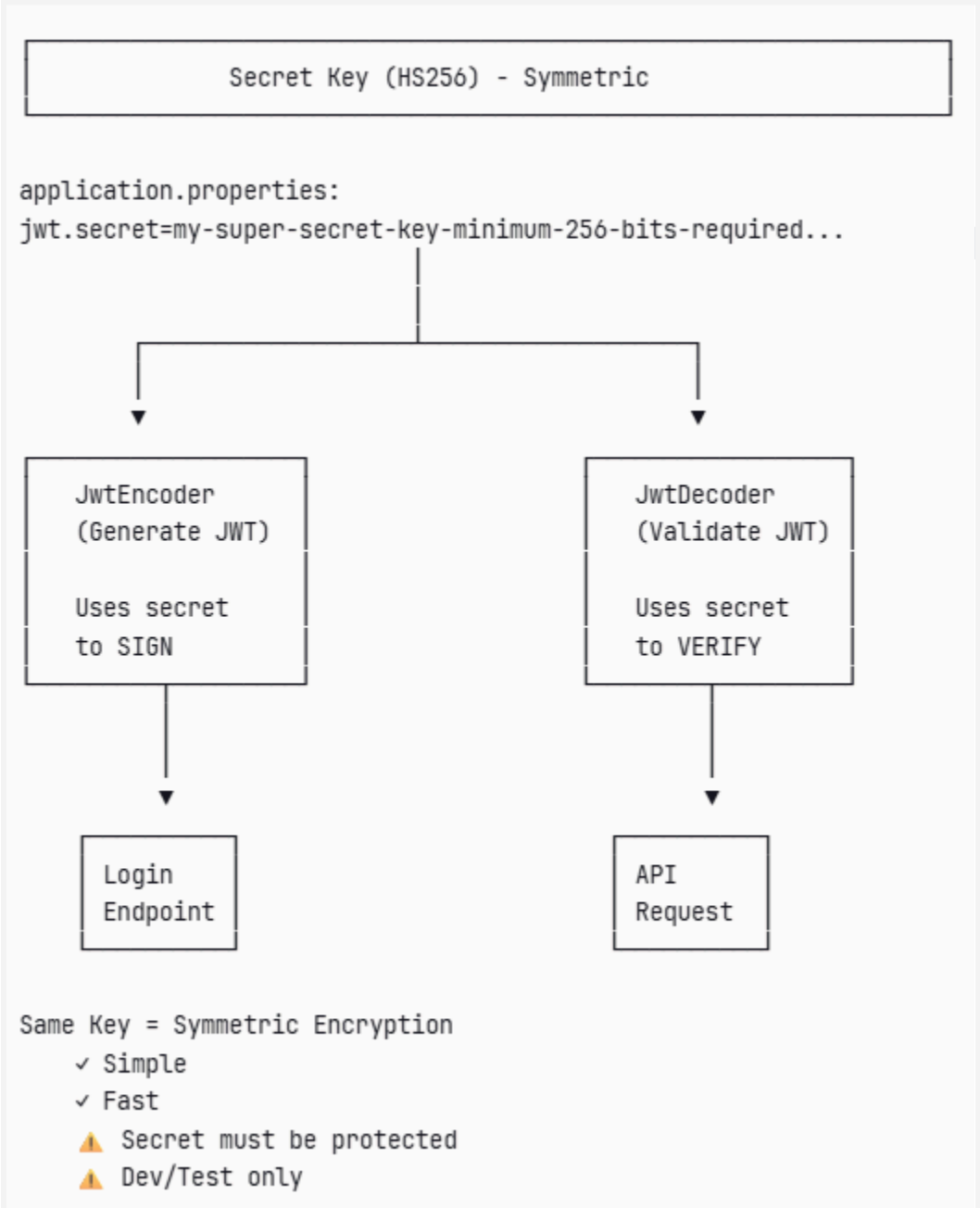
Không viết custom filter như cách hướng dẫn tại video này, sử dụng **oauth-resource-server**
//pom.xml

```
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security-oauth2-resource-server</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security-oauth2-resource-server-test</artifactId>
    <scope>test</scope>
  </dependency>
```

### #5.10 [AI] Cấu hình JWT với OAuth2 Resource Server

//todo: giới thiệu nhanh về jwt
Luồng sử dụng secret:



Prompt sử dụng:

Tôi đang làm Spring Boot 4 REST API, cần config Spring Security để hỗ trợ JWT authentication.

CONTEXT:
- Package: vn.hoidanit.springrestwithai.config
- JWT Algorithm: HS256 (symmetric key)
- JWT secret từ application.properties: ${jwt.secret}

YÊU CẦU:
1. Class: SecurityConfig
  - Inject jwt.secret bằng @Value("${jwt.secret}"). Tạo random secret trong application.properties

2. Bean SecurityFilterChain filterChain(HttpSecurity http):
  - Session: STATELESS (SessionCreationPolicy.STATELESS)
  - Public endpoints: /, /auth/**
  - Protected: anyRequest().authenticated()
  - OAuth2 Resource Server với jwt (default)

3. Bean JwtDecoder jwtDecoder():
  - Convert jwt.secret String → SecretKey (SecretKeySpec, "HmacSHA256")
  - Return NimbusJwtDecoder.withSecretKey(secretKey).macAlgorithm(MacAlgorithm.HS256).build()

4. Bean JwtEncoder jwtEncoder():
  - Convert jwt.secret String → SecretKey (SecretKeySpec, "HmacSHA256")
  - Return new NimbusJwtEncoder(new ImmutableSecret<>(secretKey))

Không dùng Lombok, không dùng @Autowired trên field.
Chỉ code, không giải thích.

### #5.11 [AI] Login Endpoint - Trả về Access Token

POST /api/v1/auth/login
{ email, password: "plain" }
↓
Load user from DB → password: "$2a$10$..."
↓
Verify: BCrypt.matches("plain", "$2a$10$...") → ✓
↓
Generate JWT
↓
Return: { accessToken: "eyJhbGci..." }
↓
Client saves token → localStorage

GET /api/v1/users/1
Authorization: Bearer eyJhbGci...
↓
Extract JWT from header
↓
JwtDecoder.decode(token):
1. Split → header, payload, signature
2. Verify signature with secret_key → ✓
3. Check expiration → ✓
4. Extract claims → sub: "test@test.com"
↓
Create Authentication object
↓
Store in SecurityContext
↓
Check authorization: .authenticated() → ✓
↓
Process request → Return data

@hoidanit

After .... (Token Expired)

GET /api/v1/users/1
Authorization: Bearer eyJhbGci... (expired)
  ↓
JwtDecoder.decode(token):
  1. Verify signature → ✓
  2. Check expiration → ✗ EXPIRED
  ↓
Return: 401 Unauthorized
  ↓
Client needs to login again (or use refresh token)

@hoidanit

Prompt generate code:

MỤC ĐÍCH: Tạo login endpoint POST /auth/login trả về JWT access token.

YÊU CẦU 4 FILES:

**FILE 1: LoginRequest.java (DTO)**
Package: vn.hoidanit.springrestwithai.model.dto.request
Fields: email: String; password: String. Getters/setters, không dùng Lombok

FILE 2: LoginResponse.java (DTO)
Package: vn.hoidanit.springrestwithai.model.dto.response
Fields: accessToken: String ;  tokenType: String (default "Bearer"); expiresIn: long (milliseconds)
Constructor with all fields, getters/setters, không dùng Lombok

FILE 3: JwtService.java
Package: vn.hoidanit.springrestwithai.service
- @Service
- Inject JwtEncoder bean
- Inject jwt.expiration từ application.properties (@Value). tạo tham số trong file application.properties
- Method: String **generateToken**(Authentication authentication)
  * Create JwtClaimsSet với:
    - subject: authentication.getEmail()
    - issuedAt: Instant.now()
    - expiresAt: now + jwt.expiration
    - claim "roles": extract từ authentication.getAuthorities()
  * Encode với JwtEncoder (JwsHeader với HS256)
  * Return token string

FILE 4: **AuthController**.java
Package: vn.hoidanit.springrestwithai.controller
- @RestController
- Inject AuthenticationManager bean
- Inject JwtService bean
- Inject jwt.expiration (@Value)
- Method: POST /login
  * @PostMapping("/login")
  * @RequestBody LoginRequest
  * Authenticate với AuthenticationManager.authenticate()
  * Generate JWT với jwtService.generateToken()
  * Return LoginResponse(accessToken, "Bearer", expiresIn)

REQUIREMENTS:
- Constructor injection (private final fields)
- Không dùng Lombok
- Không dùng @Autowired trên fields
- Code phải compile được
- Imports đầy đủ

OUTPUT: Code hoàn chỉnh, không giải thích.

@hoidanit

**Module 6: Automation Testing (JUnit & Mockito)**
*Biết cách viết unit test và integration test để phát hiện bug sớm và refactor code an toàn trong ứng dụng Java Spring*

**#6.1 Testing Mindset - Tại sao cần viết test ?**

1. Vấn đề thực tế

Tình huống quen thuộc:

- Code chạy đúng trên máy mình, deploy lên server lỗi
- Sửa bug A xong, phát sinh bug B ở chỗ khác
- Refactor code, không biết có làm hỏng gì không
- Mỗi lần release phải test lại thủ công hàng chục tính năng

2.Automation Test là gì ?

Automation Test = Viết code để test code

| Manual Testing | Automation Testing |
|---|---|
| Testers tự click, nhập liệu, kiểm tra | Code tự động chạy và kiểm tra |
| Chậm, tốn thời gian | Nhanh (hàng trăm test/vài giây) |
| Dễ nhầm khi lặp lại | Chính xác, nhất quán |
| Đánh giá UI/UX, kiểm thử thăm dò (Exploratory testing) | Kiểm thử hồi quy (Regression testing), CI/CD |

3. Tại sao Developer cần viết test ?

- Phát hiện bug sớm: Tìm lỗi ngay khi code, không đợi QA báo
- Refactor an toàn: Thay đổi code mà không sợ làm hỏng tính năng cũ
- Documentation sống: Test mô tả code hoạt động như thế nào
- Tiết kiệm thời gian: Chạy 100 test cases trong 10 giây thay vì 2 giờ manual
- Tự tin khi deploy: Biết chắc code hoạt động đúng

@hoidanit

## 4.Test Pyramid

| Loại | Tốc độ | Số lượng | Test cái gì |
|---|---|---|---|
| Unit Test | Rất nhanh | Nhiều nhất (70%) | Từng function/method riêng lẻ |
| Integration Test | Trung bình | Vừa phải (20%) | Nhiều components kết hợp |
| E2E Test | Chậm | Ít nhất (10%) | Toàn bộ flow từ UI đến DB |

## 5. Khi nào nên viết test

**Nên viết:**

- Business logic quan trọng (tính tiền, validate dữ liệu)
- Code được sử dụng nhiều nơi (utility functions)
- Bug đã từng xảy ra (viết test để không lặp lại)
- Trước khi refactor code phức tạp

**Có thể bỏ qua:**

- Prototype, POC (proof of concept)
- Code đơn giản (getter/setter)
- UI thay đổi liên tục

## 6. Mindset quan trọng

- Test không phải để chứng minh code đúng, mà để tìm ra chỗ sai
- Code không có test = code không thể maintain
- Viết test là đầu tư, không phải chi phí

Trong khóa học này: **Dùng AI để viết test nhanh hơn, nhưng vẫn phải hiểu và review.**

@hoidanit

## #6.2 Unit Test vs Integration Test vs E2E

### 1. Unit Test - Test từng "viên gạch"

**Định nghĩa:** Test một function/method riêng lẻ, tách biệt khỏi dependencies

**Đặc điểm:**

- Nhanh (chạy trong milliseconds)
- Độc lập (không cần database, network, file system)
- Dùng Mock để giả lập dependencies

Ví dụ:
Test UserService.calculateAge()
- Input: birthDate = 1990-01-01
- Expected Output: 35
- KHÔNG cần database, KHÔNG cần repository thật

**Khi nào dùng:**

- Business logic (tính toán, validate)
- Utility functions
- Chiếm 70% tổng số test

### 2. Integration Test - Test "nhiều viên gạch ghép lại"

**Định nghĩa:** Test sự kết hợp giữa nhiều components với nhau

**Đặc điểm:**

- Chậm hơn Unit Test
- Cần môi trường thật hoặc giả lập (H2 database, TestContainers)
- Test luồng dữ liệu giữa các layers

Ví dụ:
Test UserController → UserService → UserRepository → Database
- Gọi API POST /api/users
- Kiểm tra user được lưu vào database
- Kiểm tra response trả về đúng format

@hoidanit

**Khi nào dùng:**

- Test API endpoints
- Test tương tác với database
- Test các service gọi nhau
- Chiếm 20% tổng số test

3. E2E Test - Test "toàn bộ ngôi nhà"

**Định nghĩa:** Test toàn bộ flow từ UI đến Database, giống như user thật sử dụng

**Đặc điểm:**

- Chậm nhất (seconds đến minutes)
- Cần môi trường đầy đủ (frontend + backend + database)
- Dễ bị flaky (fail ngẫu nhiên)

Ví dụ:

Test flow đăng ký tài khoản:

1. Mở browser → Vào trang register
2. Nhập email, password → Click submit
3. Kiểm tra redirect về trang login
4. Kiểm tra email được gửi
5. Kiểm tra user tồn tại trong database

**Khi nào dùng:**

- Critical business flows (checkout, payment)
- Happy path của tính năng quan trọng
- Chiếm 10% tổng số test

4. Sai lầm thường gặp

| Sai lầm | Hậu quả | Cách tránh |
|---|---|---|
| Chỉ viết E2E, bỏ qua Unit Test | Test chậm, khó debug | Tuân theo Test Pyramid |
| Unit Test gọi database thật | Test chậm, không độc lập | Dùng Mock |
| Không viết Integration Test | Không phát hiện lỗi tích hợp | Test các API endpoints |
| Test quá nhiều chi tiết implementation | Test dễ vỡ khi refactor | Test behavior, không test implementation |

©hoidanit

## #6.3 Cấu Trúc Test Trong Spring Boot

### 1. Điểm mới trong Spring Boot 4 về Testing

Spring Boot 4 (released November 2025) có những thay đổi quan trọng:

| Thay đổi | Spring Boot 3 | Spring Boot 4 |
|---|---|---|
| JUnit version | JUnit 5 | **JUnit 6** |
| Cấu trúc dependencies | 1 jar lớn | **Modular** (nhiều jar nhỏ) |
| Test starter | spring-boot-starter-test | spring-boot-starter-test + test starters riêng cho từng module |
| Mock annotation | @MockBean | **@MockitoBean** |
| HTTP Test Client | TestRestTemplate | RestTestClient |

//todo: starter của v4

Check lại phần dependency (thiếu test), **remove spring-boot-starter-test**

Lưu ý: **spring-boot-starter-webmvc-test** đã bao gồm starter-test (chứa các thư viện hay dùng để testing)

### 2. Viết Test đầu tiên

**Cấu trúc folder trong Spring Boot:**

- folder test
- File test đặt cùng package với file cần test
- Tên file: [TênClass]Test.java

  Ví dụ: UserService.java → UserServiceTest.java

@hoidanit

Mỗi test nên tuân theo pattern Arrange - Act - Assert:

| Bước | Mô tả | Ví dụ |
|---|---|---|
| **Arrange** | Chuẩn bị dữ liệu, setup | Tạo objects, mock data |
| **Act** | Thực hiện hành động cần test | Gọi method |
| **Assert** | Kiểm tra kết quả | So sánh expected vs actual |

//todo: viết Test đầu tiên (remain folder main để chạy full test)

```
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.DisplayName;
import static org.junit.jupiter.api.Assertions.*;

class CalculatorTest {

    @Test
    @DisplayName("Cộng 2 số dương")
    void shouldAddTwoPositiveNumbers() {
        // Arrange (Chuẩn bị)
        int a = 5;
        int b = 3;

        // Act (Thực hiện)
        int result = a + b;

        // Assert (Kiểm tra)
        assertEquals(8, result);
    }

}
```

@hoidanit

## #6.4 [AI] Viết Unit Test cho Service Layer

### 1. Tại sao cần Mock trong Unit Test

Khi test UserService, nó phụ thuộc vào UserRepository:

```
  public User createUser(String name, String email) {
      // Gọi repository để lưu vào database
      return userRepository.save(new User(name, email));
  }
```

**Vấn đề:** Nếu test gọi database thật → chậm, không độc lập, khó setup

**Giải pháp:** Dùng Mock để giả lập repository

| Không dùng Mock | Dùng Mock |
|---|---|
| Cần database thật | Không cần database |
| Chậm (kết nối DB) | Nhanh (milliseconds) |
| Test phụ thuộc data trong DB | Test độc lập |
| Khó test edge cases | Dễ giả lập mọi scenario |

### 2. Khi nào Unit Test có ý nghĩa ?

❌ **Service không có logic → Unit test vô nghĩa**
→ Dùng Integration test

✅ **Service có logic** → Unit test có ý nghĩa
→ Test validation, transformation, business rules
→ Dùng argThat() để verify input đúng

Hỏi: **Test này có FAIL nếu tôi xóa logic trong Service** ?
→ Nếu KHÔNG → Test vô nghĩa
→ Nếu CÓ → Test có ý nghĩa

@hoidanit

**Ví dụ Service KHÔNG có logic (Unit test vô nghĩa):**

```
public User findById(Long id) {
   return userRepository.findById(id)
           .orElseThrow(() -> new UserNotFoundException("User không tồn tại"));
}
```

**Ví dụ Service CÓ logic (Unit test có ý nghĩa):**

```
public User createUser(User user) {
   // Logic 1: Check duplicate email
   if (this.userRepository.existsByEmail(user.getEmail())) {
      throw new DuplicateResourceException("Người dùng", "email",
user.getEmail());
   }
   // Logic 2: Encode password
   user.setPassword(this.passwordEncoder.encode(user.getPassword()));

   return this.userRepository.save(user);
}
```

@hoidanit

## Prompt viết Unit Test cho createUser()

Context:
Tôi dùng Spring Boot 4 và cần viết unit test cho method createUser trong UserService.

Yêu cầu:
1. Dùng JUnit 6 + Mockito
2. Dùng @ExtendWith(MockitoExtension.class)
3. Mock cả UserRepository và PasswordEncoder
4. Test các cases sau:
  - Happy path: tạo user thành công, password được encode
  - Email đã tồn tại: throw DuplicateResourceException
  - Verify password được encode TRƯỚC KHI save (dùng argThat)
  - Verify save() KHÔNG được gọi khi email duplicate
5. Dùng @Nested để nhóm tests
6. Dùng @DisplayName tiếng Việt
7. Dùng AAA pattern (Arrange - Act - Assert)

Lưu ý quan trọng:
- Tên function test viết bằng tiếng anh
- Dùng argThat() để verify rằng user được save có password đã encode (khác với password gốc)
- Verify thứ tự: existsByEmail() phải được gọi TRƯỚC save()

Output: Code đầy đủ cho file UserServiceTest.java

@hoidanit

### #6.5 [AI] Viết Integration Test cho API

Integration Test kiểm tra:

- Controller nhận request đúng không ?
- Service gọi Repository đúng không ?
- Data được lưu vào Database đúng không ?
- Response trả về đúng format không ?

#### 1. Công cụ Integration Test trong Spring Boot 4 (trở đi)

| Công cụ | Mô tả |
|---|---|
| `@SpringBootTest` | Khởi động toàn bộ Spring Context |
| MockMvc | Giả lập HTTP requests (không cần server thật) |
| RestTestClient | API mới trong Spring Boot 4, thay thế TestRestTemplate |
| H2 Database | In-memory database cho test |
| `@Transactional` | Rollback data sau mỗi test |

@hoidanit

## Prompt viết Integration Test cho createUser()

Context:
Tôi dùng Spring Boot 4 và cần viết integration test cho API tạo user.

Yêu cầu:
1. Dùng @SpringBootTest + @AutoConfigureMockMvc
2. Dùng @Transactional để rollback sau mỗi test
3. CHỈ test endpoint POST /users với các cases:

Happy path:
- 201 Created khi data hợp lệ
- Verify response body có đủ fields (id, name, email)
- Verify password KHÔNG được trả về trong response (security)
- Verify data được lưu vào database đúng
- Verify password trong database đã được encode (khác plain text)

Validation errors (400 Bad Request):
- Name để trống
- Email để trống
- Email không đúng định dạng
- Password ít hơn 6 ký tự
- Verify KHÔNG có data được lưu vào database khi validation fail

Business logic error (409 Conflict):
- Email đã tồn tại trong database
- Verify KHÔNG có user mới được tạo

4. Dùng @Nested để nhóm tests: HappyPath, ValidationErrors, BusinessErrors
5. Dùng @DisplayName tiếng Việt
6. Dùng JsonPath để verify response
7. Dùng userRepository để verify data trong database
8. Tên function test viết bằng tiếng anh
9. Dùng trực tiếp database đã setup, không tự cài thêm H2

Output: Code file UserControllerTest.java

## #6.6 Review & Đánh Giá Chất Lượng Test

### 1. Tại sao cần Review Test ?

AI generate test rất nhanh, nhưng:

| AI làm tốt | AI làm chưa tốt |
|---|---|
| Viết code syntax đúng | Hiểu business context |
| Cover happy path | Cover edge cases đặc thù |
| Follow pattern được yêu cầu | Biết test nào có ý nghĩa |
| Viết nhanh, nhiều | Đảm bảo test thực sự hữu ích |

**Hậu quả nếu không review:**

- Test pass nhưng không catch bug thật
- False confidence (tưởng code safe nhưng không phải)
- Test vô nghĩa làm chậm CI/CD

### 2. Checklist Review Test

**Checklist cho Unit Test**

- Test có ý nghĩa? Test có FAIL nếu xóa logic trong Service không?
- Mock đúng? Chỉ mock dependencies, không mock class đang test?
- Verify đủ? Có verify method được gọi đúng số lần?
- argThat() cho transformation? Data được transform có được verify?
- never() cho early return? Methods không được gọi khi fail early?
- Cover edge cases? null, empty, boundary values?
- Test độc lập? Mỗi test có thể chạy riêng lẻ?

@hoidanit

**Checklist cho Integration Test:**

- Test đủ HTTP status? 200, 201, 400, 404, 409...?
- Verify response body? JsonPath check đủ fields quan trọng?
- Verify database? Data được persist/delete đúng?
- Test validation? Tất cả @Valid rules được test?
- Test business errors? Duplicate, not found, unauthorized...?
- @Transactional? Data rollback sau mỗi test?
- Không leak data? Test này không ảnh hưởng test khác?

## 3. Test Coverage

Test Coverage đo lường % code được thực thi khi chạy test.

**High Coverage ≠ Good Tests**

| **Loại Coverage** | **Mô tả** |
|---|---|
| Line Coverage | % số dòng code được chạy |
| Branch Coverage | % số nhánh if/else được chạy |
| Method Coverage | % số methods được gọi |

ƯU TIÊN: Branch Coverage > Line Coverage > Method Coverage

**Coverage bao nhiêu là đủ ?**

| **Mức Coverage** | **Đánh giá** |
|---|---|
| < 50% | Quá thấp, nhiều code không được test |
| 50-70% | Chấp nhận được cho legacy code |
| **70-80%** | Tốt cho hầu hết projects |
| > 80% | Rất tốt, nhưng đừng chạy theo số lượng |

**Module 7: Cấu Trúc Dự Án Thực Hành với AI**
*Cấu hình AI để coding hiệu quả*

### #7.1 Layer-based vs Feature-based Package Structure

1.Layer-based (truyền thống):

```
com.example.app
├── controller/
│   ├── UserController
│   ├── OrderController
├── service/
│   ├── UserService
│   ├── OrderService
├── repository/
│   ├── UserRepository
│   ├── OrderRepository
└── model/
    ├── User
    ├── Order
```

Tổ chức code theo **vai trò kỹ thuật** — tất cả controllers gom vào một folder, tất cả services gom vào một folder...

**Ưu điểm:**

- Dễ hiểu cho người mới, nhìn vào biết ngay "controller ở đâu, service ở đâu"
- Dự án nhỏ (< 5-6 entity) thì rất gọn gàng

**Nhược điểm:**

- Khi dự án lớn lên, mỗi folder chứa hàng chục files → khó tìm
- Muốn hiểu feature "Order" phải nhảy qua 4-5 folders khác nhau
- Khi xóa hoặc refactor 1 feature, phải sửa ở nhiều nơi

@hoidanit

## 2. Feature-based

```
com.example.app
├── user/
│   ├── UserController
│   ├── UserService
│   ├── UserRepository
│   └── User
├── order/
│   ├── OrderController
│   ├── OrderService
│   ├── OrderRepository
│   └── Order
```

Tổ chức code theo **nghiệp vụ** — tất cả files liên quan tới một tính năng nằm cùng một folder.

**Ưu điểm:**

- Mở 1 folder = thấy toàn bộ code của feature đó
- Thêm/xóa feature rất dễ — chỉ thêm/xóa 1 folder
- Dự án lớn vẫn dễ navigate
- Mỗi feature độc lập, ít ảnh hưởng lẫn nhau

**Nhược điểm:**

- Người mới có thể bỡ ngỡ ban đầu nếu quen layer-based
- Shared code (utils, config, exceptions) cần folder riêng

@hoidanit

## 3. Dùng AI Vibe Coding — Nên Chọn Cấu Trúc Nào ?

**Câu trả lời: Feature-based.**

**Lý do 1:** AI hoạt động tốt hơn khi context tập trung
Khi bạn prompt AI: "Tạo CRUD cho feature Order", với feature-based, tất cả files liên quan nằm trong order/. AI chỉ cần "nhìn" vào 1 folder để hiểu toàn bộ context.

Với layer-based, AI phải "nhảy" qua controller/, service/, repository/, model/ — context bị phân tán. AI dễ generate code không nhất quán giữa các layer, hoặc quên import đúng package.

**Lý do 2:** Verify dễ hơn
Sau khi AI generate code, bạn mở 1 folder, đọc từ trên xuống, kiểm tra xong — thay vì phải nhảy qua nhiều folder rồi quên mất mình đang verify cái gì.

**Lý do 3**: Giảm xung đột khi AI generate nhiều feature
Khi bạn dùng AI Agent (như Claude Code, Cursor...) để generate nhiều feature liên tiếp, feature-based giảm thiểu khả năng AI vô tình sửa nhầm code của feature khác. Mỗi feature là một "vùng" riêng biệt.

**Lý do 4:** Dễ chia task cho AI
Với feature-based, bạn có thể chia dự án thành các task rõ ràng cho AI:
Bước 1: Tạo tất cả models/entities
Bước 2: Feature auth — login/register + test
Bước 3: Feature user — CRUD + phân trang + test
Bước 4: Feature company — CRUD + test
Bước 5: Feature role/permission + test

Mỗi bước = 1 module = 1 folder. AI focus vào đúng scope, output chất lượng hơn.

@hoidanit

## #7.2 Sử Dụng AI Agent Coding Hiệu Quả

**AI Agent Coding** (Claude Code, Cursor Agent, GitHub Copilot Agent...) khác biệt so với chat-based AI ở chỗ: nó **đọc codebase, tạo file, chạy command, và sửa code trực tiếp** — không chỉ trả lời qua chat.

**3 vấn đề thường gặp**

Vấn đề 1: AI quên giữa các session

Sáng bạn nhờ AI tạo feature User với convention: dùng DTO, response wrapper, pagination chuẩn. AI làm hoàn hảo.

Chiều bạn mở session mới, nhờ AI tạo feature Company. AI... làm khác hoàn toàn. Không dùng DTO, response format khác, pagination khác.

Lý do: Session mới = AI không nhớ gì về session cũ.

Vấn đề 2: Context bị "loãng" trong session dài

Bạn chat với AI 50-60 lượt trong 1 session. Đến lượt thứ 55, AI bắt đầu generate code mâu thuẫn với những gì nó làm ở lượt thứ 5. Vì phần đầu conversation đã bị "đẩy ra" khỏi context window.

Vấn đề 3: AI không hiểu "big picture"

AI đọc từng file riêng lẻ nhưng không tự hiểu: "À, dự án này dùng JWT cho auth, có role-based access control, và response luôn wrap trong RestResponse." Bạn phải nói cho AI biết.

**Giải Pháp: Project Rules — "Bộ Nhớ Dài Hạn" Cho AI**

Ý tưởng cốt lõi: Viết ra file những gì AI cần biết, để mỗi session mới, AI đọc lại và hiểu ngay context.

Hầu hết AI Agent đều hỗ trợ một file đặc biệt gọi là Project Rules (hoặc Instructions). AI sẽ tự động đọc file này đầu tiên trước khi làm bất cứ gì. Đây chính là "bộ nhớ dài hạn" mà bạn cần tận dụng.

@hoidanit

Tên file theo từng tool

| Tool | File/Cách cấu hình | Cách AI đọc |
|---|---|---|
| Claude Code | CLAUDE.md ở root project | Tự động đọc khi bắt đầu session |
| Cursor | .cursorrules ở root project, hoặc Settings → Rules | Tự động đọc, áp dụng cho mọi prompt |
| GitHub Copilot | .github/copilot-instructions.md | Tự động đọc (VS Code / JetBrains) |
| Windsurf | .windsurfrules ở root project | Tự động đọc khi mở project |
| Antigravity | Phần "Instructions" trong Project Settings | Tự động đọc, áp dụng cho mọi prompt |
| Gemini (in IDE) | .gemini/styleguide.md hoặc Settings → Gemini | Tự động đọc trong IDE context |
| ChatGPT<br>Claude Chat | Không có file — phải paste vào đầu conversation | Chỉ biết khi bạn paste |

**Lưu ý quan trọng:** Tên file có thể thay đổi theo version của tool. Hãy check documentation mới nhất của tool bạn đang dùng.
Tuy nhiên, ý tưởng cốt lõi không đổi: **1 file chứa conventions → AI đọc đầu tiên.**

**Nội dung cần có trong Project Rules**

| Mục | Mô tả | Tại sao cần |
|---|---|---|
| Tech Stack | Tên công nghệ sử dụng | AI biết dùng đúng version, syntax |
| Project Structure | Cấu trúc folder feature-based | AI biết tạo file ở đâu |
| Coding Conventions | DTO pattern, response wrapper, naming... | AI generate code nhất quán |
| Important Rules | Không dùng EAGER, luôn dùng DTO... | Tránh AI phạm lỗi phổ biến |
| Decision Log | Các quyết định đã đưa ra theo thời gian | AI biết dự án đang ở đâu |

@hoidanit

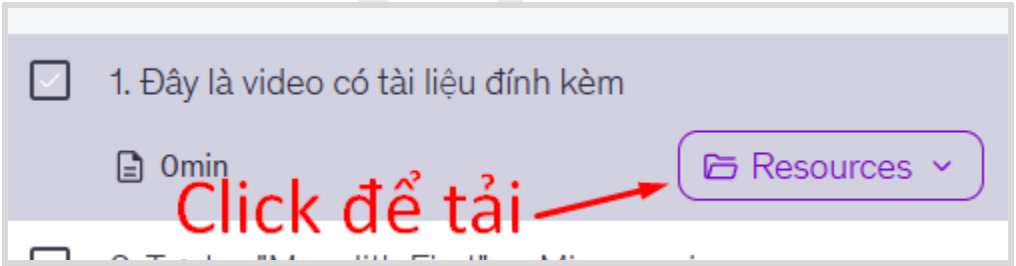
### #7.3 Flow Coding với AI (Overview)

Khi làm dự án một mình với AI Agent, thách thức lớn nhất không phải viết code — mà là **giữ cho dự án có trật tự qua nhiều ngày, nhiều session**.

Không có teammate review, không có Tech Lead định hướng. Bạn vừa là kiến trúc sư, vừa là thợ xây, vừa là người kiểm tra chất lượng.

| Phase | Mô tả | Quy tắc |
|---|---|---|
| 1. Chuẩn bị | Project Rules + Database design + API spec | Làm 1 lần, làm kỹ |
| 2. Nền móng | Tạo entities + seed data + Công cụ test API | Tạo HẾT entities cùng lúc |
| 3. Từng feature | CRUD + test + commit cho mỗi feature | 1 feature = 1 session |
| 4. Hoàn thiện | CORS, phân quyền, E2E test | Chạy full regression |

Trường hợp công ty (doanh nghiệp) chặn domain của Google, bạn download trực tiếp tại phần **tài nguyên** (resources) đính kèm (trên Udemy - nếu có)




### #7.4 Flow Coding với AI (Backend)

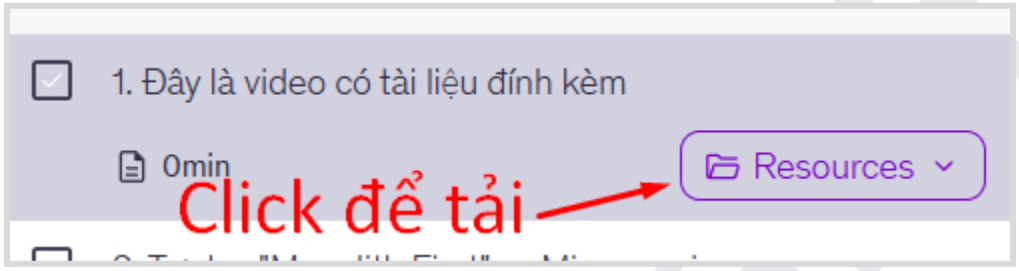
//todo

### #7.5 Setup Dự Án Thực Hành với AI

Mục tiêu: Project Rules + Database design + API spec

Download dự án thực hành tại đây

Trường hợp công ty (doanh nghiệp) chặn domain của Google, bạn download trực tiếp tại phần **tài nguyên** (resources) đính kèm (trên Udemy - nếu có)




### #7.6 Quy Trình Vibe Coding Với AI

//todo

@hoidanit

**Module 8: [Vibe Coding] Permission & Company CRUD**
*Thực hành CRUD cho Permission và Company models*

**#8.1 Tổng quan về module 8**

1.Chúng ta đang ở đâu ?
Nhắc lại project đang ở đâu (Phase 0 đã xong: exception handler, ApiResponse, SecurityConfig permitAll, JwtConfig chuẩn bị sẵn).

**Giải thích dependency graph:**

```
permission  ← (không phụ thuộc gì)

company     ← (không phụ thuộc gì)
   ↓
role        ← phụ thuộc permission (ManyToMany)
   ↓
user        ← phụ thuộc role + company
   ↓
auth        ← phụ thuộc user
```

→ Vì vậy **Phase 1 làm Permission + Company trước** (entity độc lập).

2. Phân tích

- Show database schema của permissions và companies (mở DATABASE.md)

- Show API spec — các endpoint cần implement (mở API_SPEC.md)

**Giải thích flow sẽ làm:**
Bước 1: Viết prompt cho AI tạo Permission CRUD

Bước 2: Review code AI sinh ra

Bước 3: Test bằng Swagger

Bước 4: Lặp lại cho Company

Bước 5: Viết test (unit + integration)

@hoidanit

## #8.2 [AI] Permission CRUD

Bước 1: Giải thích cấu hình Claude (Project Rules)
AI đọc các file này giống như developer mới vào dự án đọc onboarding docs.
Càng viết kỹ, AI càng code đúng.

Mở và show các file docs, giải thích:

- CLAUDE.md — entry point AI đọc đầu tiên
- PROJECT-RULES.md — convention AI phải tuân theo
- DATABASE.md — schema
- API_SPEC.md — endpoint spec

Bước 2: Viết prompt tạo Permission CRUD

Lưu ý: chạy tại chế độ "plan mode". Lên kế hoạch trước khi code.
Prompt mẫu (cho Claude Code):
**/new-feature — tạo Permission CRUD**

Yêu cầu:
- Đọc DATABASE.md để biết schema bảng permissions
- Đọc API_SPEC.md phần Permissions để biết request/response format
- Tạo đầy đủ: Entity, Repository, DTOs (request + response records),
  Service interface, ServiceImpl, Controller
- Permission có UNIQUE constraint trên (apiPath + method) → xử lý duplicate → 409
- Tất cả response wrap trong ApiResponse
- Validate input với @Valid + Jakarta Bean Validation

Bước 3: Review code AI sinh ra
Đây là lúc bạn đóng vai Tech Lead — AI viết code, bạn review. Không merge code mà bạn không hiểu (chạy unit test, integration test nếu cần thiết)
//todo

Bước 4: Nếu AI sai, sửa lại bằng prompt
//todo

### #8.3 [AI] Company CRUD

**Bước 1:** Viết prompt
Prompt mẫu:

---

/new-feature — tạo Company CRUD

Yêu cầu:
- Đọc DATABASE.md schema bảng companies
- Đọc API_SPEC.md phần Companies
- Tạo đầy đủ: Entity, Repository, DTOs, Service (Interface + Impl), Controller
- Company đơn giản hơn Permission — không có unique constraint đặc biệt
- Logo field là String (path tới file) — Phase 3 mới làm upload

**Bước 2:** Review nhanh (vì pattern giống Permission, chỉ cần check khác biệt)

Điểm nhấn: "Lần thứ 2 bạn sẽ thấy AI sinh code nhanh hơn nhiều vì nó đã có pattern từ Permission. Đây là sức mạnh của Project Rules — AI học convention từ code có sẵn."

@hoidanit

### #8.4 Setup Swagger & Test API

**Swagger** vừa là công cụ test, vừa là documentation tự động cho API. Client team (frontend) có thể đọc Swagger để biết gọi API như thế nào

Cấu hình Swagger
https://springdoc.org

Cài đặt:

```
  <dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>3.0.1</version>
  </dependency>
```

Truy cập UI:
Swagger UI: http://localhost:8080/swagger-ui/index.html

OpenAPI JSON: http://localhost:8080/v3/api-docs

```
//config security
 private static final String[] WHITELIST = {
    "/v3/api-docs/**",
    "/swagger-ui/**",
    "/swagger-ui.html"
 };
```

Cần test api login, refresh token và resources endpoint

Download file cấu hình Swagger tai đây

Trường hợp công ty (doanh nghiệp) chặn domain của Google, bạn download trực tiếp tại phần **tài nguyên** (resources) đính kèm (trên Udemy - nếu có)

@hoidanit

### #8.5 Review Permission & Company CRUD

Checklist:

- Giải thích source code
- Test với Swagger và Postman (optional)
- Đã viết code có test chưa, nếu chưa có, tạo test

  Tạo test xong, chạy tất cả test case
- Cập nhật project status
- Kiểm tra lại file vibe coding để nắm vững quy trình các bước cần làm

@hoidanit

**Module 9: [Vibe Coding] Role CRUD & ManyToMany Permission**
*Xử lý đối tượng Roles và mối quan hệ ManyToMany với Permission*

### #9.1 Tổng quan về module 9

Giải thích relationship:

```
Role ——— ManyToMany ———▶ Permission
        (join table: permission_role)
```

Show database schema — bảng roles + bảng permission_role (join table)

Show API spec:
POST /roles nhận permissionIds: [1, 2, 3] → tạo role + gắn permissions
PUT /roles nhận permissionIds → replace toàn bộ (không additive)
DELETE /roles → xóa role + clear join table

Lưu ý: Permission đã tồn tại từ Phase 1 → khi tạo Role, chỉ cần reference permissionIds

@hoidanit

### #9.2 [AI] Role CRUD

/new-feature — tạo Role CRUD

Yêu cầu:

- Đọc DATABASE.md schema bảng roles + permission_role
- Đọc API_SPEC.md phần Roles
- Entity Role có ManyToMany với Permission (join table permission_role)
- CreateRoleRequest nhận permissionIds (List<Long>)
- ServiceImpl: khi create/update, load permissions từ DB bằng permissionIds

→ nếu permissionId nào không tồn tại → throw ResourceNotFoundException

- UpdateRoleRequest nhận permissionIds → replace toàn bộ (set lại list, không additive)

- DELETE role → JPA tự clear join table (cascade)
- RoleResponse trả về danh sách permissions (id + name + apiPath + method + module)

@hoidanit

### #9.3 Review Role CRUD

Tạo vài permissions trước (từ Phase 1):

POST /roles với permissionIds: [1, 2] → expect 201 + response có permissions
POST /roles với permissionIds: [999] → expect 404 (permission not found)
PUT /roles — thay đổi permissionIds → expect permissions được replace
DELETE /roles → expect role bị xóa, permission vẫn còn

@hoidanit

**Module 10: [Vibe Coding] User CRUD & File Upload**
*Xử lý CRUD đối tượng users và upload file cập nhật thông tin người dùng*

## #10.1 Tổng quan về User CRUD

Overview: User Dependencies

Dependency map:

User ——— ManyToOne ———► Company
User ——— ManyToMany ———► Role (join table: user_role)

Điểm khác biệt so với Phase 1-2:

User có password → phải hash bằng BCrypt khi create/update
User có companyId (single FK) + roleIds (list FK)
User có avatar — sẽ gắn với file upload
Response không bao giờ trả password

Show API spec — nhấn mạnh:

- CreateUserRequest: companyId (Long) + roleIds (List<Long>) + password (plain text)
- UserResponse: không có password, có nested company info + role info

@hoidanit

### #10.2 [AI] User CRUD

/new-feature — tạo User CRUD

Yêu cầu:
- Đọc DATABASE.md schema bảng users + user_role
- Đọc API_SPEC.md phần Users
- Entity User:
  + ManyToOne Company (company_id FK, nullable)
  + ManyToMany Role (join table user_role)
  + email UNIQUE → check duplicate khi create/update
  + gender dùng Enum (MALE, FEMALE, OTHER) với @Enumerated(STRING)
- CreateUserRequest nhận: name, email, password, age, address, gender, companyId, roleIds
- ServiceImpl:
  + Hash password bằng BCrypt (PasswordEncoder bean) khi create
  + Khi update: nếu password field không null/blank → hash lại; nếu null → giữ nguyên
  + Validate companyId tồn tại (nếu provided)
  + Validate roleIds tồn tại
  + Check email unique (exclude self khi update)
- UserResponse: KHÔNG có password. Có nested company (id + name) + roles (id + name)
- DELETE user: clear user_role join table

@hoidanit

### #10.3 Review User CRUD

Yêu cầu:
- User unit test:
  + Create user → password được hash
  + Create user email trùng → exception
  + Create user với invalid companyId → exception
  + Update user không đổi password → password giữ nguyên
- User integration test: full CRUD qua MockMvc

@hoidanit

**#10.4 Tổng quan về File Upload (Avatar & Logo)**

Giải thích ADR-002 (File Upload Strategy)
Mở docs/decisions/002-file-upload-strategy.md, giải thích:

Tại sao chọn Local Storage (đơn giản, miễn phí, dễ migrate sang S3 sau)

Pattern 2 bước: Upload file → nhận fileName → PUT entity với fileName
Tại sao tách upload khỏi entity update (separation of concerns)

@hoidanit

### #10.5 [AI] File Upload (Avatar & Logo)

Viết prompt
/new-feature — tạo File Upload

Yêu cầu:
- Đọc ADR-002 (docs/decisions/002-file-upload-strategy.md) để biết design
- Tạo: FileController, FileService (interface), FileServiceImpl, FileUploadResponse DTO
- POST /api/v1/files — nhận multipart/form-data (file + folder)
- Validation:
  + folder phải thuộc whitelist: [avatars, logos]
  + extension phải thuộc whitelist: [jpg, jpeg, png, gif, webp]
  + size ≤ 5MB
  + file name sanitize (loại bỏ ký tự đặc biệt, thay bằng _)
  + Path traversal prevention (normalize + check prefix)
- File lưu tại: {base-dir}/{folder}/{timestamp}_{sanitizedName}
- Config trong application.yml (app.upload.base-dir, max-size, allowed-extensions, allowed-folders)
- Spring static resource serving: /uploads/** → file:{base-dir}/
- SecurityConfig: permitAll cho /uploads/**

@hoidanit

### #10.6 Review File Upload

Download files test tại đây

**Demo flow trên Swagger:**
Upload avatar: POST /files → file=photo.jpg, folder=avatars → nhận fileName
Update user: PUT /users → { id: 1, avatar: "1709123456789_photo.jpg" }
Truy cập: GET /uploads/avatars/1709123456789_photo.jpg → thấy ảnh
Upload logo: tương tự cho company

**Test lỗi:**
Upload file .exe → expect 400 (extension not allowed)
Upload file > 5MB → expect 400
Upload folder = "../../etc" → expect 400 (path traversal blocked)

@hoidanit

## Module 11: [Vibe Coding] Authentication & Refresh Token

*Xử lý tính năng đăng ký, đăng nhập và refresh token*

### #11.1 Tổng quan về module 11

Demo kết quả đạt được, giải thích luồng logic hoạt động của login/register kết hợp với refresh token

//giải thích cơ chế hoạt động của refresh token

@hoidanit

## #11.2 Tạo Seed Data

Khi bắt đầu một dự án Spring Boot, database hoàn toàn trống. Mỗi lần chạy app lên, bạn phải tự tay tạo dữ liệu để test — tạo user, tạo role, gán quyền... Lặp đi lặp lại rất mất thời gian.

Seed data giải quyết vấn đề này: app chạy lên, tự kiểm tra database, nếu chưa có dữ liệu thì tự tạo sẵn. Developer mở app lên là có ngay dữ liệu để làm việc.
Đặc biệt quan trọng khi:

- Onboard thành viên mới — clone repo, chạy app, có data ngay
- Demo cho khách hàng — không cần chuẩn bị data thủ công
- Chạy test — cần data nhất quán giữa các lần chạy
- Làm việc với AI — AI cần data mẫu để test endpoint

### Cách 1: File data.sql — Đơn Giản Nhất

Spring Boot tự động tìm file **src/main/resources/data.sql** và chạy khi khởi động.

```
-- data.sql
INSERT INTO companies (name, description, address)
VALUES ('Hỏi Dân IT', 'Education platform', 'Hanoi');
```

**Cấu hình:** spring.sql.init.mode = always
Nhược điểm:

- Chạy mỗi lần app khởi động → duplicate data nếu không có check
- Password phải viết sẵn dạng BCrypt hash (không thể gọi passwordEncoder.encode())
- Không có logic — không thể check "đã có data chưa" rồi mới insert
- Phụ thuộc SQL syntax → đổi database (MySQL sang PostgreSQL) có thể phải sửa
- Thứ tự insert phải đúng (company trước user) — dễ sai khi project lớn

@hoidanit

## Cách 2: CommandLineRunner / ApplicationRunner — Linh Hoạt Nhất

Viết Java code, chạy sau khi Spring Boot khởi động xong.

Download file seed data trong video tại đây

Nhược điểm:

- Cần viết Java code (nhiều hơn so với file SQL)
- Code seed có thể dài nếu data nhiều

## Cách 3: Flyway / Liquibase — Cho Production

Đây là migration tool, không phải seed tool. Nhưng có thể dùng để seed data bắt buộc phải có ở production (admin account mặc định, permission list...).

**Ưu điểm:**

- Versioned — biết chính xác migration nào đã chạy, chưa chạy
- Chạy đúng 1 lần — không bao giờ duplicate (tool tự track trong bảng riêng)
- Production-safe — dùng cho data bắt buộc phải có
- Rollback được (Liquibase)

**Nhược điểm:**

- Setup phức tạp hơn
- Phải viết SQL hoặc XML — không dùng được Java code
- Overkill cho fake data dev/test

@hoidanit

### #11.3 [AI] Refresh Token & Logout

Implement Refresh Token theo ADR-001:

**Bước 1:** RefreshToken Entity + Repository
- Đọc DATABASE.md schema bảng refresh_tokens
- Entity: id, token (unique), userId (FK), expiresAt, revoked, deviceInfo, ipAddress, createdAt
- Repository: findByToken(), findByUserIdAndRevokedFalse(), deleteByExpiresAtBefore()

**Bước 2:** Cập nhật Login response
- Login giờ trả CẢ accessToken + refreshToken
- Set cookie: refresh_token=<jwt>; HttpOnly; Secure; SameSite=Lax; Path=/api/v1/auth; Max-Age=259200
- Lưu RefreshToken record vào DB

**Bước 3:** POST /api/v1/auth/refresh
- Check cookie refresh_token trước → nếu có thì dùng
- Nếu không có cookie → check body refreshToken
- Nếu không có cả 2 → 401
- Validate: token tồn tại trong DB? chưa revoked? chưa expired?
- Nếu valid → revoke token cũ → tạo token mới (rotation)
- Trả: accessToken mới + refreshToken mới + set cookie mới

**Bước 4:** POST /api/v1/auth/logout
- Cần auth (Bearer token)
- Revoke refresh token trong DB (set revoked = true)
- Clear cookie: Set-Cookie: refresh_token=; Max-Age=0; Path=/api/v1/auth
- Response: 200 "Logout successful"

@hoidanit

### #11.4 Review Authentication & Refresh Token

Login → nhận accessToken + refreshToken + cookie được set

Đợi/giả lập access token hết hạn
POST /auth/refresh (gửi cookie hoặc body) → nhận token mới
POST /auth/refresh lại với token cũ → 401 (đã bị revoke — rotation)
POST /auth/logout → cookie bị clear
POST /auth/refresh → 401 (token revoked)

@hoidanit

**Module 12: [Vibe Coding] Authorization**
*Xử lý quyền truy cập của người dùng với role và permission*

### #12.1 Tổng quan về module 12

Có bao nhiêu cách phân quyền ?

//todo
Dựa vào role, dựa vào permission

Flow kiểm tra quyền với permission:
Request: GET /api/v1/users
→ JWT valid? (không → 401)
→ Lấy roles từ JWT → lấy danh sách permissions của role
→ Tìm Permission nào match: apiPath="/api/v1/users" + method="GET"
→ User có permission đó không?
→ Có → 200 / Không → 403

@hoidanit

### #12.2 Phân Biệt Role và Authority

Trong Spring Security, **Role** và **Authority** (hay còn gọi là GrantedAuthority) có mối quan hệ rất gần nhau nhưng khác nhau ở cách sử dụng.

Về mặt kỹ thuật, Spring Security chỉ có một interface duy nhất là **GrantedAuthority**.

- Role thực chất cũng chỉ là một Authority được thêm prefix **ROLE_** phía trước.
- Nói cách khác, Role là một trường hợp đặc biệt của Authority.

```
// Khi bạn gán role "ADMIN" cho user
// Spring Security lưu nó dưới dạng: GrantedAuthority("ROLE_ADMIN")

// Khi bạn gán authority "CREATE_USER"
// Spring Security lưu nó dưới dạng: GrantedAuthority("CREATE_USER")
```

Điểm khác biệt chính

**Role** — đại diện cho vai trò chung của user trong hệ thống. **Mang tính nhóm, khái quát.** Spring tự động thêm prefix **ROLE_:**

```
// Gán role khi tạo UserDetails
User.builder()
   .username(user.getEmail())
   .password(user.getPassword())
   .roles("ADMIN")          // → Spring lưu thành "ROLE_ADMIN"
   .build();

// Check bằng hasRole → Spring tự thêm "ROLE_" khi so sánh
@PreAuthorize("hasRole('ADMIN')")         // check "ROLE_ADMIN"
.requestMatchers("/admin/**").hasRole("ADMIN")  // check "ROLE_ADMIN"
```

**Authority** — đại diện cho quyền cụ thể, chi tiết hơn. Không có prefix, bạn tự đặt tên:

```
// Gán authority khi tạo UserDetails
User.builder()
   .username(user.getEmail())
   .password(user.getPassword())
   .authorities("CREATE_USER", "DELETE_POST", "VIEW_REPORT")
   .build();

// Check bằng hasAuthority → so sánh chính xác string
@PreAuthorize("hasAuthority('CREATE_USER')")
.requestMatchers("/users").hasAuthority("CREATE_USER")
```

//todo: giải thích về FACTOR_PASSWORD (MFA)

**Bước 1**: User nhập email + password
→ DaoAuthenticationProvider verify ✓
→ Gắn authority: FACTOR_PASSWORD
→ Lúc này user "partially authenticated"

**Bước 2:** User nhập OTP / quét vân tay / passkey
→ OTP Provider verify ✓
→ Thêm authority: FACTOR_OTP (hoặc FACTOR_PASSKEY)
→ Lúc này user "fully authenticated"

Bổ trợ:

**Authority** (Thẩm quyền / Quyền hạn): Thiên về "Tư cách". Nó trả lời cho câu hỏi: "Bạn là ai trong hệ thống này?" hoặc "Bạn có tư cách gì?".

Ví dụ: Bạn có thẩm quyền của một ADMIN hoặc một MANAGER.

**Permission** (Quyền truy cập / Phép thực hiện): Thiên về "Hành động". Nó trả lời cho câu hỏi: "Bạn được phép làm hành động cụ thể nào?".

Ví dụ: Bạn có quyền CREATE_USER, DELETE_COMPANY.

@hoidanit

## #12.3 [AI] Role-Based Access Control (RBAC)

### 1. Authorization Scope

Cách hoạt động

**Bước 1:** Client truy cập endpoint, gửi kèm access token (dưới dạng endpoint)

**Bước 2:** Spring dùng filter, tách token từ header request, **decode** (giải mã token)

**Bước 3:** Sau khi decode, Spring nạp thông tin vào Security Context
Có thể can thiệp quá trình này với **JwtAuthenticationConverter**

**Bước 4:** Các logic check role/authority được thực hiện dựa vào kết quả của bước 3 ở trên

### Giải thích cơ chế của JwtAuthenticationConverter

Bên trong JwtAuthenticationConverter có một delegate là JwtGrantedAuthoritiesConverter. Cái này mặc định:
**Bước 1:** Tìm claim "**scope**" trong JWT payload. Nếu không có thì tìm "scp".
**Bước 2:** Split giá trị bằng dấu cách (space) hoặc nếu là array thì lấy từng phần tử.
**Bước 3:** Thêm prefix "SCOPE_" vào mỗi giá trị → tạo thành GrantedAuthority.

```
// JWT payload
{
  "sub": "test@test.com",
  "userId": 1,
  "scope": "ADMIN USER",
}
```

JwtGrantedAuthoritiesConverter xử lý:
"ADMIN USER" → split → ["ADMIN", "USER"]
→ thêm prefix → [SCOPE_ADMIN, SCOPE_USER]

SecurityContext authorities = [SCOPE_ADMIN, SCOPE_USER]

hoidanit

```
// Bean
private JwtAuthenticationConverter jwtAuthenticationConverter() {
   JwtGrantedAuthoritiesConverter grantedAuthoritiesConverter = new
JwtGrantedAuthoritiesConverter();
   grantedAuthoritiesConverter.setAuthoritiesClaimName("roles");   // đọc từ claim "roles"
   grantedAuthoritiesConverter.setAuthorityPrefix("ROLE_");       // prefix ROLE_ thay vì SCOPE_

   JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
   converter.setJwtGrantedAuthoritiesConverter(grantedAuthoritiesConverter);
   return converter;
}
```

## 2.Cách dùng

```
// Trên SecurityFilterChain
.authorizeHttpRequests(auth -> auth
   .requestMatchers("/api/admin/**").hasRole("ADMIN")
   .requestMatchers("/api/user/**").hasAnyRole("USER", "ADMIN")
   .anyRequest().authenticated())

// Hoặc trên method với @PreAuthorize
@PreAuthorize("hasRole('ADMIN')")
@DeleteMapping("/users/{id}")
public void deleteUser(@PathVariable Long id) { ... }
```

//todo
Xử lý lỗi 403 forbidden

@hoidanit

### #12.4 [AI] Permission-Based Access Control

Download files sử dụng tại đây

1.Giải thích cơ chế hoạt động

Tham khảo docs\decisions**003-authorization-permission-strategy.md**

2. Viết custom filter
Sử dụng Custom **AuthorizationManager** trong SecurityFilterChain

### #12.5 Review RBAC

//nhận xét về ưu/nhược điểm, và giới thiệu các cách làm nâng cao

@hoidanit

**Module 13: Hoàn Thiện Dự Án Thực Hành**
*Review toàn bộ dự án đã làm và hướng phát triển tiếp theo*

### #13.1 [AI] Pagination & Sorting

Thêm pagination + sorting cho tất cả list endpoints:

Yêu cầu:
- Tất cả GET list endpoints nhận query params: page, size, sort
  VD: GET /api/v1/users?page=1&size=10&sort=name,asc
- Dùng Spring Data Pageable
- Response format (đã define trong API_SPEC):
```
 {
   "statusCode": 200,
   "data": {
     "meta": { "page": 1, "pageSize": 10, "pages": 5, "total": 50 },
     "result": [...]
   }
 }
```
- Áp dụng cho: /users, /companies, /roles, /permissions
- Default: page=1, size=10, sort=createdAt,desc
- Lưu ý: Spring Pageable page bắt đầu từ 0, nhưng API trả về page bắt đầu từ 1
  → cần convert (page request - 1 khi gọi repository, page response + 1 khi trả client)

@hoidanit

### #13.2 [AI] Fix Tests & Permissions

//todo: import collection vào postman

//làm phần sort

//fix phần unit test (vì liên quan về lỗi permission)

Prompt:
Context:
Tôi vừa tích hợp RBAC permission check theo URL + method vào SecurityFilterChain.
Flow hoạt động như sau:
Request → JWT valid → lấy roles từ JWT → query DB: role có permission (apiPath + method) match không?
→ Không có → 403 Forbidden

Vấn đề: Toàn bộ integration test hiện tại bị 403 vì test data không seed Permission và không gán vào Role của test user.

Yêu cầu:

Tạo một class **TestDataFactory** dùng chung cho tất cả integration tests,
với nhiệm vụ:
1. Seed Permission với đúng apiPath + method khớp endpoint đang test
2. Tạo Role và gán permissions đó vào
3. Tạo test user với role đó
4. Generate JWT từ test user → trả về token string để dùng trong test

Sau đó, cập nhật lại file integration test hiện tại (UserControllerTest.java)
để dùng TestDataFactory trong @BeforeEach.
Đảm bảo có 2 loại test case:
- 200: user có đủ permission
- 403: user KHÔNG có permission (tạo user với role rỗng)

### #13.3 [AI] Search & Filter

Tài liệu của Spring về Specification, tham khảo tại đây

Download file tài liệu tại đây

Thêm search cho User list:
- GET /api/v1/users?search=john → tìm theo name hoặc email (LIKE %keyword%)
- Dùng JPA Specification hoặc @Query
- Kết hợp với pagination

@hoidanit

### #13.4 [AI] Scheduled Job - Cleanup Expired Tokens

Tạo scheduled job cleanup expired refresh tokens:

- Chạy mỗi ngày lúc 2:00 AM
- Delete tất cả refresh_tokens có expires_at < NOW()
- Log số lượng tokens đã xóa
- Config: @EnableScheduling trên Application class
- Tạo class ScheduledTasks trong package config/

@hoidanit

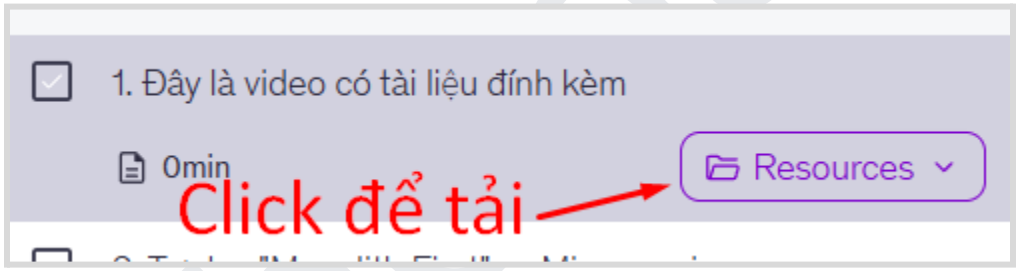
**#13.5 Review Backend với Giao Diện Frontend (Fix CORS)**

1.Fix các lỗi của backend
Download các files thay đổi tại đây
//todo
Fix cors
e2e test với giao diện FE được cung cấp sẵn

2. Setup dự án frontend
Download dự án frontend tại đây

Môi trường chạy dự án frontend sử dụng **Node.js v24.14.0** download tại đây

Trường hợp công ty (doanh nghiệp) chặn domain của Google, bạn download trực tiếp tại phần **tài nguyên** (resources) đính kèm (trên Udemy - nếu có)




@hoidanit

### #13.6 [AI] Review Toàn Bộ Project

/review-pr — review toàn bộ project

Check:
1. Tất cả files < 300 lines?
2. Tất cả methods < 50 lines?
3. Không có @Autowired field injection?
4. Tất cả @RequestBody có @Valid?
5. Không có Entity trả về từ Controller?
6. Tất cả response wrap trong ApiResponse?
7. Không có sensitive data trong JWT claims hoặc logs?
8. Test coverage: mỗi module có unit test + integration test?
9. Documentation: API_SPEC.md, DATABASE.md up to date?
10. CONTEXT.md cho mỗi feature module?

@hoidanit

## #13.7 What's next - Học gì tiếp theo ?

### I. Nhận Xét Về Ưu / Nhược Điểm

**Ưu điểm:**

1.Tiếp cận thực tế & hiện đại

Khóa học không dạy theo lối truyền thống "học syntax rồi mới làm".
Thay vào đó, học viên được tiếp xúc ngay với Vibe Coding — cách làm việc thực tế của developer trong kỷ nguyên AI.
Đây là điểm khác biệt lớn so với các khóa học khác trên thị trường.

2. Cân bằng giữa AI và Fundamentals

Khóa học giải quyết đúng nghịch lý lớn nhất: dùng AI để code nhanh, nhưng vẫn hiểu rõ code mình tạo ra. Học viên không chỉ biết "copy-paste từ AI" mà còn biết verify, debug và quyết định khi AI sai.

3. Dự án thực hành đầy đủ

Toàn bộ quy trình từ thiết kế database → CRUD → Authentication → Authorization → Testing → File Upload đều được thực hành trên một dự án thống nhất, thay vì các ví dụ nhỏ rời rạc.

4. Workflow chuyên nghiệp

Học viên được học cách làm việc như một team thực sự: Project Rules, API Spec, Database Design, commit theo feature — những thứ quan trọng nhưng thường bị bỏ qua trong các khóa học khác.

5. Mindset đúng ngay từ đầu

Module 0 dành riêng để định hình tư duy trước khi code — một đầu tư thường bị xem nhẹ nhưng lại quyết định phần lớn sự thành công của học viên.

@hoidanit

**Nhược Điểm & Giới Hạn**

1. Yêu cầu nền tảng ban đầu

Khóa học phù hợp nhất với người đã có kiến thức lập trình cơ bản (Java hoặc ngôn ngữ khác). Người hoàn toàn mới bắt đầu có thể gặp khó khăn ở các module đầu tiên.

2. Tốc độ thay đổi của AI Tools

Các công cụ AI (Cursor, GitHub Copilot, Claude…) cập nhật liên tục. Một số thao tác trong video có thể khác với phiên bản mới nhất của tool — học viên cần linh hoạt thích nghi.

3. Chưa bao gồm deployment

Khóa học tập trung vào backend development, chưa đề cập đến việc deploy ứng dụng lên môi trường production (Docker, cloud, CI/CD).

4. Frontend chưa được xây dựng cùng

Backend được review với giao diện frontend có sẵn, nhưng học viên chưa tự tay xây dựng phần frontend từ đầu trong khóa học này.

## II. What's Next — Học Gì Tiếp Theo ?

Sau khi hoàn thành khóa học này, bạn đã nắm vững nền tảng để xây dựng một REST API hoàn chỉnh với Spring Boot. Dưới đây là lộ trình gợi ý để tiếp tục phát triển:

**Hướng 1: Hoàn thiện Backend — Đi sâu hơn**

| Chủ đề | Mô tả |
|---|---|
| Spring WebFlux | Reactive programming, xử lý non-blocking |
| Caching với Redis | Tăng performance cho API |
| Message Queue (Kafka/RabbitMQ) | Xây dựng hệ thống bất đồng bộ |
| Microservices | Tách nhỏ monolith thành các service độc lập |

**Hướng 2: Fullstack — Xây dựng sản phẩm hoàn chỉnh**
Kết hợp backend Spring Boot đã học với một frontend framework:

- ReactJS / NextJS — lựa chọn phổ biến nhất hiện tại
- Angular — phổ biến trong các dự án doanh nghiệp lớn

**Hướng 3: Nâng cao kỹ năng AI-assisted Development**
Kỷ nguyên AI đang thay đổi nhanh. Hãy liên tục cập nhật:

- Thực hành với các AI agent mới (Claude Code, Cursor, Copilot...)
- Học cách viết Project Rules & Prompt hiệu quả hơn
- Tham gia cộng đồng để chia sẻ workflow thực tế

**Hướng 4: DevOps & Deployment — Đưa sản phẩm lên production**

| Chủ đề | Mô tả |
|---|---|
| Docker & Docker Compose | Container hóa ứng dụng |
| CI/CD với GitHub Actions | Tự động hóa build & deploy |
| Deploy lên Cloud (AWS/GCP/Azure) | Chạy ứng dụng thực trên internet |
| Monitoring (Prometheus + Grafana) | Theo dõi sức khỏe ứng dụng |

Tham khảo khóa học Deploy với Docker tại đây

Tham khảo khóa học Deploy với Docker (và Github Action) tại đây

@hoidanit

**Module 14: Fix Các Lỗi Thường Gặp**
*Hướng dẫn fix các lường thường gặp trong quá trình chạy dự án*

### #14.1 Lỗi Trùng Port (EADDRINUSE: address already in use)

Lỗi **EADDRINUSE** - Port đang bị chiếm (Error: Address Already in Use)

Lỗi này xảy ra khi port bạn đang dùng đã có process khác chiếm.

Ví dụ dưới đây dùng port 3000, thay bằng port thực tế của bạn.

**Cách 1:** Dễ nhất (không cần biết về lập trình)
Restart lại máy tính (auto close port nếu như bạn không biết bạn đang làm gì)

**Cách 2:** Tìm và kill process đang dùng port
Windows:

```
# Tìm process đang dùng port (thay 3000 bằng port của bạn)
netstat -ano | findstr :3000

# Kết quả có dạng: TCP  0.0.0.0:3000  ...  LISTENING  1234
# Lấy số PID ở cuối (ví dụ: 1234) rồi kill
taskkill /PID 1234 /F
```

Nếu bạn muốn sử dụng thông qua GUI, tham khảo TCPView tại đây

Mac/Linux:

```
# Thay 3000 bằng port của bạn
lsof -ti:<PORT> | xargs kill -9
```

### #14.2 Lỗi Class Not Found

//todo
Ví dụ build failed

**Nguyên nhân chính:** folder cũ target/classes vẫn chứa class files của package cũ, và class mới chưa được compile vào đó.

**Bước 1:** Clean Java workspace:
Nhấn **Ctrl+Shift+P** → gõ Java: Clean Java Language Server Workspace
→ chọn Restart and delete

Sau đó chạy lại ứng dụng

**Bước 2:** clean compile sẽ xóa folder target và build lại từ đầu.
Reload file pom.xml (reload project)

Chạy lại Maven build để compile:
**./mvnw clean compile**

Hoặc nếu bạn dùng Maven global:
mvn clean compile

### #14.3 Hướng Dẫn Debug Xem Code Chạy
//todo

@hoidanit